EXECUTION-LEVEL TESTPLAN / SM INSTRUCTION GENERATOR

SM 指令发生器 Testplan

SM Instruction Generator Testplan

版本 1.0 • 2026-11-09

执行级计划：参数、初态、步骤、检查点、通过条件、suite、门禁和失败证据。

所有 Generator / SM 动态测试：NOT_RUN；实现门禁尚未打开。

P0 只检查计划静态一致性；不运行发生器、runner、adapter、模型或 DUT。

附 2 张逻辑流程图与可点击章节导航。

阅读导航

1. 目的、测试对象与本次交付
2. 基线、身份与可追踪性
3. 阶段、准入与真实状态
4. 八个配置 Profile 及覆盖规则
5. 数据集、地址对象与初始化
6. 通用执行步骤与观察点
7. Oracle、负例与结果分类
8. 计算、依赖与控制流的具体见证
9. 存储、同步与异步专项步骤
10. TMEM 专属用例与顺序
11. Matrix、scale 与 MALU 接口见证
12. Cluster、Fabric 与系统服务
13. 六类回归 Suite 与准入
14. 固定 Seed、计划运行矩阵与覆盖归属
15. 环境 Schedule、超时与不前进诊断
16. 必须保留的证据与回放包
17. 验收、关闭和状态真值表
18. 文件、实际命令与维护方式
19. 独立审查与实施依赖
20. 全部 106 个执行用例

版本：v1.0；日期：2026-11-09（Asia/Shanghai）。计划编号：SM-INSTR-GEN-TESTPLAN。上位输入：SM 指令发生器 vPlan。本文是用例配置、步骤和回归安排；**106 个功能用例及 873 条计划运行记录均为 NOT_RUN，当前动态配置全部关闭**。

1. 目的、测试对象与本次交付

本 testplan 把 vPlan 的 106 个 SIG 测试点一对一展开为 106 个 TC 用例。每例给出前置条件、参数、初始化、执行顺序、观察点、独立通过/失败判据、覆盖关联、固定 seed、超时策略和证据产物。它承接“验证什么”，落实为“用什么配置、按什么步骤验证，以及如何判断结果”。

对象包括验证端指令／整程序发生器、独立合法性与资源审计、IR/后端适配、参考模型、结果/协议检查器及未来 SM/SoC 环境。指令发生器不是 SM instruction fetch；typed IR、PTX 文本和项目 native 编码分层处理。当前 native 编码、完整 generator、参考执行环境和 SM/SoC 后端仍未实现，本文不提供假运行命令。

本版交付完整用例 JSON、8 个配置 profile、6 类 suite、873 条固定 seed 的计划运行矩阵 CSV/JSON、两张流程图和 DOCX/PDF。可实际运行的 `check_testplan.py` 只验证计划数据完整性，不执行 106 个用例，也不把计划记录改为通过。

2. 基线、身份与可追踪性

源版本固定为 PTX ISA9.4 和已发布的 SM 架构/vPlan。四个源文件及 SHA-256 写入所有主数据的 `source_files`：vPlan testpoints、vPlan features、vPlan 正文、SM 架构正文；源变化必须重新审查用例和覆盖分母。

身份	本版规则	不代表什么
SIG-*	上位 106 个测试点，源 title/phase/feature/CV 保留	不是已执行 case
TC-*	将同一 SIG 前缀替换为 TC，106 个一对一	不因进入多个 suite 增加 case 数量
PLAN-*	suite、case 与 repeat index 唯一组合，873 条	不是执行结果或 coverage 事件
GEN-* / CV-*	沿用 25 个验证特性、75 个覆盖点	不等于全部合法 tuple 已展开或覆盖
PTX 源节	沿用 217 指令/语法说明及 35 特殊寄存器追踪	215 实际指令说明+2 语法，不是 217 个 opcode

每个 TC 的 `source_title/phase/feature_ids/coverage_ids` 与源逐值核对。25 个特性的用例数为：通常每特性 4 例，SYNC/ASYNC 各 5 例，TMEM/MATRIX 各 6 例，共 106 例。P1/P2/P3/P4 分别为 50/27/23/6 例。

3. 阶段、准入与真实状态

阶段	执行对象	准入依赖	当前情况
P0_PLAN_STATIC	JSON/CSV/schema/文档及关联	Python、jsonschema 和仓库文件	可运行计划静态检查，不产生动态覆盖
P1_GENERATOR_UNIT	generator、独立 audit、fixture/期望制品、回放/分布逻辑	unit API 与独立检查实现	NOT_RUN；对应动态 profile 关闭
P2_REFERENCE_PROGRAM	独立语义/数值/资源/内存模型和可选编译流程	模型/工具与适用范围冻结	NOT_RUN；对应动态 profile 关闭
P3_SM_RTL	实际 SM driver、admit/return/检查事件	完整 SM/adapter/monitor 及合法 recipe	NOT_RUN；对应动态 profile 关闭
P4_SOC	多 SM、cluster/DSM/TMA/Fabric 及系统服务	系统服务、故障和公平合同	NOT_RUN；对应动态 profile 关闭

当前用例 `current_status=NOT_RUN`，`current_gate=BLOCKED_MISSING_IMPLEMENTATION`。二者分别表示未执行和准入原因；不能将阻塞解释为失败，也不能将“已列入回归”解释为已运行。将来执行状态必须来自独立 run 结果和真实事件，不能靠修改本计划的常量伪造证据。

4. 八个配置 Profile 及覆盖规则

Profile	阶段与 backend 合同	关键配置	执行开关
TP-P0-STATIC	本目录 metadata validator	只检查源、映射、状态、seed 与矩阵	true，仅静态工具
TP-GEN-BASE	未来 generator/unit API	普通候选、独立 audit、无真实 DUT sink	false
TP-GEN-NEGATIVE	未来 legalizer/unit API	单因素负例、baseline 必须合法、UB 隔离	false
TP-REF-BASE	未来独立 reference API	profile/allowed-outcome 关系检查	false
TP-SM-BASE	未来 SM driver/monitors	实际事件、S0 资源和任意允许完成顺序	false

Profile	阶段与 backend 合同	关键配置	执行开关
TP-SM-STRESS	未来 SM 压力 recipe	接近合法 admission 容量、返回扰动、保留 cleanup	false
TP-SOC-BASE	未来跨 SM 系统服务	cluster/pair/remote owner 与 typed 完成单位	false
TP-SOC-FAULT	未来定义明确的系统 fault 注入	可报告失败、partial/full、不可盲重试副作用	false

`config_profiles.json` 是配置源。exact target、合法形式库 hash、项目 capability hash、R_SM、分配粒度、单 CTA shared 上限等必须实际解析；`null` 表示未冻结，不能当 0 或默认支持。S0 的 warp32、≤64warp/≤8CTA、SMEM256KiB、TMEM 最高 288KiB 是项目配置，不能成为所有 NVIDIA target 的通用规格。

profile 的初始程序预算建议为 body1024 条、cleanup 预留 128 条、CFG32 块、loop 上限 8、call depth4。**这些是待校准的验证预算，不是 ISA 限制。** 具体 case 参数优先，但必须经过独立合法性/资源/预算审查；不得截断必要 witness、稀疏/scale 对象或 cleanup 来适配默认值。超过预算而无评审覆盖时保持 BLOCKED。

5. 数据集、地址对象与初始化

每个 run 使用独立、可重建的 RF/predicate/CC、内存对象、descriptor、barrier 与异步账本。普通输入、scratch、guard/canary、reserved、期望输出和故障侧拷贝分开；不得读取未定义源或依赖旧 run 留下的内容。

对象	初始化要求	结束检查
GPR/predicate/CC	每 thread 定义与 guard 明确；DUT CC 通过合法 producer 建立，不发明可写 CC/FP CSR	值、掩码、CC 和未改写状态正确
global/local/shared	对齐、完整 range、初始化字节与 owner 明确；地址算术检查完整值再编码	canary、write mask、别名和对象生命周期
SMEM	launch 联合预算；不同 CTA 相同 offset 不别名，reserved 不重复/越权计数	所有客户端、RMW/barrier/proxy/远端引用关闭
TMEM	通过合法 allocation 创建；结果地址发布给实际参与者	full/ld-st/peer 责任排除后匹配 dealloc

对象	初始化要求	结束检查
mbarrier	指令 init 布局，不随机填 opaque 字段；phase 与 typed counts 清楚	phase 必要观察、引用关闭、再 inval/复用
matrix/scale/meta	carrier/padding、logical/packed/compressed K、scale 分组与版本明确	不变源 snapshot、collector 失效和结果布局
runtime/CFT/texture 对象	使用该 phase 真实服务或明确 reference fixture	句柄、报告、远端完成及错误合同

P1 可在独立软件 fixture 中声明初始状态，但不能将 fixture 操作当成真实 DUT 寄存器写。只有合法操作序列可进入 P3/P4；SYNTHETIC 日志、损坏侧拷贝和 reference fault injection 必须带来源标记，禁止回流真实 DUT coverage 库。

6. 通用执行步骤与观察点

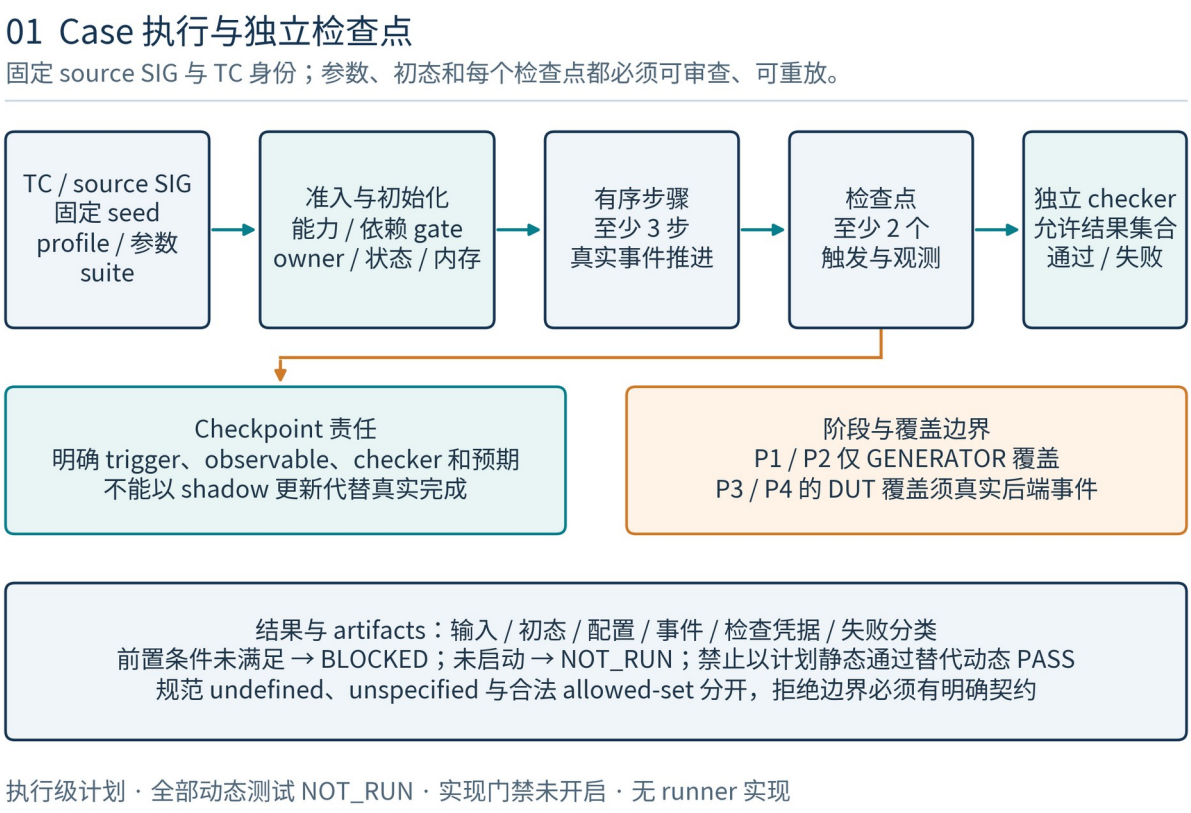


图 1：单用例执行、独立检查和观察点。准入、生成、接收、完成与结果检查分别记录。

每个用例的专属步骤在第 20 章，通用顺序如下：

1. 读取 TC 与 source/hash、suite 和 run seed；检查 profile、backend、oracle、recipe、预算及目标形式是否就绪。

2. 物化本次配置、数据、RNG 子流、环境 schedule 和期望判据；建立全新的对象/责任账本。
3. 生成或载入语义候选，独立审计 type/target/参与者、CFG/依赖、对象边界和全路径资源/HB；负例核对破坏 rule 及预期拒绝层。
4. 通过准入后才调用该 phase 的真实 API；P1/P2 只产生发生器/模型事件，P3/P4 才有实际 SM/SoC 事件。
5. 在 case 指定观察点核对值、允许集合、mask、source capture、typed partial/full、phase/report 和 owner。执行失败立即保存首错，保留必要收尾路径。
6. 结清或正式取消义务，核对完整输出和 canary，生成 checker/coverage/diagnostic；没有 checked 证据不能报告功能 PASS。
7. 按 suite 需要对同一 immutable manifest 回放或缩减，保存原始与缩减包；确认源与数据未漂移。

日志身份至少含 run ID、epoch、CTA owner/generation、warp/thread 或 collective 动态实例、TC/SIG/form ID、operation/fragment、阶段及错误分类。sourceIR→lowered/native→实际实例的 provenance 必须可追踪，不能用源级重复寄存器名假定 DUT 物理 hazard 发生。

7. Oracle、负例与结果分类

情形	检查方式	不能采用的捷径
确定性单例结果	独立 bit/整数/正确舍入模型	host 溢出、普通 double 或 DUT 相同 helper 自证
多个合法结果	审查过的 allowed set/关系/线性化判据	无 data race 就强制唯一结果
approximate 数学	每形式范围、误差和特例	对所有 SFU 统一 epsilon
static 非法/禁用	指定 legalizer/编译层的具体 rule 拒绝	任意非零 exit、缺工具即 PASS
ISA 明示 UB	隔离于正常数值执行	假定硬件 trap 或唯一输出
unspecified/ implementation-specific	已审查 profile/关系可检查，无 oracle 则 BLOCKED	一概等同 UB 或随意给 golden
定义的系统失败	原操作 report/partial/full 与副作用合同	失败当正常零数据、盲重试归约
timeout/不前进	wait graph、work bound 与公平假设诊断	固定周期超时自动等同 DUT 死锁

PASS 要对应本例的正向或明确预期拒绝/故障判据，不等于“没有崩溃”。功能 FAIL、环境 BLOCKED、未启动 NOT_RUN 和 TIMEOUT_UNCLASSIFIED 分别保留；无证据的 skip 不得缩小强制覆

盖分母。弱内存/并发 atomic 可以无 data race 仍有多种合法返回/最终值，只有允许集合被证明为单例才逐值比较。

8. 计算、依赖与控制流的具体见证

整数用例枚举 0/±1、signed min/max、hi/lo/wide 和 2/4word carry 链；clmad 用独立 GF(2)多项式模型，不能复用普通整数乘法 golden。位移 0、width−1、width 及更大值按各形式规则检查，机器相关项有独立 profile。

F32 融合 FMA 可使用 `a=1+2^-23`、`b=1−2^-23`、`c=−1` 的见证：在规定 RNE 且未改变语义的 F32 形式下，融合数学结果为`−2^-46`；先将乘积舍入到 F32 再相加会得到零。该见证用于区分路径，不能把所有 mad 自动等同 FMA，也不能推广到任意 target/profile。NaN/Inf、signed zero、FTZ 和 finite-only 格式分别合法化；有限-only 格式不凭空生成 NaN 编码。

RAW/WAR/WAW、宽返回片段、predicate 和 CC 按最终注入流检查。重命名/优化已消除的 hazard 只能记源级生成，不能记 DUT hazard 检查。CFG 覆盖 1/31/32/33thread 尾 warp、有界 loop 和分歧 call/ret；`.uni` 仅用于对应 guard/target 一致的动态参与者。warp collective 按自身 mask/退出/同参规则，不共享“mask 非零即合法”的宽松条件。

P1 的数值与 CFG 测试检查候选/期望制品和独立 audit，不宣称 SM 执行。REPLAY/REDUCE 用例必须物化全部输入、版本与 schedule；缩减后重新审计合法性并重现原 signature，不能将新 UB 或新 timeout 当原故障缩小结果。

9. 存储、同步与异步专项步骤

地址测试从 typed object 生成完整 range 和别名，不将 local stack、SMEM、TMEM、multimem VA 或 CFT handle 相互裸 cast。普通确定子集、atomic 并发和 weak-memory litmus 分用 oracle；MMIO 访问次数/顺序、volatile、readonly/nc、hint、discard 及 completion-bearing priority 分别检查。

SMEM 使用 8CTA×32KiB 恰好占用 256KiB 池的 conditional witness，同时检查 target 单 CTA 限、RF/warp/reserved；8CTA×64KiB 是池超配负例，9CTA 是 S0 槽数负例。前一个不能仅因等于池容量便无条件 admit。不同 owner 的 offset0、byte/half RMW、packed 逐元素 atomic、关闭 b128 和可选晚到 scrub 分别验证。

mbarrier v0/v1 都为 b64/8B 对齐，但 v1 到达数上限 511，primary/conditional/report 不同。事务期望用合法原子 arrive.expect_tx，或先 expect_tx 并保留 pending arrival 再 arrive，覆盖 complete 早/晚到；不能先放开最后 arrival 再补期望。try_wait false 时 report 未定义，不能消费 payload；True acquire 才建立规定可见性。

classic cp.async、bulk/TMA、wgmma、tcgen05 full/A-read/ld-st 和 Fabric local-read/full 分账。数据 bank commit、collector 失效生效、proxy 发布和 barrier 对象原子更新 ack 是不同观察点；FIFO 接收不等于对象更新或目的可消费。TMA report 按规定 pattern/采样，不是通用 ECC。

10. TMEM 专属用例与顺序

TMEM 用例源于 `SIG-TMEM-001..006`，对应 `TC-TMEM-001..006`；第 20 章给出全部配置与步骤，matrix/async/sync 另外关联其相关义务。

TC	重点	必须观察的终点
TC-TMEM-001	分配顺序、exclusive/非 exclusive、512/576target、permit	匹配范围/属性；relinquish 不释放 allocation
TC-TMEM-002	地址类型、warp collective、quarter lane、CTA group2	同参/参与与 owner，非法形式停在 preflight
TC-TMEM-003	ld/st pack/unpack、ld.red、ld.spcompress 布局	独立 lane/column→寄存器/metadata 结果
TC-TMEM-004	cp/shift/ld/st 并发、多片 wait	真实 TMEM/RF 提交；首片和 A-read 不能代 full
TC-TMEM-005	在途访问、wait 后 dealloc 及退出重用	所有本地/peer 访问排除、迟到 generation 隔离
TC-TMEM-006	even/odd 跨两个 endpoint 的双端资源与 parent/child	双边完整责任结清，不半发布/半释放

统一程序链为合法 alloc→发布地址→按 layout 访问/搬移/计算→各自完成与可见性→排除两端访问→匹配 dealloc→退出。permit、allocation、MCS full、restricted A-read、ld/st 及 SMEM owner 不合成一个 done 位。group2 遵守 S0 的不同可互通 SM 放置与同 kernel group 一致性；不能将 exclusive CTA1 扩成全 pair 互斥并阻断 cluster 进度。

11. Matrix、scale 与 MALU 接口见证

A/B 格式、C/D 格式、scale_A/B 格式、BLOCK16/32、独立 offset、Cshift 分别配置，但最终必须通过 native tuple 与项目接口联合合法性。raw Ti16 关闭，decoded 项目协议另列；finite-only 低位格式、稀疏 metadata 与 scale 合法范围按具体形式检查。

PROJECT_MALU_ADAPTER 可分开采用两类见证，不能无证据把二者组合成同一 native tuple：

- 未启用 AB block scale 的合法 F16 输入，K=4、所有 A/B=1、C=8、合法 Cshift=3；精确项目数学结果为 5。`enable-input-d=false` 时另测旧 D 为 NaN 却不读取/参与，正式零 C 使结果为 4。
- AB scale 启用时选择已批准格式组合，K=64、Cshift=0，分别 BLOCK16/32 及最早块边界切 beat；例如 BLOCK32 下 A/B offset15/31，第一 beat 只能先取 1 个元素，不能因 LANES=4 跨 B 块。后续 K/offset 不随 stall 重起。

两个命名见证已分别落入 TC-MATRIX-002 与 TC-MATRIX-006 的 canonical 参数；offset 见证 A/B 各有 3 项 unity scale，分别覆盖 17/32/15 与 1/32/31 元素。它们仍须在对应目标/profile 明确允许时才执行。C 缩放在指数/累加路径表达，不预量化再缩放；C 每 dot 只加入一次。稀疏 logical K 与 compressed K、packing 和 scale 分组不同；结构零与 Inf/NaN 按所选 profile 处理，不一概跳过。 TC-MATRIX-001 显式枚举原生 i8/ti16 与 project INT8/INT4/B1、decoded Ti16、F32/F64 扩展，能力和编码分别门禁；TC-MATRIX-003 按 TF32 1:2、批准 2:4 与 FP4 成对 4:8 分型，不从六种数学 2-of-4 组合推断全部原生 metadata。整数/B1 只用合法 S32 和整数溢出判据，不注入浮点 NaN/Inf。

复用压力覆盖高低精度/scale/finalizer、operand 返回和 TMEM 写回。MALU 仅 dot，普通 FP FMA/SFU 另域。Rubin 峰值比例不是抽样权重或本 testplan 时延判据，现有串行 MALU 检查也不证明完整 matrix 前端已实现。

12. Cluster、Fabric 与系统服务

cluster/DSM 的 target、rank 映射、joint admission、peer 存活和 remote owner 引用是前置条件。工作取消只取消未启动 cluster；16B opaque response 与 mbarrier 完成分别检查，观察失败后不非法重试。griddepcontrol 需要 runtime 服务，不当成 CTA barrier。

Fabric 的 get/pullred 按 bytes，put/red 按 16B units；8B counter 需 256B 对齐且与数据不重叠。issue/submit/期望登记/primary wait/report 必须完整，fabric.wait 仅证明相应 local SMEM 源读结

束。CFT=fabric、local SMEM=async、barrier=generic 按规范方向衔接；成功不清同 phase 旧错误，partial counter 不证明 full，失败不自动取消已经发生的远端副作用。

texture/surface 当前关闭；P1 可验证 capability 拒绝，P4 正向需要完整采样/格式/边界/查询服务，不能用 global load 替代。stack、setmaxnreg、debug、nanosleep 和 sreg 按各自 ABI/稳定性处理；没有通用 FP CSR。per-CTA global 依赖 directive/driver，size 可能为 0，不能解释为 SMEM。

13. 六类回归 Suite 与准入

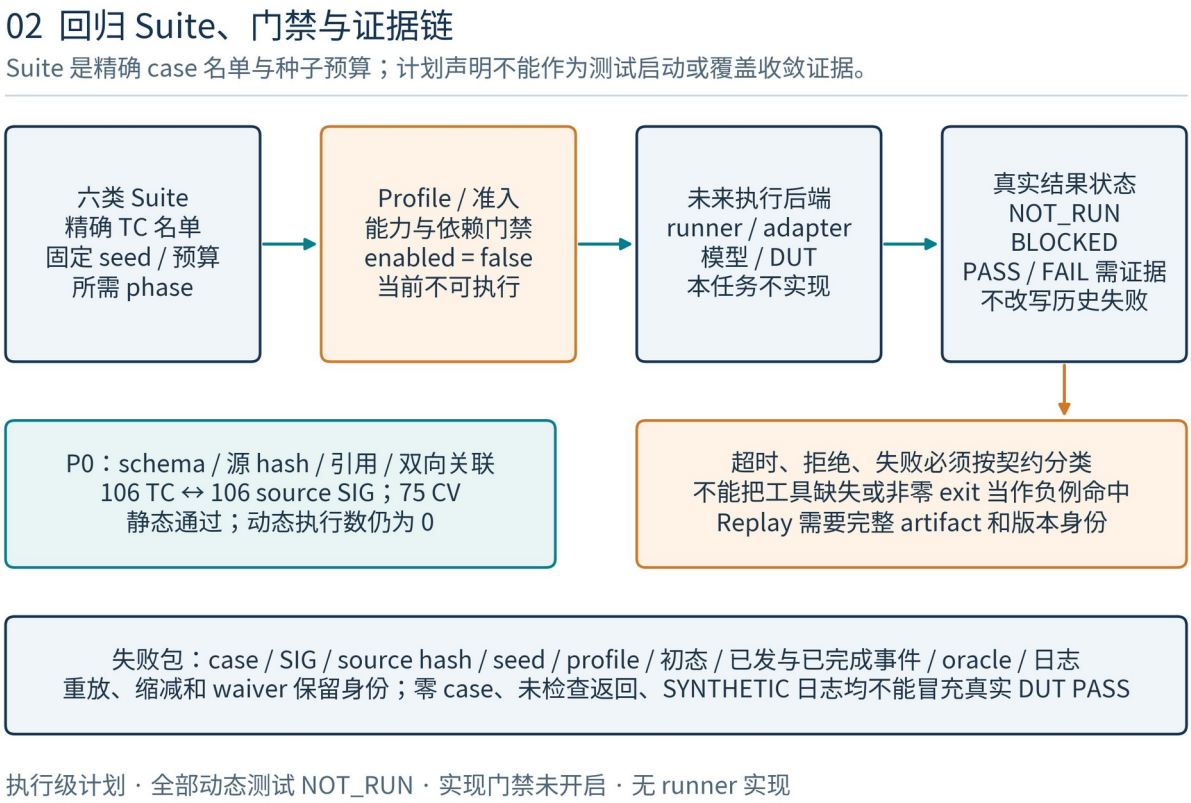


图 2：回归分组、阶段门禁、失败证据与真实状态。计划 run 不是执行结果。

Suite	用例数	每例独立基线数	计划 run 数	目的
SMOKE	25	1	25	每 feature 第一个 P1 用例，最小流水/审计入口
DIRECTED	106	1	106	全部源测试点的确定边界与协议见证
NEGATIVE	52	2	104	已声明单因素拒绝、错误分类或项目保护路径
RANDOM	49	8	392	只随机 case 明确允许、审计通过的参数维度

Suite	用例数	每例独立基线数	计划 run 数	目的
STRESS	34	4	136	case/phase 内的合法压力、乱序和收尾；P1/P2 不算 DUT
REPLAY	55	2	110	immutable manifest、artifact 一致性或原故障重现

各 TC 可属于多个 suite，106 是唯一用例数，873 是展开后的计划记录数。suite 不改变 case phase、profile 或 CV scope；全部 `profile_override=null`。STRESS 仅使用本例已定义并审计的压力 recipe，不能临时猜 FIFO 深度/固定时延或超配资源；recipe 缺失则该 run BLOCKED。

执行推进顺序是 P0→P1/P2 entry gates→对应 P3/P4 环境 smoke→定向→随机→压力→triage/closure。表中 SMOKE 为 P1 入口集合，不代替 P3/P4 真实 adapter 的 smoke recipe。CSV/JSON 行序仅为确定性枚举，不是启动顺序；未来调度按 phase 和显式依赖 DAG 准入。TC-REGRESS-004 是不可重入的只读闭合观察器，仅消费外层已完成的 22 个 P3 叶子证据，不调用自身或任何 case；缺必需证据时 BLOCKED，23 个源 P3 用例总数不变。低阶段成功不能开放缺实现的高阶段；前置 gate 失败时停止升级，保留被阻塞列表与原因。

REPLAY 表中的“2”是两个独立基线实验，**不是对同一故障包换 seed 回放两次**。每个基线完成后，回放使用原始完整 manifest/数据/schedule/seed，不再派生新 seed；对确定性结果比对一致值，对允许多结果的程序仍用合法关系，保故障缩减要求原 signature 成立。

14. 固定 Seed、计划运行矩阵与覆盖归属

按源 `testpoints` 数组 0-based 序号 i 设置：

$$seed_{case} = 0x534D0000 + i$$

固定 suite index 为 SMOKE0、DIRECTED1、NEGATIVE2、RANDOM3、STRESS4、REPLAY5，repeat_index 从 0 开始：

$$seed_{run} = seed_{case} + suiteIndex \times 0x100000 + repeatIndex \times 0x10000$$

本版 873 个 seed 均为唯一 uint32，无溢出。run ID 为 `PLAN-<suite>-<TC>-R<两位 repeat>`，case、源 SIG、phase、profile、timeout、seed 与当前状态逐列写入 run_matrix.csv，同内容保留在 `planned_runs.json`。

seed 只决定输入的一部分；manifest 仍需所有源/tool/schema/backend/capability/profile/bin registry、初始化、RNG 子流、rbits 和环境 schedule。不得因相同 seed 就省略不同 target、工具或数据 hash。重复执行同一真实证据不增加覆盖；P1/P2 仅 GENERATOR，DUT_SCENARIO 只由 P3/P4 真实事件触发。

计划记录、候选、合法化、driver 接受、DUT admitted、typed partial/full 和 checked 分账。特别是 GENERATOR/CHECKED 是独立 audit 结论，不是 DUT 结果已检查。75 个 CV 关联完整不等于全部 bins 已展开；未冻结的必需合法形式保持 UNRESOLVED 并阻止相应关闭。

15. 环境 Schedule、超时与不前进诊断

各 run 物化独立 opcode/operand/CFG/resource/timing/fault 随机子流。允许的返回置换只作用于可并发责任；ready 阻塞期间 payload 稳定，旧 generation 不能改新 owner。全局 worker 顺序、wall clock 和未记录 PRNG 调用不能影响同 manifest 刺激。

Timeout 策略	初始 host 预算建议	其他边界	耗尽后处理
GEN_BOUNDED	300 秒	每候选最多 256 次约束尝试；body/cleanup 预算独立	UNSAT 有 rule 证据；否则 TIMEOUT_UNCLASSIFIED
REFERENCE_BOUNDED	900 秒	初始 1,000,000 semantic steps，按 CFG/dot 工作量审查	保存最后事件、模型和 wait graph，诊断后分类
SM_PROGRESS	1800 秒	cycle/progress 阈值须由真实 workload/backend 合同解析	采集实际进度/外部 assumption，不自动判 DUT 死锁
SOC_PROGRESS	3600 秒	远端服务、report 与 cancel/drain 阈值需冻结	保留 endpoint/owner/phase/副作用证据，禁止盲重试

这些数字是待校准的验证 host 预算，不是 NVIDIA 硬件时延保证。cycle 阈值当前为 null，相关 run 不能以此准入。超过预算但无诊断不能 PASS；缺外部服务与 DUT 内部环等待应通过 wait graph、已接受责任及公平 assumption 分别归因。正常 drain 保留必要 wait/commit/load/free，abort/reset 需正式取消/隔离合同。

16. 必须保留的证据与回放包

每个真实 run 目录将以 run ID 标识，至少包含：完整 manifest、effective config 与源 hash、program/IR 与可用 backend 产物、初始化字节/descriptor、环境 schedule、独立 audit、期望/允许集合、实际事件、checker/coverage、状态与首错 diagnostic。缺失 backend 时目录只能记录准入 BLOCKED，不伪造成功代码或波形。

失败额外保存首错前后窗口、source→native provenance、live owner/obligation、wait graph、barrier phase/tx-unit、服务 assumption、外部已 commit/可取消副作用，以及原始与缩减包。重跑成功不得覆盖首错；重复失败要关联同 signature，不生成多个虚假独立 coverage 命中。

回放先核对全部输入 hash 和工具，再恢复独立初态并按原 artifact 运行。缩减每次重新独立审计 def-use、CFG、参与者、资源/HB、profile 和原 failure predicate，不能通过删除 wait/peer 或创造 UB 保留一个新的 timeout。未能稳定复现标 NONDETERMINISTIC 并保留环境记录。

17. 验收、关闭和状态真值表

实际情况	结果归类	允许的证据说明
只有计划或 P0 校验	NOT_RUN；计划静态结果可 PASS	不宣称 106 动态测试通过
generator/oracle/backend/capability 缺失	BLOCKED	记录缺失项和 phase，不静默 skip
正向结果及所有必需义务/判据匹配	PASS	有非空实际 checked、完整 manifest 及覆盖证据
指定负例在指定层按具体 rule 拒绝	该负例 PASS	不计该功能正向 DUT 覆盖
定义的系统 fault 按合同完成/报告	该 fault case 按具体判据判定	失败副作用与 partial/full 完整记录
数值/协议/身份/副作用不匹配	FAIL	保留首错/原始输入和可重现 signature
超时且根因不明	TIMEOUT_UNCLASSIFIED	不自动当 DUT FAIL 或重跑后 PASS
零 case/零 checked/未审查 waiver	未满足关闭条件	不用旧 MALU 数字或缩分母补齐

退出只针对冻结的 enabled case/variant/bin 范围，全部 mandatory 可达项需实际证据。P3 的 23 个源 case 与 P4 的 6 个系统 case 分 gate 关闭，不能借 P3 结果声称 SoC 完成。期望工具拒绝/静态关闭可以作为相应负例结果，不能证明原生指令已执行。

18. 文件、实际命令与维护方式

文件	用途
SM_INSTR_GEN_TESTPLAN.md / DOCX / PDF	本执行计划及阅读版
testcases.json	106 个完整 case，SIG 一对一与具体步骤/判据
config_profiles.json	8 配置，明确动态开关和未解析项
campaigns.json	6suite、名单、repeat、准入、stop 和超时策略
planned_runs.json / run_matrix.csv	873 条相同的计划记录与固定 seed
check_testplan.py / schemas/	实际可运行的 P0 数据与关联检查
reports/	P0、自检、阅读版 QA 和独立交付检查
research/	分工细化稿与技术审查；不是执行结果

从仓库根运行实际计划静态校验：

```
python3 verif/core/testplan/sm_instr_gen/check_testplan.py
```

该工具验证 schema、source hash、106/75 关联、8profile/6suite、seed 推导、873 条展开、CSV 逐值和当前状态。它不调用 generator、仿真器或 SM。阅读版构建见 BUILDING.md，本目录未实现 runner；不得把未来接口合同写成当前可运行命令。

源变化后需重新审查 TC/配置/矩阵/覆盖和证据兼容性；更新 case 或 suite 必须重新生成并校验 `planned_runs.plan_inputs` 及 CSV。维护时保留稳定 TC/SIG identity，不按显示排序重新分配 seed，不把某 suite 的排除项伪装为已通过。

19. 独立审查与实施依赖

本 testplan 保留 vPlan 的阶段和数值/顺序/资源合同，不扩大 ISA 合法组合或现有 RTL 能力。普通 ALU/FP 与 dot-only MALU 分开；native 矩阵与 PROJECT_EXACT 各有 profile，Ti16 raw 缺口不猜编码；scope/proxy、mbarrier report、TMA/Fabric 完成单位和全程序 cleanup 分别验证。

三名分工 agent 细化 ISA/系统、计算/控制/回放、存储/异步/TMEM 资源 case；主 agent 细化环境/覆盖/回归、配置和矩阵并合并；工具 agent 负责 schema/静态检查、两图与阅读版。原始分工稿是 authoring provenance，最终 `testcases.json` 是审查和执行合同源；若发现差异应记录合并修正。

三轮检查分别核对源与矩阵、技术判据/实施门禁、产物和阅读质量。P0 通过、图表可读或所有 ID 齐全均不代表发生器/模型/SM/SoC 已实现或动态测试已执行。

20. 全部 106 个执行用例

以下按源顺序分特性展开。每例包含固定 seed/profile/phase/suite、前置与初态、参数、步骤、观察点、通过/失败和预期产物。所有数值和参数仅在其 target/recipe 合法且已解析时生效，未知门禁不可绕过；预期产物名表示后续真实执行必须生成，不表示这些结果文件当前已存在。

20.1 GEN-ISA-01：规范、target 与项目 capability 合法性

TC-ISA-001：全部目录及别名身份

TC-ISA-001 项目	执行内容
源与阶段	SIG-ISA-001；P1_GENERATOR_UNIT；CV-ISA-001
配置与 seed	TP-GEN-BASE；fixed_seed=0x534d0000（1397555200）；GEN_BOUNDED
回归与当前状态	SMOKE, DIRECTED, NEGATIVE；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. 固定并核对源 vPlan/testpoint、PTX9.4 inventory、target/capability、规则及模型版本 hash；fixed_seed 由主 campaign 在运行前赋值。 2. 发生器、unit harness 和独立合法性/结构审计器必须可用；共享 source 身份不等于共享待测约束函数。 3. 执行本 case 前冻结 timeout 预算；缺预算、模型、工具或服务时保持 BLOCKED，不以空执行关闭 case。
参数 1	timeout_budget_ref = campaign 配置中与 timeout_policy 对应的有限预算；不在本 case 猜测 SM 固定周期。
参数 2	instruction_descriptions = 215
参数 3	control_syntax_descriptions = 2
参数 4	sreg_descriptions = 35
参数 5	directive_descriptions = 34
参数 6	identity_key = source section, not mnemonic
参数 7	target_snapshot_count_min = 2
参数 8	mutations = [{"field":"one source section identity","action":"duplicate one record while omitting a different record","expected_layer":"INDEPENDENT_INVENTORY_AUDIT"}]

TC-ISA-001 项目	执行内容
初始化	1. 固定源目录和两份独立 target 快照只读；输出索引、计数和 mapping 账本为空。 2. {}、@、sreg、directive 均有独立 entry_kind，不占普通执行 opcode 计数。
执行步骤	1. 载入两份源 inventory 并校验 SHA，按 source section 遍历 217 个说明叶子和 35 个 sreg 说明；另索引 34 个 directive。 2. 生成每节的 identity 与别名/同名多类型记录；保留 mov 两说明节和同节多个名称，不以 mnemonic 去重。 3. 用独立提取的 source 集合逐项核对 section、名称数组、物理/印刷页码和 entry_kind，再按各 target 筛选可用形式。 4. 在隔离索引副本中只制造一个身份替换错误（重复一节并遗漏一节），确认集合审计报错；不将该副本送发生器正向流。
观察点	1. 基线索引为 217 唯一指令/语法说明，其中 215 instruction 与 2 syntax，另 35 sreg；不是 opcode 或 tuple 数。 2. 同名/别名记录未丢失；变异副本定位到重复与缺失的具体 section 而不是仅总数相等就通过。
通过条件	1. 所有源身份/页码匹配且 scope 计数独立；target 快照不被隐式继承。 2. 基线可审计，身份替换副本被独立审计拒绝并保留差异。 3. 实际执行量、检查量及所需覆盖非空，所有必需检查满足；本 case 当前 NOT_RUN 不因计划/静态检查而变为 PASS。
失败条件	1. 源 hash 变化后继续合并旧覆盖，或只比较总数漏掉身份替换。 2. 语法/sreg/directive 被当可执行 opcode，或同名节被去重。
预期产物	1. manifest.json 2. case_result.json 3. coverage_delta.json 4. source_identity_diff.json 5. identity_index.json 6. target_snapshots.json

TC-ISA-002：target 与 capability 单因素变异

TC-ISA-002 项目	执行内容
源与阶段	SIG-ISA-002；P1_GENERATOR_UNIT；CV-ISA-002
配置与 seed	TP-GEN-BASE；fixed_seed=0x534d0001（1397555201）；GEN_BOUNDED
回归与当前状态	DIRECTED, NEGATIVE；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION

TC-ISA-002 项目	执行内容
前置条件	1. 固定并核对源 vPlan/testpoint、PTX9.4 inventory、target/capability、规则及模型版本 hash；fixed_seed 由主 campaign 在运行前赋值。 2. 发生器、unit harness 和独立合法性/结构审计器必须可用；共享 source 身份不等于共享待测约束函数。 3. 执行本 case 前冻结 timeout 预算；缺预算、模型、工具或服务时保持 BLOCKED，不以空执行关闭 case。
参数 1	timeout_budget_ref = campaign 配置中与 timeout_policy 对应的有限预算；不在本 case 猜测 SM 固定周期。
参数 2	baseline_source = independently audited legal-form registry
参数 3	mutation_cardinality = 1
参数 4	mutation_axes = ["exact_target","family_membership","one_operand_type","one_modifier"]
参数 5	target_examples = 仅使用已核验 recipe 展示 sm_90a WGMMA 及 sm_107 相关 a/f 门限；不是所有形式用同一 target。
参数 6	unknown_rule_expected = BLOCKED_RULE_UNRESOLVED
初始化	1. 每个 baseline 有独立 LEGAL verdict 及 source/target 证据；baseline IR 冻结。 2. project capability 状态单独记录，不把 target 合法等同 project 已实现。
执行步骤	1. 为每个 target 边界载入一个已独立核验合法 baseline，先确认原样可被相应合法化路径接受。 2. 复制 baseline 并只改变一个 target/family/type/modifier 字段；保留其余 IR、对象、参与者及 seed 子流不变。 3. 用独立规则表先预测被违反的规则与拒绝层，再提交 baseline 与 mutant；未知规则单列 BLOCKED。 4. 对照结构差分确认恰一字段改变，并检查拒绝原因只对应预期约束；恢复 baseline 再次审计以排除全拒绝器。
观察点	1. 基线仍合法且有实际 accept 记录；规范 target 和 project capability 分别出结论。 2. mutant 差分恰一字段，拒绝 rule/layer 匹配；unknown rule 不被当 accept 或负例 PASS。
通过条件	1. 全部选择的 baseline 接受，单因素 mutant 按独立规则判断且拒绝可解释。 2. 实际执行量、检查量及所需覆盖非空，所有必需检查满足；本 case 当前 NOT_RUN 不因计划/静态检查而变为 PASS。

TC-ISA-002 项目	执行内容
失败条件	1. 多个无关字段被静默重写，或只按 sm 数字大小继承 a/f 能力。 2. 基线和 mutant 全部拒绝却报负例通过，或缺规则自动排除。
预期产物	1. manifest.json 2. case_result.json 3. coverage_delta.json 4. baseline_mutant_diff.json 5. legality_verdicts.json 6. rule_provenance.json

TC-ISA-003：关闭能力与 lowering 证据

TC-ISA-003 项目	执行内容
源与阶段	SIG-ISA-003；P2_REFERENCE_PROGRAM；CV-ISA-001
配置与 seed	TP-REF-BASE；fixed_seed=0x534d0002（1397555202）；REFERENCE_BOUNDED
回归与当前状态	DIRECTED, NEGATIVE；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. 固定并核对源 vPlan/testpoint、PTX9.4 inventory、target/capability、规则及模型版本 hash；fixed_seed 由主 campaign 在运行前赋值。 2. 所用独立参考解释器/允许结果模型及 lowering 或服务 reference route 必须可用；P2 结果不计真实 SM 执行。 3. 执行本 case 前冻结 timeout 预算；缺预算、模型、工具或服务时保持 BLOCKED，不以空执行关闭 case。 4. 本 case 至少一条可执行且有证明的 reference lowering 路线已注册；否则整体保持 BLOCKED。
参数 1	timeout_budget_ref = campaign 配置中与 timeout_policy 对应的有限预算；不在本 case 猜测 SM 固定周期。
参数 2	route_trials = ["native_disabled","lowering_certificate_absent","service_absent","separately_enabled_proven_lowering"]
参数 3	mutation_cardinality = 1
参数 4	disabled_route_diagnostic = CAPABILITY_DISABLED
参数 5	lowering_proof = rule/version/hash and independent observable-relation check
初始化	1. 规范合法 baseline 已确认，project native/lowering/service 开关初始关闭。 2. 功能 capability 与 lowering 证明登记独立，外部 service 没有默认可用假设。

TC-ISA-003 项目	执行内容
执行步骤	1. 载入一项规范合法但项目关闭的 baseline，保存原始语义与输入状态。 2. 逐个尝试 native、lowering、service 路线，每次仅改变该路线选择，确认关闭/缺证书/缺服务被对应层拒绝而非 no-op。 3. 在独立 future-enabled profile 中仅登记已证明 lowering 及其版本；若没有证书则保留该 trial BLOCKED，不临时编造 helper。 4. 对可用证明路线执行原语义和 lowered 程序参考模型，比较可见寄存器、guard、CC 和副作用；保存 relation 与证明 hash。
观察点	1. 关闭路径没有任何 functional issue；每次路线选择变更可重放且拒绝原因明确。 2. 启用 lowering 只有同时具备 capability 和有效证书时产生，reference 观测关系满足声明。
通过条件	1. 未实现路径明确拒绝；至少一项实际可用的证明路线完成独立 relation 检查才能关闭本 case。 2. 实际执行量、检查量及所需覆盖非空，所有必需检查满足；本 case 当前 NOT_RUN 不因计划/静态检查而变为 PASS。
失败条件	1. 无证明 fallback、静默 no-op，或 service 缺失被当成功。 2. 仅成功输出 lowered 文本就计执行等价，或缺证书仍关闭 case。
预期产物	1. manifest.json 2. case_result.json 3. coverage_delta.json 4. route_decisions.json 5. lowering_certificate_manifest.json 6. reference_relation.json

TC-ISA-004：入口一致性

TC-ISA-004 项目	执行内容
源与阶段	SIG-ISA-004；P3_SM_RTL；CV-ISA-003
配置与 seed	TP-SM-BASE；fixed_seed=0x534d0003（1397555203）；SM_PROGRESS
回归与当前状态	DIRECTED, NEGATIVE, STRESS；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION

TC-ISA-004 项目	执行内容
前置条件	1. 固定并核对源 vPlan/testpoint、PTX9.4 inventory、target/capability、规则及模型版本 hash；fixed_seed 由主 campaign 在运行前赋值。 2. 项目接口版本、SM RTL/driver/monitor 及本 case 启用服务必须可用；native 编码未冻结时不得发送伪二进制。 3. 执行本 case 前冻结 timeout 预算；缺预算、模型、工具或服务时保持 BLOCKED，不以空执行关闭 case。
参数 1	timeout_budget_ref = campaign 配置中与 timeout_policy 对应的有限预算；不在本 case 猜测 SM 固定周期。
参数 2	positive_route = selected enabled project instruction/IR adapter route
参数 3	negative_change = one contract-defined invalid project route token
参数 4	negative_class = NONISA_ROBUSTNESS
参数 5	expected_rejection_layer = PROJECT_ADAPTER
参数 6	encoding = only frozen backend contract; no guessed native bits
初始化	1. 真实 adapter 状态清空，启用 route 和合法输入对象已登记。 2. 负例 fault ID 及其接收/拒绝/错误 ack 合同已书面定义。
执行步骤	1. 生成并独立审计合法 baseline，绑定实例 ID/route/owner 后送真实 driver，记录 valid/ready 和 accept 事件。 2. 等待对应真实完成/检查事件，确认 baseline 副作用与参考值一致；不以 preflight 通过代替 issue。 3. 复制 baseline，仅通过项目 fault API 将 route token 改为合同定义非法值，其他 payload/握手稳定性保持合法。 4. 送 adapter 隔离负例并观察规定拒绝/错误事件、无非预期写入及资源收尾；分别统计 preflight、accept、reject、complete 和 checked。
观察点	1. 正向有真实 accept、complete 及 checked 证据，动态身份/route 不串。 2. 负例在项目合同层被处理且无不允许副作用；preflight 拒绝从未计 DUT 执行。
通过条件	1. 正向真实执行关系正确，负例只按已定义项目错误合同判定。 2. 实际执行量、检查量及所需覆盖非空，所有必需检查满足；本 case 当前 NOT_RUN 不因计划/静态检查而变为 PASS。
失败条件	1. 无真实 issue 事件却报告执行覆盖，或合法/非法实例被错误路由。 2. 非法 route 导致非预期写入/重复 ack/责任泄漏，或被宣称 PTX UB trap 一致性。

TC-ISA-004 项目	执行内容
预期产物	1. manifest.json 2. case_result.json 3. coverage_delta.json 4. adapter_event_trace.jsonl 5. route_scoreboard.json 6. fault_contract.json

20.2 GEN-IR-01：语义 IR、序列化与 adapter 边界

TC-IR-001：IR 完整往返

TC-IR-001 项目	执行内容
源与阶段	SIG-IR-001；P1_GENERATOR_UNIT；CV-IR-001
配置与 seed	TP-GEN-BASE；fixed_seed=0x534d0004（1397555204）；GEN_BOUNDED
回归与当前状态	SMOKE, DIRECTED, REPLAY；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. 固定并核对源 vPlan/testpoint、PTX9.4 inventory、target/capability、规则及模型版本 hash；fixed_seed 由主 campaign 在运行前赋值。 2. 发生器、unit harness 和独立合法性/结构审计器必须可用；共享 source 身份不等于共享待测约束函数。 3. 执行本 case 前冻结 timeout 预算；缺预算、模型、工具或服务时保持 BLOCKED，不以空执行关闭 case。
参数 1	timeout_budget_ref = campaign 配置中与 timeout_policy 对应的有限预算；不在本 case 猜测 SM 固定周期。
参数 2	operand_kinds = ["register","immediate","packed_bits","symbolic_address","descriptor","metadata"]
参数 3	semantic_fields = ["width","signedness","byte_order","mask","owner_generation","numerical_profile","source_identity","completion_obligation"]
参数 4	json_order = irrelevant
参数 5	raw_ti16_encoding = not synthesized
初始化	1. schema 及 semantic IR 版本冻结；symbolic object/owner 可由独立 registry 解析。 2. 语义基线 AST 包含小宽度、跨 word 及向量 operand，无 native 编码假设。

TC-IR-001 项目	执行内容
执行步骤	1. 按六类 operand 构造合法 typed AST，并显式设置 mask、owner/generation、profile 与完成义务。 2. 使用发生器序列化器输出 IR，再由独立 schema 解码器恢复 AST。 3. 仅重排无语义 JSON 键次序，重复 decode 并比较规范化语义；同时检查 packed 载体和地址引用没有转成普通整数。 4. 选择一字段（如 signedness）在隔离副本变异，确认语义比较定位差异而非按字符串顺序决定相等。
观察点	1. 完整往返后所有语义字段与引用身份相等；无隐式 host byte order 依赖。 2. 键重排不改变语义，单字段变异被比较器识别。
通过条件	1. 每类 operand 实际往返并通过独立语义比较；变异对照不被错误视为相等。 2. 实际执行量、检查量及所需覆盖非空，所有必需检查满足；本 case 当前 NOT_RUN 不因计划/静态检查而变为 PASS。
失败条件	1. 位宽、符号、mask、owner 或完成义务丢失/默认填充改变语义。 2. 仅比较 JSON 字符串或复用同一序列化实现作为独立 oracle。
预期产物	1. manifest.json 2. case_result.json 3. coverage_delta.json 4. input_ast.json 5. serialized_ir.json 6. decoded_ast.json 7. semantic_diff.json

TC-IR-002：编码后端门禁

TC-IR-002 项目	执行内容
源与阶段	SIG-IR-002；P1_GENERATOR_UNIT；CV-IR-002
配置与 seed	TP-GEN-BASE；fixed_seed=0x534d0005（1397555205）；GEN_BOUNDED
回归与当前状态	DIRECTED, NEGATIVE；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. 固定并核对源 vPlan/testpoint、PTX9.4 inventory、target/capability、规则及模型版本 hash；fixed_seed 由主 campaign 在运行前赋值。 2. 发生器、unit harness 和独立合法性/结构审计器必须可用；共享 source 身份不等于共享待测约束函数。 3. 执行本 case 前冻结 timeout 预算；缺预算、模型、工具或服务时保持 BLOCKED，不以空执行关闭 case。

TC-IR-002 项目	执行内容
参数 1	timeout_budget_ref = campaign 配置中与 timeout_policy 对应的有限预算；不在本 case 猜测 SM 固定周期。
参数 2	output_modes = ["semantic_ir","native_requested"]
参数 3	native_backend_state = UNFROZEN_OR_UNAVAILABLE
参数 4	mutations = [{"field":"schema_version","kind":"unknown_version"}, {"field":"reserved_field","kind":"one_invalid_reserved_value"}]
参数 5	mutation_cardinality_per_trial = 1
参数 6	reject_layer = SCHEMA_OR_BACKEND_GATE
初始化	1. semantic IR schema 可用，native 后端标记未冻结/不可用；native 输出 buffer 为空。 2. 保留字段的约束来自实际已冻结 IR schema，若无定义该 trial BLOCKED。
执行步骤	1. 提交合法 semantic IR 输出请求，校验可产生带版本和 source identity 的 IR 制品。 2. 保持语义不变只将输出模式改为 native，检查明确 backend gate 拒绝且 fetch 流无任何 payload。 3. 从原始 IR 分别构造未知 schema 版本和单个保留字段非法值的隔离变异，各 trial 只变一个字段。 4. 用独立 schema/backend 审计核对拒绝层及版本记录，并确认未把 PTX 文本 UTF-8 或 IR JSON 字节作为机器指令输出。
观察点	1. IR 成功输出与 native 被阻塞的结果分开，native buffer 和 DUT issue 计数保持零。 2. 未知版本/保留位变异被各自规则拒绝，合法 IR 基线不受影响。
通过条件	1. semantic 输出符合 schema，native 未冻结时明确拒绝，无伪二进制或隐式 fallback。 2. 实际执行量、检查量及所需覆盖非空，所有必需检查满足；本 case 当前 NOT_RUN 不因计划/静态检查而变为 PASS。
失败条件	1. 返回任意文本字节并标 native 成功，或未知 schema 被静默接受。 2. 同时改变多个字段导致拒绝原因不可归属。
预期产物	1. manifest.json 2. case_result.json 3. coverage_delta.json 4. backend_gate.json 5. schema_negative_diff.json 6. output_artifact_manifest.json

TC-IR-003：降低保持

TC-IR-003 项目	执行内容
源与阶段	SIG-IR-003；P2_REFERENCE_PROGRAM；CV-IR-001
配置与 seed	TP-REF-BASE；fixed_seed=0x534d0006（1397555206）；REFERENCE_BOUNDED
回归与当前状态	DIRECTED, NEGATIVE；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. 固定并核对源 vPlan/testpoint、PTX9.4 inventory、target/capability、规则及模型版本 hash；fixed_seed 由主 campaign 在运行前赋值。 2. 所用独立参考解释器/允许结果模型及 lowering 或服务 reference route 必须可用；P2 结果不计真实 SM 执行。 3. 执行本 case 前冻结 timeout 预算；缺预算、模型、工具或服务时保持 BLOCKED，不以空执行关闭 case。
参数 1	timeout_budget_ref = campaign 配置中与 timeout_policy 对应的有限预算；不在本 case 猜测 SM 固定周期。
参数 2	lowering_set = only reviewed rules with observable-relation certificates
参数 3	observable_state = ["destinations","predicate","CC","memory_side_effects","thread_control"]
参数 4	temporary_policy = declared dead temporaries or saved/restored live state
参数 5	negative_change = one missing certificate or one declared live CC clobber per isolated trial
初始化	1. 原始语义和 lowered 程序解释器可独立实现；输入状态有活跃 predicate/CC 和 guard 关闭分支。 2. helper 临时寄存器集合、可见状态及允许 clobber 已经声明。
执行步骤	1. 选择一条有证书 lowering，在相同初始状态运行原语义 reference 并保存可见终态/事件。 2. 运行对应 lowered 程序 reference，记录 helper 临时寄存器和 CC 读写轨迹。 3. 按声明观测 relation 比较终态与 side effects，忽略仅被证明 dead 的临时状态；检查 guard/rounding 未改变。 4. 在隔离 trial 删除该证书或只引入一次 live CC clobber，确认分别得到缺证明拒绝或 relation 反例，不执行无证据 fallback。
观察点	1. 原/降级可见行为满足证书定义，live CC/predicate/guard 不被 helper 破坏。 2. 缺证书与数值/状态 relation 不等价是不同结果，均保存具体证据。

TC-IR-003 项目	执行内容
通过条件	1. 至少一项真实 reference lowering 关系成立；单因子对照能定位证书缺失或 state clobber。 2. 实际执行量、检查量及所需覆盖非空，所有必需检查满足；本 case 当前 NOT_RUN 不因计划/静态检查而变为 PASS。
失败条件	1. 忽略 live CC 或内存副作用，只比较最终目的数值就宣称等价。 2. 无证书 helper 被隐式选择，或两个 oracle 共用待测 lowering 函数。
预期产物	1. manifest.json 2. case_result.json 3. coverage_delta.json 4. original_reference_trace.jsonl 5. lowered_reference_trace.jsonl 6. lowering_relation.json 7. clobber_diff.json

TC-IR-004：总线 stall 与采样

TC-IR-004 项目	执行内容
源与阶段	SIG-IR-004；P3_SM_RTL；CV-IR-003
配置与 seed	TP-SM-BASE；fixed_seed=0x534d0007（1397555207）；SM_PROGRESS
回归与当前状态	DIRECTED, RANDOM, STRESS, REPLAY；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. 固定并核对源 vPlan/testpoint、PTX9.4 inventory、target/capability、规则及模型版本 hash；fixed_seed 由主 campaign 在运行前赋值。 2. 项目接口版本、SM RTL/driver/monitor 及本 case 启用服务必须可用；native 编码未冻结时不得发送伪二进制。 3. 执行本 case 前冻结 timeout 预算；缺预算、模型、工具或服务时保持 BLOCKED，不以空执行关闭 case。
参数 1	timeout_budget_ref = campaign 配置中与 timeout_policy 对应的有限预算；不在本 case 猜测 SM 固定周期。
参数 2	ready_schedule = ["bounded_hold_low","single_accept","alternating_ready","drain_high"]
参数 3	fragment_plan = from selected frozen adapter schema
参数 4	identity_key = ["tag","generation","fragment_index"]

TC-IR-004 项目	执行内容
参数 5	tag_reuse = only after permitted retirement; generation then changes
初始化	1. adapter 接口/分片合同冻结；source 事务和 payload hash 已固定，scoreboard 无在途条目。 2. 驱动器与 monitor 独立采样，最终 drain 窗口允许进度。
执行步骤	1. 准备一个多片 IR 实例与一个在合法回收后复用 tag 但 generation 不同的实例，分别登记 payload hash。 2. 对第一实例保持 valid 并让 ready 在有限窗口为低，逐拍记录 payload/identity；随后放行各片。 3. 仅按接口允许的回收点复用 tag 并提交新 generation，交错其他合法事务及随机 ready。 4. 拉高 ready 完成 drain，按(tag,generation,fragment)核对唯一 accept 和 route；单独等待指令的真实完成，区分适配接收与执行结束。
观察点	1. valid 阻塞期间全部 payload、tag/generation 和 fragment 身份稳定；每片恰一次 accept。 2. tag 复用不混 generation，adapted 事件不被计为指令已完成。
通过条件	1. 所有计划分片在有界公平 ready 条件下唯一接收且 hash/route 一致；完成计数来自相应真实事件。 2. 实际执行量、检查量及所需覆盖非空，所有必需检查满足；本 case 当前 NOT_RUN 不因计划/静态检查而变为 PASS。
失败条件	1. 阻塞期间变值、重复接收副作用、丢片或旧 generation 串入新事务。 2. 只记录 adapter accept 便提前报告指令完成。
预期产物	1. manifest.json 2. case_result.json 3. coverage_delta.json 4. handshake_trace.jsonl 5. payload_hashes.json 6. fragment_scoreboard.json

20.3 GEN-DEP-01：寄存器、谓词、CC 与访存依赖

TC-DEP-001：依赖链与无关流混合

TC-DEP-001 项目	执行内容
源与阶段	SIG-DEP-001；P1_GENERATOR_UNIT；CV-DEP-001
配置与 seed	TP-GEN-BASE；fixed_seed=0x534d0008（1397555208）；GEN_BOUNDED

TC-DEP-001 项目	执行内容
回归与当前状态	SMOKE, DIRECTED, RANDOM；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. 发生器/对应 unit harness 和独立审计器已实现；当前缺失，因此本 case 为 NOT_RUN/BLOCKED_MISSING_IMPLEMENTATION。 2. 锁定 PTX9.4 form/target 证据、项目 capability、IR/schema 及数值 profile；缺规则的 variant 为 BLOCKED 而非 PASS。 3. 独立 checker 不调用 generator 或 DUT 的同一合法性、decode、rounding helper；共享常量及独立边界已评审。
参数 1	parameter_status = 下列规模/调度值是待冻结的测试预算建议，不是 SM 时延、native 编码或当前运行结果。
参数 2	seed_strategy = 固定 seed 由主 testplan 赋值；RNG 子流和所有抽样结果写入完整 manifest，边界枚举不依赖随机偶然命中。
参数 3	dependency_shapes = ["RAW","WAR","WAW","independent","partial_write","predicate","CC"]
参数 4	planned_instruction_counts = [8,64,256]
参数 5	dependency_distances = [0,1,2,7,31]
参数 6	carrier_widths = [32,64,128]
参数 7	wide_rule = 64/128 为载体/片段，不据此发明 128bit ALU opcode；仅已定义 IR 别名关系建重叠。
初始化	1. 输入和旧目的均初始化，thread/warp 命名空间隔离。 2. 每个模板记录期望依赖边与所选合法操作，不以相同字符串寄存器名代替别名模型。
执行步骤	1. 合成每类依赖与无关流的边界距离程序，给每次定义分配稳定 source ID。 2. 独立分析器从最终 IR 重建 path-sensitive def-use/live range 和合法 alias 图，不读取 generator 预填边作为真值。 3. 将重建图与目标模板比较，记录修复/重命名/优化前后哪些 hazard 消失。 4. 验证所有读有定义，保存 source 到 lowered 映射契约；本 P1 只关闭生成形状 bin，不计实际 SM hazard。
观察点	1. 每个目标依赖边真实存在，合法宽片段关系已明确；未定义读不被默认 0 掩盖。 2. 无关流与不同 thread 同编号寄存器未被错误合并成依赖。
通过条件	1. 全部枚举形状/距离有独立图证据；合法化修复后消失的目标形状不算命中。 2. P1 边/图覆盖与 DUT hazard 覆盖分账，源信息可复现。

TC-DEP-001 项目	执行内容
失败条件	1. checker 复用 generator 边集自证、误认跨 thread 别名或假造宽度重叠。 2. 缺定义/错误 live range 进入正常样本，或重命名消除依赖后仍算 DUT hazard。
预期产物	1. 完整 manifest 与 source/target/capability/profile/tool/IR hash（计划产物） 2. generated_IR_and_expected_edges.json 3. independently_rebuilt_defuse_graph.json 4. source_lowered_provenance.json

TC-DEP-002：隐式状态与分歧定义

TC-DEP-002 项目	执行内容
源与阶段	SIG-DEP-002；P1_GENERATOR_UNIT；CV-DEP-002
配置与 seed	TP-GEN-BASE；fixed_seed=0x534d0009（1397555209）；GEN_BOUNDED
回归与当前状态	DIRECTED, NEGATIVE, RANDOM；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. 发生器/对应 unit harness 和独立审计器已实现；当前缺失，因此本 case 为 NOT_RUN/BLOCKED_MISSING_IMPLEMENTATION。 2. 锁定 PTX9.4 form/target 证据、项目 capability、IR/schema 及数值 profile；缺规则的 variant 为 BLOCKED 而非 PASS。 3. 独立 checker 不调用 generator 或 DUT 的同一合法性、decode、rounding helper；共享常量及独立边界已评审。
参数 1	parameter_status = 下列规模/调度值是待冻结的测试预算建议，不是 SM 时延、native 编码或当前运行结果。
参数 2	seed_strategy = 固定 seed 由主 testplan 赋值；RNG 子流和所有抽样结果写入完整 manifest，边界枚举不依赖随机偶然命中。
参数 3	branch_shapes = ["diamond","nested_diamond"]
参数 4	guard_masks = ["all_true","all_false","even_lanes","odd_lanes","upper_half"]
参数 5	CC_chain_words = [2,4]
参数 6	single_fault_variants = ["remove_dominating_init","helper_temp_alias","CC_clobber_between_producer_consumer","call_then_unredefined_CC_read"]
参数 7	repair_policy = 仅显式允许时插入支配定义/替换临时寄存器，并另存修复后正例；原负例不计通过。

TC-DEP-002 项目	执行内容
初始化	1. 正向基线中 masked 写之前有旧值，所有启用线程的 CC 与 predicate 有可证明 producer。 2. call/helper clobber 规则来自冻结 ABI；不要求 CC 跨 call 保存。
执行步骤	1. 创建 CC/双 predicate/masked 定义跨分支使用的合法程序，独立审计所有可到达路径。 2. 对每个基线只注入一个定义缺失或 helper alias/clobber 故障，保留 mutation 位置与规则 ID。 3. 由独立分析器判定哪条路径/哪个 lane 缺定义；按配置拒绝或显式修复并重新全路径审计。 4. 比较修复前后 IR、clobber/live-range 和目标覆盖；非法候选不得发给 DUT。
观察点	1. predicated-off 定义未制造虚假 producer；分支 merge 保守保留每 lane 定义来源。 2. CC 活链与 helper/call 边界的处理符合 ABI，不将未初始化 CC 当恒零。
通过条件	1. 正常链通过、单因素错误被独立定位；允许修复的产物有新身份与复审记录。 2. CV-DEP-002 只由独立 generator 侧审计 CHECKED 采样，拒绝与接受分开。
失败条件	1. 只模拟一次随机路径便认定全路径有定义、把 CC 当 warp 共享状态或任意保留过 call。 2. 静默修复并仍算原负例命中、错误候选进入 functional stream。
预期产物	1. 完整 manifest 与 source/target/capability/profile/tool/IR hash（计划产物） 2. baseline_mutation_repair_triples.json 3. lane_path_definition_report.json 4. CC_predicate_clobber_report.json

TC-DEP-003：别名相关性

TC-DEP-003 项目	执行内容
源与阶段	SIG-DEP-003；P2_REFERENCE_PROGRAM；CV-DEP-001
配置与 seed	TP-REF-BASE；fixed_seed=0x534d000a（1397555210）；REFERENCE_BOUNDED
回归与当前状态	DIRECTED, RANDOM；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. 发生器/对应 unit harness 和独立审计器已实现；当前缺失，因此本 case 为 NOT_RUN/BLOCKED_MISSING_IMPLEMENTATION。 2. 锁定 PTX9.4 form/target 证据、项目 capability、IR/schema 及数值 profile；缺规则的 variant 为 BLOCKED 而非 PASS。 3. 独立 checker 不调用 generator 或 DUT 的同一合法性、decode、rounding helper；共享常量及独立边界已评审。 4. 参考程序解释/编译路线及独立 oracle 均存在并版本锁定；结果只作 P2 证据，不填 SM RTL 覆盖。

TC-DEP-003 项目	执行内容
参数 1	parameter_status = 下列规模/调度值是待冻结的测试预算建议，不是 SM 时延、native 编码或当前运行结果。
参数 2	seed_strategy = 固定 seed 由主 testplan 赋值；RNG 子流和所有抽样结果写入完整 manifest，边界枚举不依赖随机偶然命中。
参数 3	spaces = ["global","shared","local"]
参数 4	access_sizes_bytes = [1,2,4,8,16]
参数 5	object_bytes = 128
参数 6	overlap_classes = ["same_range","contained","one_byte_overlap","adjacent","disjoint"]
参数 7	owner_classes = ["same_CTA","different_CTA_same_shared_offset","different_thread_local","same_global_via_alias"]
参数 8	ordering = ["single_thread_program_order","defined_release_acquire_HB","alias_proxy_fence_when_required"]
初始化	1. 每个 typed 对象有 symbolic owner/base/长度、初始化 byte 内容和 canary；不同 CTA/shared offset0 独立。 2. 别名映射表和对应 proxy/HB 合同冻结；无同步竞争不生成单一顺序 golden。
执行步骤	1. 按实际对象构造重叠/相邻/不相交访问并建立必要 HB，另构造整数地址相等但 owner 不同的反例对。 2. 独立 range checker 规范化为(owner, physical-object, byte interval)并重算依赖，不只看原始整数地址。 3. 参考程序模型执行允许顺序，独立内存模型验证边和最终 bytes/允许集合。 4. 保存实际 byte 写 mask、alias/proxy 映射与 HB 证明，检查 canary 和未写字节。
观察点	1. 区间交集与依赖图一致；不同 owner/local thread 即使 offset 相同也不别名。 2. 逐值比较只用于允许集合已证明单例的观察；其他情况按合法顺序/集合检查。
通过条件	1. range/alias/HB 关系一致，参考观察不出现模型禁止结果。 2. byte/half/wide 交叠保持定义的 mask，不依赖 cache 命中决定正确性。
失败条件	1. shared/local owner 错误、地址别名漏 fence、无 HB 竞争被误写成唯一 golden。 2. race-free 被无条件当唯一结果、partial byte 写破坏邻接 canary。

TC-DEP-003 项目	执行内容
预期产物	1. 完整 manifest 与 source/target/capability/profile/tool/IR hash（计划产物） 2. typed_object_alias_map.json 3. byte_range_dependency_graph.json 4. HB_and_allowed_outcomes.json 5. memory_byte_diffs.json

TC-DEP-004：乱序/分片结果

TC-DEP-004 项目	执行内容
源与阶段	SIG-DEP-004；P3_SM_RTL；CV-DEP-003
配置与 seed	TP-SM-BASE；fixed_seed=0x534d000b（1397555211）；SM_PROGRESS
回归与当前状态	DIRECTED, RANDOM, STRESS；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. 发生器/对应 unit harness 和独立审计器已实现；当前缺失，因此本 case 为 NOT_RUN/BLOCKED_MISSING_IMPLEMENTATION。 2. 锁定 PTX9.4 form/target 证据、项目 capability、IR/schema 及数值 profile；缺规则的 variant 为 BLOCKED 而非 PASS。 3. 独立 checker 不调用 generator 或 DUT 的同一合法性、decode、rounding helper；共享常量及独立边界已评审。 4. 真实 SM RTL、已冻结项目机器入口/IR adapter 及正式驱动协议可用；不使用 PTX 文本字节冒充 native 编码。 5. 实际 admit/writeback/completion 与 lane/fragment/producer monitor 可用，外部服务的公平/延迟合同已配置。
参数 1	parameter_status = 下列规模/调度值是待冻结的测试预算建议，不是 SM 时延、native 编码或当前运行结果。
参数 2	seed_strategy = 固定 seed 由主 testplan 赋值；RNG 子流和所有抽样结果写入完整 manifest，边界枚举不依赖随机偶然命中。
参数 3	result_width_bits = [64,128]
参数 4	fragment_count_candidates = [2,4]
参数 5	fragment_delivery_orders = ["in_order","last_first","interleaved_two_producers"]
参数 6	planned_delays_cycles = [0,1,7,31]
参数 7	consumer_distances = [0,1,8]

TC-DEP-004 项目	执行内容
参数 8	stale_return_variant = 仅正式 generation/abort 协议允许时注入旧返回；缺该合同则 variant BLOCKED，不伪造合法接口事件。
初始化	1. 宽结果与消费者、短算术、CC 流使用独特 thread/producer 标识；所有旧目的初值已知。 2. 返回 fragment 编号和整体 ready 的正式观测点冻结；允许乱序的接口才重排返回。
执行步骤	1. 构造宽 load/move 等合法长延迟 producer，后跟依赖 consumer 及同名后写，独立审计实际 lowered hazard。 2. 通过真实 adapter 执行，在允许处重排/延迟 fragment 并交错短算术/其他 thread。 3. 逐片记录 capture/writeback/ready 与 consumer 读值，独立 scoreboard 关联完整 producer。 4. 在有正式旧 generation 注入合同的 variant 观察 late return 隔离，最后 drain 并核对全部动态义务。
观察点	1. 第一片或最后编号片先到均不能在缺片时发布整体 ready。 2. 源捕获、WAW 最终值与旧 generation 唤醒都按正确动态身份，不靠寄存器号猜 producer。
通过条件	1. 真实形成的 hazard 及完整结果被检查，consumer 从不读取部分/旧值，所有责任结清。 2. 迟到返回 variant 若启用，旧包不覆盖/唤醒新 producer；不支持时显式 blocked 而非伪 PASS。
失败条件	1. 首片 ready、漏片、重复 writeback、跨 generation 唤醒、WAR 源被后写污染。 2. 驱动违反返回协议却把 DUT 反应当正常功能失败，或没有实际 hazard 仍计覆盖。
预期产物	1. 完整 manifest 与 source/target/capability/profile/tool/IR hash（计划产物） 2. fragment_and_ready_event_trace.jsonl 3. actual_hazard_provenance.json 4. producer_scoreboard.json 5. late_generation_classification.json

20.4 GEN-ALU-01：整数、位操作、packed dot 与 video

TC-ALU-001：乘加与进位边界

TC-ALU-001 项目	执行内容
源与阶段	SIG-ALU-001；P1_GENERATOR_UNIT；CV-ALU-001
配置与 seed	TP-GEN-BASE；fixed_seed=0x534d000c（1397555212）；GEN_BOUNDED
回归与当前状态	SMOKE, DIRECTED, RANDOM；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION

TC-ALU-001 项目	执行内容
前置条件	1. 发生器/对应 unit harness 和独立审计器已实现；当前缺失，因此本 case 为 NOT_RUN/BLOCKED_MISSING_IMPLEMENTATION。 2. 锁定 PTX9.4 form/target 证据、项目 capability、IR/schema 及数值 profile；缺规则的 variant 为 BLOCKED 而非 PASS。 3. 独立 checker 不调用 generator 或 DUT 的同一合法性、decode、rounding helper；共享常量及独立边界已评审。
参数 1	parameter_status = 下列规模/调度值是待冻结的测试预算建议，不是 SM 时延、native 编码或当前运行结果。
参数 2	seed_strategy = 固定 seed 由主 testplan 赋值；RNG 子流和所有抽样结果写入完整 manifest，边界枚举不依赖随机偶然命中。
参数 3	operand_width_candidates = [16,32,64]
参数 4	signedness = ["signed","unsigned"]
参数 5	value_classes = ["0","1","-1_if_signed","MIN","MAX","MAX-1","alternating_bits","single_high_bit"]
参数 6	form_selection = 逐 form 过滤 add/sub/mul/mad 及.cc/.hi/.lo/.wide；不把 64bit carrier 自动解释成合法 mul.wide.u64。
参数 7	carry_chain_words = [2,4]
参数 8	instruction_budget_per_program = [8,32,128]
初始化	1. 每个 thread 的输入寄存器与 CC 初始化来自各自命名空间；正向链不依赖未定义 CC。 2. 枚举跨 word 全 1 进位、全 0 借位和 signed MIN/MAX 相邻边界；未启用的形式不进入正常流。
执行步骤	1. 从审计通过 form 实例枚举边界对，并分别构造 mul 高/低/宽结果及 2/4word 进位链的 typed IR。 2. 独立 validator 核对位宽、sign、合法修饰符、读取定义和 CC producer；将单项错误 variant 留在 preflight。 3. 以任意精度整数算完整数学结果，再按该指令规定截取/扩展/wrap/sat，单独生成 carry/borrow 期望制品。 4. 把 generator 自带期望（若有）与独立结果对照，保存每个源表达式、完整乘积与最终位串；本 P1 不执行 SM。
观察点	1. 独立审计后每个正常读有支配定义，.hi/.wide 期望确由完整乘积取得。 2. 每条 chain 的中间 word 与 CC 逐项可重算；signed/unsigned 解释不共用 host 溢出语义。

TC-ALU-001 项目	执行内容
通过条件	1. 已启用边界 variant 及 CC 期望全部匹配独立整数模型；未启用 variant 明确 blocked/rejected。 2. 合法/非法审计分类、候选制品与 CV-ALU-001 命中清单可追溯，P1 不增加 DUT 检查数。
失败条件	1. 低积被当作高积、符号扩展错误、CC 串 thread 或使用 host signed overflow 决定 golden。 2. 缺 oracle/零 variant 却报 PASS，或未定义 CC/非法 tuple 进入正常流。
预期产物	1. 完整 manifest 与 source/target/capability/profile/tool/IR hash（计划产物） 2. boundary_vectors.json 3. integer_full_product_and_CC_oracle.json 4. independent_audit.json

TC-ALU-002：位处理与 carryless

TC-ALU-002 项目	执行内容
源与阶段	SIG-ALU-002；P1_GENERATOR_UNIT；CV-ALU-002
配置与 seed	TP-GEN-BASE；fixed_seed=0x534d000d（1397555213）；GEN_BOUNDED
回归与当前状态	DIRECTED, NEGATIVE, RANDOM；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. 发生器/对应 unit harness 和独立审计器已实现；当前缺失，因此本 case 为 NOT_RUN/BLOCKED_MISSING_IMPLEMENTATION。 2. 锁定 PTX9.4 form/target 证据、项目 capability、IR/schema 及数值 profile；缺规则的 variant 为 BLOCKED 而非 PASS。 3. 独立 checker 不调用 generator 或 DUT 的同一合法性、decode、rounding helper；共享常量及独立边界已评审。
参数 1	parameter_status = 下列规模/调度值是待冻结的测试预算建议，不是 SM 时延、native 编码或当前运行结果。
参数 2	seed_strategy = 固定 seed 由主 testplan 赋值；RNG 子流和所有抽样结果写入完整 manifest，边界枚举不依赖随机偶然命中。
参数 3	bit_width_candidates = [16,32,64]
参数 4	shift_counts = ["0","1","width-1","width","width+1","0xffffffff"]
参数 5	bitfield_positions = ["0","1","width-1","width","width+1"]
参数 6	bitfield_lengths = ["0","1","width-1","width","width+1"]
参数 7	clmad_width = 64

TC-ALU-002 项目	执行内容
参数 8	clmad_values = ["0","1","bit0","bit63","all_ones","alternating_0x55_0xaa"]
初始化	1. 各位处理指令使用其合法位宽与 count 语义，位号和 bitfield 边界不按 host 移位定义推导。 2. clmad 的 a/b/c 均为 u64 位向量，另保留普通整数乘法结果作不等价对照而非 golden。
执行步骤	1. 枚举 shift/bitfield/count 组合及 clmad hi/lo 基向量，逐 form 过滤已定义与实现相关输入域。 2. 独立 bit-vector 解释器按规格处理超宽 shift、mask、sign extension、查找失败和 LUT；无合同项标 UNRESOLVED。 3. 独立 GF(2)模型用 XOR 多项式生成 128bit 积，选 hi/lo64 再 XOR c，检查 generator 候选期望。 4. 将明示 UB 候选隔离到 preflight，不为其生成唯一数值或 trap 期望；保存规则与拒绝原因。
观察点	1. bit63 乘 bit63 等基向量的 hi/lo 选段能由 GF(2)多项式重算。 2. width 及 width+1 的处理由每条原生语义/已审查 profile 决定，不触发 host 语言未定义移位。
通过条件	1. 所有有定义或 profile 化 variant 的期望符合独立位模型；UB 隔离与 oracle 缺失 blocked 均有记录。 2. clmad 不复用普通有进位乘加模型，P1 审计不计真实 DUT 覆盖。
失败条件	1. GF(2)被普通乘法替代、移位边界按 host 编译器偶然值比较、UB 值被判功能不一致。 2. 明示非法输入混入正常流或全部边界被静默过滤。
预期产物	1. 完整 manifest 与 source/target/capability/profile/tool/IR hash（计划产物） 2. bitvector_boundary_manifest.json 3. gf2_hi_lo_oracle.json 4. undefined_vs_profile_classification.json

TC-ALU-003：packed 与 video 语义

TC-ALU-003 项目	执行内容
源与阶段	SIG-ALU-003；P2_REFERENCE_PROGRAM；CV-ALU-001
配置与 seed	TP-REF-BASE；fixed_seed=0x534d000e（1397555214）；REFERENCE_BOUNDED
回归与当前状态	DIRECTED, RANDOM；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION

TC-ALU-003 项目	执行内容
前置条件	1. 发生器/对应 unit harness 和独立审计器已实现；当前缺失，因此本 case 为 NOT_RUN/BLOCKED_MISSING_IMPLEMENTATION。 2. 锁定 PTX9.4 form/target 证据、项目 capability、IR/schema 及数值 profile；缺规则的 variant 为 BLOCKED 而非 PASS。 3. 独立 checker 不调用 generator 或 DUT 的同一合法性、decode、rounding helper；共享常量及独立边界已评审。 4. 参考程序解释/编译路线及独立 oracle 均存在并版本锁定；结果只作 P2 证据，不填 SM RTL 覆盖。
参数 1	parameter_status = 下列规模/调度值是待冻结的测试预算建议，不是 SM 时延、native 编码或当前运行结果。
参数 2	seed_strategy = 固定 seed 由主 testplan 赋值；RNG 子流和所有抽样结果写入完整 manifest，边界枚举不依赖随机偶然命中。
参数 3	families = ["dp2a","dp4a","enabled_scalar_video","enabled_SIMD_video"]
参数 4	element_classes = ["zero","one","signed_min","signed_max","unsigned_max","alternating_sign","one_hot_element"]
参数 5	accumulator_classes = ["0","-1","S32_MIN","S32_MAX"]
参数 6	program_lengths = [4,32,128]
初始化	1. 用逐 element 独特标记构造 packed 载体，C/merge operand 初始化为可区分旧位与新位的 pattern。 2. 冻结每条 video 的 extract/sign/clamp/secondary-op/merge 表以及 dp2a/dp4a 合法输入 sign 组合。
执行步骤	1. 生成单 element 激活、全 element 极值及混合符号的参考程序；所有 carrier/padding 经独立审计。 2. 参考程序执行器运行各启用形式，输出逐 thread 结果和最终载体；记录操作身份及源 tuple。 3. 独立标量化模型逐 element extract/extend 计算数学结果，再依原文 clamp/merge 或 S32 结果规则打包。 4. 逐位置比较参考结果、未写入的 merge 位和 C 累加；不能调用 DUT decoder 或用 matrix dot 模型代替全部语义。
观察点	1. 单 element 激活只改变规定输出位置，符号扩展与高/低片段选择独立核对。 2. 饱和边界、C 极值和 secondary op 先后顺序的中间值可重算。

TC-ALU-003 项目	执行内容
通过条件	1. 参考程序结果与独立逐元素模型一致，未指定写入位置保持初始 merge 位。 2. 每个启用 form 至少有非零边界样本；无 DUT 执行覆盖声明。
失败条件	1. 跨 element 串位、packed 被归约为错误单标量、饱和/merge 顺序错或 C 重复加入。 2. 只检查载体低位而遗漏其余 element，或 oracle 调用了被测解释器同一 helper。
预期产物	1. 完整 manifest 与 source/target/capability/profile/tool/IR hash（计划产物） 2. reference_program_and_trace.json 3. scalarized_element_oracle.json 4. packed_merge_diffs.json

TC-ALU-004：算术与 CC 压力

TC-ALU-004 项目	执行内容
源与阶段	SIG-ALU-004；P3_SM_RTL；CV-ALU-003
配置与 seed	TP-SM-BASE；fixed_seed=0x534d000f（1397555215）；SM_PROGRESS
回归与当前状态	DIRECTED, RANDOM, STRESS；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. 发生器/对应 unit harness 和独立审计器已实现；当前缺失，因此本 case 为 NOT_RUN/BLOCKED_MISSING_IMPLEMENTATION。 2. 锁定 PTX9.4 form/target 证据、项目 capability、IR/schema 及数值 profile；缺规则的 variant 为 BLOCKED 而非 PASS。 3. 独立 checker 不调用 generator 或 DUT 的同一合法性、decode、rounding helper；共享常量及独立边界已评审。 4. 真实 SM RTL、已冻结项目机器入口/IR adapter 及正式驱动协议可用；不使用 PTX 文本字节冒充 native 编码。 5. 实际 admit/writeback/completion 与 lane/fragment/producer monitor 可用，外部服务的公平/延迟合同已配置。
参数 1	parameter_status = 下列规模/调度值是待冻结的测试预算建议，不是 SM 时延、native 编码或当前运行结果。
参数 2	seed_strategy = 固定 seed 由主 testplan 赋值；RNG 子流和所有抽样结果写入完整 manifest，边界枚举不依赖随机偶然命中。
参数 3	resident_warp_candidates = [1,2,4,8]
参数 4	instructions_per_warp = [64,512,4096]
参数 5	stream_mix = ["integer_add_shift","multiply_hi_wide","CC_chain","packed_dot_video"]

TC-ALU-004 项目	执行内容
参数 6	planned_external_stall_cycles = [0,1,7,31]
参数 7	delay_guard = 仅在正式返回/反压协议允许处施加有限 stall；SM_PROGRESS 上限由服务公平合同和工作量派生。
初始化	1. 每 warp/thread 使用不同输入指纹与独立 CC 链；跨 call 使用 CC 前重新定义。 2. 初始化完整 RF/数据和预期结果表；冻结真实 capability、服务配置和 admit/complete/check 计数。
执行步骤	1. 生成边界与随机混合程序，独立审计并保留 source→lowered→实际 DUT 操作的映射。 2. 通过正式 adapter 发送，分别记录握手/admit、source capture、结果片段与 CC 相关事件。 3. 用独立 timing 子流对合法返回/ready 施加有限 stall，交错同目的/依赖流与无关流。 4. scoreboard 按 thread 和动态 producer 比较完整值/CC，drain 全部责任后封存真实覆盖与 manifest。
观察点	1. 未捕获 source 保持保护；CC 与普通结果的每个 consumer 引用正确 producer。 2. admitted/completed/checked 可关联，stall 不使同一指令重复计数或丢失。
通过条件	1. 全部启用合法指令结果/CC 一致且无跨 thread 串扰，实际 checked 数量非零并匹配动态流。 2. 在配置公平假设下 drain 闭合，无将吞吐峰值/固定产品时延当通过条件。
失败条件	1. 错 forward、过早 ready、旧写覆盖新结果、CC 串扰、重复副作用或 orphan 指令。 2. 在公平前提有效时持续无进度；外部永久 stall 须另归类而非直接报 DUT deadlock。
预期产物	1. 完整 manifest 与 source/target/capability/profile/tool/IR hash（计划产物） 2. admit_writeback_CC_trace.jsonl 3. integer_scoreboard_results.json 4. progress_wait_graph.json 5. materialized_timing_schedule.json

20.5 GEN-FP-01：普通浮点、混合/packed 与 SFU

TC-FP-001：舍入与特殊值分桶

TC-FP-001 项目	执行内容
源与阶段	SIG-FP-001；P1_GENERATOR_UNIT；CV-FP-001
配置与 seed	TP-GEN-BASE；fixed_seed=0x534d0010（1397555216）；GEN_BOUNDED
回归与当前状态	SMOKE, DIRECTED, RANDOM；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION

TC-FP-001 项目	执行内容
前置条件	1. 发生器/对应 unit harness 和独立审计器已实现；当前缺失，因此本 case 为 NOT_RUN/BLOCKED_MISSING_IMPLEMENTATION。 2. 锁定 PTX9.4 form/target 证据、项目 capability、IR/schema 及数值 profile；缺规则的 variant 为 BLOCKED 而非 PASS。 3. 独立 checker 不调用 generator 或 DUT 的同一合法性、decode、rounding helper；共享常量及独立边界已评审。
参数 1	parameter_status = 下列规模/调度值是待冻结的测试预算建议，不是 SM 时延、native 编码或当前运行结果。
参数 2	seed_strategy = 固定 seed 由主 testplan 赋值；RNG 子流和所有抽样结果写入完整 manifest，边界枚举不依赖随机偶然命中。
参数 3	format_candidates = ["F16","BF16","F32","F64"]
参数 4	operation_candidates = ["add","sub","mul","fma","mad","min","max","testp","copysign"]
参数 5	rounding_candidates = ["rn","rz","rm","rp"]
参数 6	value_classes = ["+0","-0","min_subnormal","max_subnormal","min_normal","max_finite","+Inf","-Inf","qNaN","sNaN_if_defined","half_ULP_ties","cancellation"]
参数 7	F32_example_input_bits = ["00000000","80000000","00000001","007fffff","00800000","7f7fffff","7f800000","ff800000","7fc00001","7f800001"]
参数 8	modifier_selection = 逐 form 过滤 FTZ/noFTZ/sat/satfinite/relu，不能对 F64 等未允许形式强加修饰符。
初始化	1. 锁定独立 FP oracle 版本和 NaN/zero/FTZ 规则，generator golden 与 oracle 来源不相同。 2. 每种启用 format 枚举正负临界值与每个允许 rounding；未规定 NaN payload 不建立唯一位模式。

TC-FP-001 项目	执行内容
执行步骤	1. 从原始格式编码与精确有理/整数表示生成边界向量和相邻 ULP 样本，不靠 host float cast 产生 golden。 2. 独立 validator 检查 tuple、carrier、rounding 与 modifier；建立可检查位 mask/允许 NaN 集合。 3. 独立 oracle 输出逐形式正确舍入值或允许集合，特殊值走显式分类；普通 fma 按融合语义。 4. 核对候选分桶、generator 期望制品和可序列化位串；本 P1 只检查激励/期望，不声称 FP 执行通过。
观察点	1. 半 ULP 的 even/odd 尾数、进位跨指数、subnormal/normal 切换及 overflow 方向均有非空样本。 2. NaN payload、±0 和 FTZ 比较 mask 来自规则；0×Inf 等特殊组合不被有限 Fraction 模型吞掉。
通过条件	1. 边界类别与独立 oracle 一致；没有规定的 payload 仅按允许集合检查。 2. 未审查的 tuple/oracle 为 blocked，P1 不填 DUT 浮点覆盖。
失败条件	1. 把 double 运算无条件当 FP32/FP64 精确 FMA、NaN 全部逐 bit 比对或错误清除负零。 2. 为了凑覆盖对不存在的 modifier/格式组合给出 golden。
预期产物	1. 完整 manifest 与 source/target/capability/profile/tool/IR hash（计划产物） 2. FP_input_bits_and_classes.json 3. rounding_and_specialvalue_oracle.json 4. NaN_allowed_masks.json

TC-FP-002：packed/mixed target 联合约束

TC-FP-002 项目	执行内容
源与阶段	SIG-FP-002；P1_GENERATOR_UNIT；CV-FP-002
配置与 seed	TP-GEN-BASE；fixed_seed=0x534d0011（1397555217）；GEN_BOUNDED
回归与当前状态	DIRECTED, NEGATIVE, RANDOM；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. 发生器/对应 unit harness 和独立审计器已实现；当前缺失，因此本 case 为 NOT_RUN/BLOCKED_MISSING_IMPLEMENTATION。 2. 锁定 PTX9.4 form/target 证据、项目 capability、IR/schema 及数值 profile；缺规则的 variant 为 BLOCKED 而非 PASS。 3. 独立 checker 不调用 generator 或 DUT 的同一合法性、decode、rounding helper；共享常量及独立边界已评审。

TC-FP-002 项目	执行内容
参数 1	parameter_status = 下列规模/调度值是待冻结的测试预算建议，不是 SM 时延、native 编码或当前运行结果。
参数 2	seed_strategy = 固定 seed 由主 testplan 赋值；RNG 子流和所有抽样结果写入完整 manifest，边界枚举不依赖随机偶然命中。
参数 3	tuple_groups = ["mixed_F16x2_BF16x2_F32x2","alternate_FP8_FP6_FP4_UE8M0_x4"]
参数 4	target_candidates = ["sm_90a","sm_100a","sm_103a","sm_107f"]
参数 5	element_markers = ["all_distinct","one_element_nonzero","alternating_sign","one_special_element"]
参数 6	carrier_variants = ["E2M1x4_b16","E2M1p4x4_b32","FP6_byte_padded_b32","F32x2_b64"]
参数 7	mutations = ["target_only","carrier_width_only","documented_required_padding_only","one_type_role_only"]
初始化	1. 正向基线逐 form 源规则已审查；不把 9.4 新增推成 sm107 自动支持。 2. 每 element 值可独立辨认；NaN/Inf 只用于该格式实际支持的编码。
执行步骤	1. 构造每个启用 mixed/alternate tuple，检查其载体、padding 和逐 element 角色。 2. 独立 validator 分别重算正向合法性以及单因素 target/carrier mutation 的拒绝层/理由。 3. 独立逐 element oracle 计算期望；用单 element 非零样本证明四项输出不能被合并为 dot。 4. 保存正/负基线、mutation diff 与允许输出；非法与 UB 只在 preflight 隔离，不生成硬件 trap 期望。
观察点	1. mixedpacked 的 sm107f family 与 alternate x4 专属 target 依据分别定位，未按 SM 数字统一继承。 2. 修改一个 element 只影响其规定结果位置；E2M1 紧凑与 padded 载体转换不串位。
通过条件	1. 正例与独立 tuple/element 模型一致，负例命中指定规则，未静默改写后算负例成功。 2. oracle 缺失的 variant 明确 blocked；P1 结果不计真实 packed 硬件覆盖。
失败条件	1. 把 x4 当 K=4 归约、强行允许非法 target、忽略禁止 padding 或错误 NaN 编码。 2. 任何非零退出码均算 mutation 命中，缺工具被报成正确拒绝。

TC-FP-002 项目	执行内容
预期产物	1. 完整 manifest 与 source/target/capability/profile/tool/IR hash（计划产物） 2. packed_tuple_baselines.json 3. single_fault_mutations.json 4. per_element_expected_results.json 5. independent_target_audit.json

TC-FP-003：SFU 误差与 FMA 对照

TC-FP-003 项目	执行内容
源与阶段	SIG-FP-003；P2_REFERENCE_PROGRAM；CV-FP-001
配置与 seed	TP-REF-BASE；fixed_seed=0x534d0012（1397555218）；REFERENCE_BOUNDED
回归与当前状态	DIRECTED, RANDOM；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. 发生器/对应 unit harness 和独立审计器已实现；当前缺失，因此本 case 为 NOT_RUN/BLOCKED_MISSING_IMPLEMENTATION。 2. 锁定 PTX9.4 form/target 证据、项目 capability、IR/schema 及数值 profile；缺规则的 variant 为 BLOCKED 而非 PASS。 3. 独立 checker 不调用 generator 或 DUT 的同一合法性、decode、rounding helper；共享常量及独立边界已评审。 4. 参考程序解释/编译路线及独立 oracle 均存在并版本锁定；结果只作 P2 证据，不填 SM RTL 覆盖。
参数 1	parameter_status = 下列规模/调度值是待冻结的测试预算建议，不是 SM 时延、native 编码或当前运行结果。
参数 2	seed_strategy = 固定 seed 由主 testplan 赋值；RNG 子流和所有抽样结果写入完整 manifest，边界枚举不依赖随机偶然命中。
参数 3	F32_fma_witness = {"a":"1+2^-23","b":"1-2^-23","c":"-1","rounding":"rn","ftz":false,"fused_math":"-2^-46","separate_mul_add_result":"+0"}
参数 4	SFU_candidates = ["sin","cos","lg2","ex2","div","rcp","rsqrt"]
参数 5	domain_sampling = ["source_defined_endpoints","adjacent_representable_values","zero","signed_zero","powers_of_two","minmax_exponents","Inf_NaN_if_defined"]
参数 6	error_policy = 每 form 从固定规则读取 ULP/relative/absolute 界与适用区间；无规则不使用统一 epsilon。

TC-FP-003 项目	执行内容
初始化	1. 参考程序执行器与独立高精度 oracle 可分别提供结果，不共用被测数值 helper。 2. 精确、full、approx 及 ftz forms 的域/误差/特殊值规则已版本锁定。
执行步骤	1. 生成明确融合与先乘后加不同的 F32 witness 及邻域值，另建立 SFU 分段边界集合。 2. 独立模型先证明 witness 的两个精确关系，再让参考解释/编译路线执行对应合法程序。 3. 对正确舍入形式逐位或允许 NaN 集合比较，对 approx 形式按各自区间和误差公式检查。 4. 记录输入编码、参考输出、高精度值、误差计算及被采用规则；缺规则 variant 为 blocked。
观察点	1. rn 且非 FTZ 的 F32 witness 产生融合-2^-46 而独立 mul 后 add 为+0，足以区分两条语义。 2. SFU 域边界/特殊输入不沿用普通有限数 epsilon；精确与 approx 判据互不替代。
通过条件	1. 所有启用参考执行结果落入对应允许集合/误差界，规则 ID 与计算过程可重算。 2. 非空 FMA witness 被检查；仅 P2 reference 证据，不声称 SM SFU/FP 已实现。
失败条件	1. FMA 被错误拆分、approx 套单一 epsilon、超定义区间仍使用不适用误差公式。 2. oracle 实际调用同一被测 libm 函数或将设备未公开数值假装精确 bitfit。
预期产物	1. 完整 manifest 与 source/target/capability/profile/tool/IR hash（计划产物） 2. fused_vs_split_witness.json 3. reference_execution_trace.jsonl 4. SFU_error_domain_report.json

TC-FP-004：混合执行域

TC-FP-004 项目	执行内容
源与阶段	SIG-FP-004；P3_SM_RTL；CV-FP-003
配置与 seed	TP-SM-BASE；fixed_seed=0x534d0013（1397555219）；SM_PROGRESS
回归与当前状态	DIRECTED, RANDOM, STRESS；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. 发生器/对应 unit harness 和独立审计器已实现；当前缺失，因此本 case 为 NOT_RUN/BLOCKED_MISSING_IMPLEMENTATION。 2. 锁定 PTX9.4 form/target 证据、项目 capability、IR/schema 及数值 profile；缺规则的 variant 为 BLOCKED 而非 PASS。 3. 独立 checker 不调用 generator 或 DUT 的同一合法性、decode、rounding helper；共享常量及独立边界已评审。 4. 真实 SM RTL、已冻结项目机器入口/IR adapter 及正式驱动协议可用；不使用 PTX 文本字节冒充 native 编码。 5. 实际 admit/writeback/completion 与 lane/fragment/producer monitor 可用，外部服务的公平/延迟合同已配置。

TC-FP-004 项目	执行内容
参数 1	parameter_status = 下列规模/调度值是待冻结的测试预算建议，不是 SM 时延、native 编码或当前运行结果。
参数 2	seed_strategy = 固定 seed 由主 testplan 赋值；RNG 子流和所有抽样结果写入完整 manifest，边界枚举不依赖随机偶然命中。
参数 3	resident_warp_candidates = [1,2,4,8]
参数 4	operations_per_warp = [64,512,2048]
参数 5	domain_candidates = ["F32","F64","half_BF16_packed","mixed_alternate_packed","SFU"]
参数 6	dependent_consumer_distances = [0,1,7,31]
参数 7	planned_stall_windows = [0,1,8,32]
初始化	1. 每 operation 携带允许的类型/rounding/FTZ/profile 身份，初始化含非对称 element 值与特殊值。 2. 真实 capability 不支持的 domain 显式记录 blocked，不静默删去后声称跨域全通过。
执行步骤	1. 独立审计混合程序并生成各 operation 的精确/允许集合/误差 oracle，冻结完整 manifest。 2. 经真实 adapter 发射并用合法反压切换高/低精度、长短时延及不同 warp。 3. 记录 admit、完整 writeback、consumer 与 mode/profile 关联；每个 DUT 实际形成的依赖单独确认。 4. scoreboard 按每 operation 的规则检查结果并 drain；比较不以吞吐目标或统一固定时延为条件。
观察点	1. 前后两种 mode 的特殊值/舍入/profile 未串用，packed 各 element 保持独立。 2. dependent consumer 不得提前使用部分/旧结果；实际 CHECKED 与完整返回一一关联。
通过条件	1. 所有实际启用操作满足各自数值判据且完成义务闭合，真实 checked 样本非零。 2. 合法 stall 不改变结果/身份；资源压力下在公平合同内持续前进。
失败条件	1. mode 切换使 rounding/FTZ/NaN 规则污染其他指令，或者共享 SFU 只提供错误统一近似值。 2. 零实际执行、缺失 oracle 或仅数 admit 便宣称 FP 通过。
预期产物	1. 完整 manifest 与 source/target/capability/profile/tool/IR hash（计划产物） 2. FP_domain_operation_manifest.json 3. DUT_value_profile_trace.jsonl 4. per_operation_numeric_check.json 5. progress_and_actual_hazards.json

20.6 GEN-CVT-01：转换、packing 及寄存器稀疏变换

TC-CVT-001：数值转换边界

TC-CVT-001 项目	执行内容
源与阶段	SIG-CVT-001；P1_GENERATOR_UNIT；CV-CVT-001
配置与 seed	TP-GEN-BASE；fixed_seed=0x534d0014（1397555220）；GEN_BOUNDED
回归与当前状态	SMOKE, DIRECTED, REPLAY；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. 固定并核对源 vPlan/testpoint、PTX9.4 inventory、target/capability、规则及模型版本 hash；fixed_seed 由主 campaign 在运行前赋值。 2. 发生器、unit harness 和独立合法性/结构审计器必须可用；共享 source 身份不等于共享待测约束函数。 3. 执行本 case 前冻结 timeout 预算；缺预算、模型、工具或服务时保持 BLOCKED，不以空执行关闭 case。
参数 1	timeout_budget_ref = campaign 配置中与 timeout_policy 对应的有限预算；不在本 case 猜测 SM 固定周期。
参数 2	form_selection = independently validated per-target integer<->FP and FP<->FP recipes
参数 3	value_classes = ["zero_signed","normal_midpoint_and_neighbors","min_subnormal_boundary","max_finite_overflow_boundary","integer_range_boundary","NaN_Inf_where_defined"]
参数 4	modifier_trials = ["rn/rz where legal","sat/satfinite where legal","pzo where legal","rs with explicit rbits where legal"]
参数 5	rbits_policy = manifest contains exact operand bits; repeat identical then change only rbits
初始化	1. 每条 recipe 已声明源/目的格式、carrier、rounding/FTZ 及 target；未知规则形式不进入正向。 2. 独立转换模型使用数学值/整数表示，不调用 DUT decoder。
执行步骤	1. 按每条合法 recipe 计算可表示边界与精确中点，构造中点两侧、溢出/下溢及符号样本。 2. 分别生成合法 rounding/saturation/pzo trial；每次仅改变一个允许 modifier 并保留相同数值输入。 3. 对 rs 形式把 rbits 作为显式 operand 写入 manifest，先同 rbits 重复，再单改 rbits 进行配对试验。 4. 比较生成/序列化结果与独立转换模型；bitcast 路径作为不同操作，不能替代 cvt 数值规则。

TC-CVT-001 项目	执行内容
观察点	1. 每条结果使用该 recipe 的 rounding/FTZ/NaN 规则，不能以目标位数截断替代。 2. 同输入和 rbits 可重放；rbits 改变不牵连其他随机子流或隐式全局 RNG。
通过条件	1. 全部选定边界样本与独立规则一致，所有执行形式具有合法 target 证据。 2. 实际执行量、检查量及所需覆盖非空，所有必需检查满足；本 case 当前 NOT_RUN 不因计划/静态检查而变为 PASS。
失败条件	1. 舍入/饱和/pzo 被忽略，或 rs 结果依赖未记录隐式随机状态。 2. 不支持的 modifier 被当通用功能，或误把 bitcast 等同数值转换。
预期产物	1. manifest.json 2. case_result.json 3. coverage_delta.json 4. conversion_vectors.json 5. rounding_reference.json 6. rbits_manifest.json 7. serialized_operands.json

TC-CVT-002：稀疏载体约束

TC-CVT-002 项目	执行内容
源与阶段	SIG-CVT-002；P1_GENERATOR_UNIT；CV-CVT-002
配置与 seed	TP-GEN-BASE；fixed_seed=0x534d0015（1397555221）；GEN_BOUNDED
回归与当前状态	DIRECTED, NEGATIVE；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. 固定并核对源 vPlan/testpoint、PTX9.4 inventory、target/capability、规则及模型版本 hash；fixed_seed 由主 campaign 在运行前赋值。 2. 发生器、unit harness 和独立合法性/结构审计器必须可用；共享 source 身份不等于共享待测约束函数。 3. 执行本 case 前冻结 timeout 预算；缺预算、模型、工具或服务时保持 BLOCKED，不以空执行关闭 case。
参数 1	timeout_budget_ref = campaign 配置中与 timeout_policy 对应的有限预算；不在本 case 猜测 SM 固定周期。
参数 2	operations = ["spcompress","spdecompress"]
参数 3	recipe_source = enabled legal spfactor/idxsize/num/elemsize/descriptor tuples
参数 4	register_total_limit = 253

TC-CVT-002 项目	执行内容
参数 5	boundary_selection = highest reachable legal total <=253; do not assume exactly253 is attainable
参数 6	negative_mutation = {"field":"one output register binding","change":"alias one live input register without changing vector lengths","expected_rule":"VECTOR_OPERAND_OVERLAP_FORBIDDEN","expected_layer":"LEGALIZER_ONLY","classification":"PTX_UB_QUARANTINED"}
初始化	1. 输入/输出 vector 分配互不重叠，descriptor 与 elemsize 一致；总量按相应公式计算。 2. 合法 recipe 及 register-set 审计器可用；不存在固定 253 恰可达到的假设。
执行步骤	1. 遍历当前 target 启用 recipe，独立计算 data/cdata/mdata 长度及寄存器总量，选择小规模与最高可达合法边界。 2. 为每个 recipe 分配互不重叠集合并生成 descriptor/metadata，核对 padding 和输入 span 等条件。 3. 复制一个合法 baseline，仅把一个输出寄存器绑定改成已用输入，保持 num、长度、descriptor 和总量不变。 4. 只向 legalizer 提交重叠副本，要求单一重叠违规诊断并标 UB quarantine；对合法 baseline 继续结构审计，不发送负例到普通数值执行流。
观察点	1. 合法长度、总量<=253、descriptor 一致性和无重叠均独立成立。 2. 负例恰改变一个 binding 且只验证 legalizer 拒入，不设硬件 trap 或普通值 oracle。
通过条件	1. 所有启用 recipe 的可达边界被正确选取，重叠 mutant 被隔离，合法基线不被错误过滤。 2. 实际执行量、检查量及所需覆盖非空，所有必需检查满足；本 case 当前 NOT_RUN 不因计划/静态检查而变为 PASS。
失败条件	1. 按字面 253 制造不存在的 tuple，或漏检 vector 复用/总量条件。 2. 执行重叠 UB 并把设备返回/崩溃当预期通过。
预期产物	1. manifest.json 2. case_result.json 3. coverage_delta.json 4. sparse_recipe_sizes.json 5. register_allocation.json 6. single_binding_diff.json 7. legalizer_rejections.json

TC-CVT-003：选择与补零允许集合

TC-CVT-003 项目	执行内容
源与阶段	SIG-CVT-003；P2_REFERENCE_PROGRAM；CV-CVT-001
配置与 seed	TP-REF-BASE；fixed_seed=0x534d0016（1397555222）；REFERENCE_BOUNDED
回归与当前状态	DIRECTED, NEGATIVE；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. 固定并核对源 vPlan/testpoint、PTX9.4 inventory、target/capability、规则及模型版本 hash；fixed_seed 由主 campaign 在运行前赋值。 2. 所用独立参考解释器/允许结果模型及 lowering 或服务 reference route 必须可用；P2 结果不计真实 SM 执行。 3. 执行本 case 前冻结 timeout 预算；缺预算、模型、工具或服务时保持 BLOCKED，不以空执行关闭 case。 4. PTX § 9.7.10.30/.31 PDF392–395 的选择与 implementation-specific 说明作为 oracle 边界；不得扩展为任意输出都合法。
参数 1	timeout_budget_ref = campaign 配置中与 timeout_policy 对应的有限预算；不在本 case 猜测 SM 固定周期。
参数 2	compress_patterns = ["two_equal_extrema","three_or_more_NaNs","minus_zero_plus_zero","repeated_second_extreme"]
参数 3	decompress_metadata_classes = ["0<=index<num_target: defined placement","index>num_target: implementation-specific in source","index==num_target: source-boundary interpretation requires review"]
参数 4	oracle = allowed-selection set plus exact chosen-value/metadata consistency and zero fill
参数 5	unknown_target_behavior = BLOCKED_SOURCE_REVIEW, not invented unique golden
参数 6	negative_mutation = {"field":"one output metadata index","change":"choose an element outside source-permitted winner set in a checker-only result fixture","expected_layer":"REFERENCE_RESULT_CHECKER"}
参数 7	pattern_applicability = NaN patterns only use source-legal formats encoding NaN; finite-only FP and integer recipes do not receive invented NaN bit strings.
参数 8	negative_fixture_baseline = Use a group with a strict permitted winner set so a single metadata-index mutation has a demonstrably disallowed selection.

TC-CVT-003 项目	执行内容
初始化	1. 允许的 selection/tie/NaN 规则已登记，metadata 补零模型独立。 2. implementation-specific/OOB 行为若无目标定义，只保留符号允许集/阻塞，不标 PTX UB。
执行步骤	1. 仅在支持相应值类的合法输入格式构造等值极值、≥3 个 NaN 和±0 组，独立枚举规范允许的 winner 集合；整数/finite-only 格式不伪造 NaN。 2. 通过 reference route 取得压缩数据与 metadata，检查选中索引和值一致、NaN 优先及-0<+0 规则；不强制唯一 tie-break。 3. 用 in-range metadata 执行解压 reference，逐位置检查散布与未填零；对 index>num_target 单列 implementation-specific trial，仅在已登记目标行为模型下检查。 4. 将 index==num_target 解释缺口标 review gate；另在 checker-only 输出 fixture 中仅改一个 winner index，确认非法结果被允许集合检查器拒绝。
观察点	1. 所有正向 selection 属于允许集合，chosen data 与 metadata 相符，合法解压补零逐位置正确。 2. implementation-specific、source 解释缺口和真正 UB 分类分开；没有对前者施加通用唯一 golden。
通过条件	1. 定义域内结果及允许集合关系正确；checker-only 单索引反例被识别。 2. 目标未定义行为没有被偷偷当固定值或 UB 执行预期。 3. 实际执行量、检查量及所需覆盖非空，所有必需检查满足；本 case 当前 NOT_RUN 不因计划/静态检查而变为 PASS。
失败条件	1. tie-break 被硬编码成唯一索引，NaN/±0 优先规则或补零被破坏。 2. 把 source 声明 implementation-specific 的索引行为当硬件 trap 或把==边界解释缺口直接通过。
预期产物	1. manifest.json 2. case_result.json 3. coverage_delta.json 4. selection_allowed_sets.json 5. reference_sparse_results.json 6. decompression_position_checks.json 7. behavior_classification.json

TC-CVT-004：多寄存器返回

TC-CVT-004 项目	执行内容
源与阶段	SIG-CVT-004；P3_SM_RTL；CV-CVT-003
配置与 seed	TP-SM-BASE；fixed_seed=0x534d0017（1397555223）；SM_PROGRESS

TC-CVT-004 项目	执行内容
回归与当前状态	DIRECTED, STRESS；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. 固定并核对源 vPlan/testpoint、PTX9.4 inventory、target/capability、规则及模型版本 hash；fixed_seed 由主 campaign 在运行前赋值。 2. 项目接口版本、SM RTL/driver/monitor 及本 case 启用服务必须可用；native 编码未冻结时不得发送伪二进制。 3. 执行本 case 前冻结 timeout 预算；缺预算、模型、工具或服务时保持 BLOCKED，不以空执行关闭 case。
参数 1	timeout_budget_ref = campaign 配置中与 timeout_policy 对应的有限预算；不在本 case 猜测 SM 固定周期。
参数 2	large_vector_recipe = largest selected legal .num recipe within operand/register budgets
参数 3	return_schedule = ["first_fragment_only_then_stall","middle_fragment_stall","drain"]
参数 4	dependency_tests = ["consumer_waits_for_all_required_fragments","later_writer_waits_for_uncaptured_sources"]
初始化	1. 真实 wide/vector 转换 route 及 RF 片段/源捕获 monitor 可用，所有目的片段标未就绪。 2. 源和 guard 保存在合法活跃寄存器，recipe 满足本 target 长度与 253 总量等约束。
执行步骤	1. 为大.num 稀疏变换或宽转换生成非重复分量 pattern，先用独立模型建立全部目的片段期望。 2. 发射操作并在首片返回后施加有限 RF backpressure，同时放入依赖 consumer 和可能后写源的合法指令。 3. 观察中间片段和源捕获状态，确认 consumer 尚不能读取未完成整体且后写不会覆盖未捕获源。 4. 放行 drain 并收齐全部片段，再执行 consumer 与后写，逐片和最终结果核对且检查每片恰结算一次。
观察点	1. 首片返回不使多片目的整体 ready，允许的片段级 ready 也必须匹配消费者确切需求。 2. 源值直到实际捕获前受保护，全部片段写回后 consumer 读取完整正确结果。
通过条件	1. 全部必需片段被真实检查，源/目的依赖和反压恢复正确，无固定延迟假设。 2. 实际执行量、检查量及所需覆盖非空，所有必需检查满足；本 case 当前 NOT_RUN 不因计划/静态检查而变为 PASS。
失败条件	1. 首片触发整体 ready、片段丢失/重复，或 WAR 覆盖未捕获源。 2. RF backpressure 改变输入、metadata 或 profile。

TC-CVT-004 项目	执行内容
预期产物	1. manifest.json 2. case_result.json 3. coverage_delta.json 4. rf_fragment_trace.jsonl 5. source_capture_ledger.json 6. consumer_scoreboard.json

20.7 GEN-CTRL-01：CFG、predication、ITS 与调用

TC-CTRL-001：有界控制图

TC-CTRL-001 项目	执行内容
源与阶段	SIG-CTRL-001；P1_GENERATOR_UNIT；CV-CTRL-001
配置与 seed	TP-GEN-BASE；fixed_seed=0x534d0018（1397555224）；GEN_BOUNDED
回归与当前状态	SMOKE, DIRECTED, RANDOM；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. 发生器/对应 unit harness 和独立审计器已实现；当前缺失，因此本 case 为 NOT_RUN/BLOCKED_MISSING_IMPLEMENTATION。 2. 锁定 PTX9.4 form/target 证据、项目 capability、IR/schema 及数值 profile；缺规则的 variant 为 BLOCKED 而非 PASS。 3. 独立 checker 不调用 generator 或 DUT 的同一合法性、decode、rounding helper；共享常量及独立边界已评审。
参数 1	parameter_status = 下列规模/调度值是待冻结的测试预算建议，不是 SM 时延、native 编码或当前运行结果。
参数 2	seed_strategy = 固定 seed 由主 testplan 赋值；RNG 子流和所有抽样结果写入完整 manifest，边界枚举不依赖随机偶然命中。
参数 3	basic_block_counts = [4,8,16]
参数 4	branch_nesting_depths = [0,1,2,3]
参数 5	loop_trip_counts = [0,1,2,8]
参数 6	maximum_call_depth_candidates = [0,1,3]
参数 7	indirect_target_set_sizes = [1,2,4]
参数 8	instruction_budget_including_cleanup = [32,128,512]

TC-CTRL-001 项目	执行内容
初始化	1. 每 thread 入口 PC、输入、guard、call frame 和初始化集合明确；控制预算含收尾。 2. 每个循环有独立检查的 ranking/有限 trip count，间接目标表只含合法同型目标。
执行步骤	1. 按 diamond/嵌套分支/loop/call 模板合成 CFG，枚举 0/1/边界迭代和可到达边。 2. 独立 CFG 分析器重算目标合法性、支配/后支配、可达性、每 lane def-use 与 loop ranking。 3. 在全部可到达路径检查对象/调用/cleanup 闭合；预算不足时拒绝扩展 body 而非截断尾部。 4. 输出静态边/路径证明及候选分布；本 P1 只确认生成程序合法与有界。
观察点	1. 0 次循环不读取 body 才定义的值，分支 merge 之后每个正常读均有合法来源。 2. 每个 backedge 有 ranking 证据，间接目标与返回/ABI 匹配。
通过条件	1. 所有正向 CFG 有终止与全路径合法性证据，实际未生成的 edge 不算命中。 2. UNSAT/预算不足在有界时间内明确诊断，零程序不能 PASS。
失败条件	1. 只模拟一条随机路径替代全路径审计、非法间接目标或用 watchdog 超时证明循环正确。 2. 截断指令流遗留 call frame/异步 cleanup 或未初始化读取。
预期产物	1. 完整 manifest 与 source/target/capability/profile/tool/IR hash（计划产物） 2. CFG_and_source_IR.json 3. dominance_reachability_ranking.json 4. path_definition_and_cleanup_report.json

TC-CTRL-002：谓词与 uniform 证明

TC-CTRL-002 项目	执行内容
源与阶段	SIG-CTRL-002；P1_GENERATOR_UNIT；CV-CTRL-002
配置与 seed	TP-GEN-BASE；fixed_seed=0x534d0019（1397555225）；GEN_BOUNDED
回归与当前状态	DIRECTED, NEGATIVE, RANDOM；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. 发生器/对应 unit harness 和独立审计器已实现；当前缺失，因此本 case 为 NOT_RUN/BLOCKED_MISSING_IMPLEMENTATION。 2. 锁定 PTX9.4 form/target 证据、项目 capability、IR/schema 及数值 profile；缺规则的 variant 为 BLOCKED 而非 PASS。 3. 独立 checker 不调用 generator 或 DUT 的同一合法性、decode、rounding helper；共享常量及独立边界已评审。
参数 1	parameter_status = 下列规模/调度值是待冻结的测试预算建议，不是 SM 时延、native 编码或当前运行结果。

TC-CTRL-002 项目	执行内容
参数 2	seed_strategy = 固定 seed 由主 testplan 赋值；RNG 子流和所有抽样结果写入完整 manifest，边界枚举不依赖随机偶然命中。
参数 3	guard_patterns = ["all_true","all_false","even_lanes","odd_lanes","lower_half","single_lane"]
参数 4	guarded_effect_candidates = ["register_write","store","atomic","async_issue_if_enabled"]
参数 5	uniform_sites = ["branch_guard_and_target","indirect_index_or_call_target"]
参数 6	negative_mutation = 仅改变一个 lane 的 guard/target 以破坏相应.uni 合同，单独 preflight 拒绝。
初始化	1. 旧目的值、存储 canary 与副作用计数期望初始化；predication 与 active/membermask 分开。 2. uniform 证明使用可到达动态 active group 的符号值关系，不凭某个 seed 碰巧相等。
执行步骤	1. 生成各 guard 模式与 guarded effect 候选，并在有符号一致证明的 site 选择.uni。 2. 独立 lane symbolic 分析重算有效执行 mask 和 guard/target 一致性，审计副作用期望制品。 3. 对单因素违反.uni 的候选独立拒绝，保留合法 baseline 和准确破坏关系。 4. 在 unit 参考语义中验证被屏蔽操作不创建 write/store/atomic/async 义务，保存审计 trace；不冒充真实 DUT 事件。
观察点	1. all_false 不制造新 producer 或副作用，mixed mask 只更新允许 lane。 2. .uni 仅证明该控制 site 合同，不给其他数据指令统一广播许可。
通过条件	1. 合法 proof 被复算，负例精确拒绝，candidate 副作用集合符合 per-lane guard。 2. P1 synthetic/unit 事件不增加真实 admitted/checked 计数。
失败条件	1. mask 语义混淆、随机数据碰巧相等被当证明、被屏蔽 side effect 仍进入正常 trace。 2. 非法.uni 被执行并以硬件 trap 或具体返回值作 ISA oracle。
预期产物	1. 完整 manifest 与 source/target/capability/profile/tool/IR hash（计划产物） 2. guard_active_effect_sets.json 3. uniform_symbolic_proofs.json 4. single_mutation_preflight_results.json

TC-CTRL-003：call/ret 和 thread exit

TC-CTRL-003 项目	执行内容
源与阶段	SIG-CTRL-003；P2_REFERENCE_PROGRAM；CV-CTRL-001

TC-CTRL-003 项目	执行内容
配置与 seed	TP-REF-BASE；fixed_seed=0x534d001a（1397555226）；REFERENCE_BOUNDED
回归与当前状态	DIRECTED, RANDOM；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. 发生器/对应 unit harness 和独立审计器已实现；当前缺失，因此本 case 为 NOT_RUN/BLOCKED_MISSING_IMPLEMENTATION。 2. 锁定 PTX9.4 form/target 证据、项目 capability、IR/schema 及数值 profile；缺规则的 variant 为 BLOCKED 而非 PASS。 3. 独立 checker 不调用 generator 或 DUT 的同一合法性、decode、rounding helper；共享常量及独立边界已评审。 4. 参考程序解释/编译路线及独立 oracle 均存在并版本锁定；结果只作 P2 证据，不填 SM RTL 覆盖。
参数 1	parameter_status = 下列规模/调度值是待冻结的测试预算建议，不是 SM 时延、native 编码或当前运行结果。
参数 2	seed_strategy = 固定 seed 由主 testplan 赋值；RNG 子流和所有抽样结果写入完整 manifest，边界枚举不依赖随机偶然命中。
参数 3	call_depths = [1,2,3]
参数 4	return_site_counts = [1,2,3]
参数 5	thread_exit_patterns = ["none","one_lane","lower_half_after_sync","tail_warp_existing_lanes"]
参数 6	frame_size_candidates_bytes = [8,32,128]
参数 7	loop_rounds = [1,2,8]
初始化	1. 每 thread 有独立 frame/return 上下文和不同参数；ABI、stack alignment 及 clobber 规则冻结。 2. 退出前引用/async 义务闭合，后续 collective 成员按该条规则重新合法审计。
执行步骤	1. 生成分歧 call、多个 ret site、匹配 save/restore 及部分线程退出的合法参考程序。 2. 独立全路径检查 callee 签名、返回参数、frame lifetime 和退出后的参与者集合。 3. 参考执行器按 thread 推进 PC/stack，独立控制模型逐事件核对 return 目标与定义的结果。 4. 检查 restore 后不再访问旧对象，结束时保留合法退出集合与全部 frame 释放证据。
观察点	1. 不同 thread 即使在同 warp 从不同 ret site 返回，均回到其正确 caller。 2. 成员退出不等同当前不 active；后续 mask 满足逐指令合同。

TC-CTRL-003 项目	执行内容
通过条件	1. 参考 trace 满足独立控制/ABI/生命周期关系，无跨 thread return PC 或栈对象串扰。 2. 只比较规范保证的 clobber/结果，CC 不强制跨 call 保存。
失败条件	1. 共享 warp return PC、过期 frame 访问、非法返回目标或退出后仍有资源引用。 2. 对本来合法的 ret/exit 调度差异指定唯一不必要时序。
预期产物	1. 完整 manifest 与 source/target/capability/profile/tool/IR hash（计划产物） 2. per_thread_call_return_trace.jsonl 3. ABI_frame_object_map.json 4. exit_and_collective_legality.json

TC-CTRL-004：ITS 解锁场景

TC-CTRL-004 项目	执行内容
源与阶段	SIG-CTRL-004；P3_SM_RTL；CV-CTRL-003
配置与 seed	TP-SM-BASE；fixed_seed=0x534d001b（1397555227）；SM_PROGRESS
回归与当前状态	DIRECTED, RANDOM, STRESS；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. 发生器/对应 unit harness 和独立审计器已实现；当前缺失，因此本 case 为 NOT_RUN/BLOCKED_MISSING_IMPLEMENTATION。 2. 锁定 PTX9.4 form/target 证据、项目 capability、IR/schema 及数值 profile；缺规则的 variant 为 BLOCKED 而非 PASS。 3. 独立 checker 不调用 generator 或 DUT 的同一合法性、decode、rounding helper；共享常量及独立边界已评审。 4. 真实 SM RTL、已冻结项目机器入口/IR adapter 及正式驱动协议可用；不使用 PTX 文本字节冒充 native 编码。 5. 实际 admit/writeback/completion 与 lane/fragment/producer monitor 可用，外部服务的公平/延迟合同已配置。
参数 1	parameter_status = 下列规模/调度值是待冻结的测试预算建议，不是 SM 时延、native 编码或当前运行结果。
参数 2	seed_strategy = 固定 seed 由主 testplan 赋值；RNG 子流和所有抽样结果写入完整 manifest，边界枚举不依赖随机偶然命中。
参数 3	warp_count = 1
参数 4	handoff_round_candidates = [2,4,16]
参数 5	waiter_group_candidates = ["lane0_wait_lane1_publish","lower_half_wait_upper_half_publish"]

TC-CTRL-004 项目	执行内容
参数 6	communication = 已初始化 shared flag/data，使用被支持的 release/acquire 或对应原子协议，禁止无同步 data race。
参数 7	progress_budget = 由冻结的驻留/调度/返回公平上限及 handoff 轮数派生，不能假定任意未驻留 CTA 会解锁。
初始化	1. producer 与 waiter 均已驻留，解除依赖的线程不需要被 waiter 持有的完成资源。 2. flag 与 payload 初始化，scope/proxy 和每轮 collective 成员合法，round 标签不重复。
执行步骤	1. 先独立审计同 warp 合法 handoff 程序的 HB、必要执行者、循环进度与完整 cleanup。 2. 真实执行中让部分线程等待 flag/事件，其他线程保留可执行 producer 路径，施加允许的有限服务延迟。 3. monitor 关联每轮 producer 发布、waiter acquire 与相同 PC 多轮 collective 的动态实例。 4. 依据实际 wait-for 图与公平合同判进度，等待全部线程/资源 drain 后比较 payload 与 round 标签。
观察点	1. 等待线程没有永久阻止同 warp 可解除依赖的线程执行。 2. waiter 只在正确发布/观察后消费数据，collective 不跨 round 匹配。
通过条件	1. 所有已接收合法 handoff 在明确假设下完成，数据/HB 正确且无实例混并。 2. 超时诊断区分外部永久 stall、假设不成立和 DUT 真实等待环，非空真实 progress 证据保存。
失败条件	1. 可执行 producer 被永久压制、读取未发布数据或不同循环轮次被合成同一 collective。 2. 以任意固定周期或未驻留 CTA 假设判断 DUT，导致合法长时延误报。
预期产物	1. 完整 manifest 与 source/target/capability/profile/tool/IR hash（计划产物） 2. ITS_handoff_program.json 3. real_thread_and_HB_events.jsonl 4. wait_for_graph_and_fairness.json 5. round_payload_checks.json

20.8 GEN-COLL-01：Warp 通信与 collective 参与者

TC-COLL-001：成员与 source lane

TC-COLL-001 项目	执行内容
源与阶段	SIG-COLL-001；P1_GENERATOR_UNIT；CV-COLL-001
配置与 seed	TP-GEN-BASE；fixed_seed=0x534d001c（1397555228）；GEN_BOUNDED

TC-COLL-001 项目	执行内容
回归与当前状态	SMOKE, DIRECTED, RANDOM；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. 发生器/对应 unit harness 和独立审计器已实现；当前缺失，因此本 case 为 NOT_RUN/BLOCKED_MISSING_IMPLEMENTATION。 2. 锁定 PTX9.4 form/target 证据、项目 capability、IR/schema 及数值 profile；缺规则的 variant 为 BLOCKED 而非 PASS。 3. 独立 checker 不调用 generator 或 DUT 的同一合法性、decode、rounding helper；共享常量及独立边界已评审。
参数 1	parameter_status = 下列规模/调度值是待冻结的测试预算建议，不是 SM 时延、native 编码或当前运行结果。
参数 2	seed_strategy = 固定 seed 由主 testplan 赋值；RNG 子流和所有抽样结果写入完整 manifest，边界枚举不依赖随机偶然命中。
参数 3	warp_size = 32
参数 4	existing_lane_count_candidates = [1,2,16,31,32]
参数 5	membermask_candidates = ["0xffffffff","0x0000ffff","0x55555555","0x80000001"]
参数 6	partition_width_candidates = [1,2,4,8,16,32]
参数 7	source_delta_classes = ["0","1","partition_last","clamp_boundary","beyond_clamp_if_defined"]
参数 8	positive_mask_rule = 逐指令约束并与真实 existing/live 成员一致；不存在 lane 或违反同步 source 的情况不算正例。
初始化	1. 每 lane 输入含唯一 lane 标签，active/guard/membermask 分开初始化。 2. shfl/vote/match/redux/elect 等逐条参与、source 与 target 规则已审查，legacy 默认按 target 门禁。
执行步骤	1. 枚举 mask/partition/source 边界与 partial warp，只保留相应 form 允许的正向组合。 2. 独立 validator 重算成员/参数一致性、live/exited 与 source lane 合法性。 3. 独立逐 lane 模型生成交换/ballot/match 等期望制品，source 无效时只采用原文实际规定而非猜默认值。 4. 保存合法选择、被过滤规则及有限分母；本 P1 不执行真实 warp 会合。
观察点	1. 非连续 subset 与 partial warp 仍遵守当前指令合同，executing lane 不被漏出 membermask。 2. clamp/partition 边界的 source 选择可独立重算，不混 CUDA lane 与 packed element。

TC-COLL-001 项目	执行内容
通过条件	1. 正常样本逐条合法且逐 lane 期望一致，未定义 source 只隔离不指定值。 2. 有非空 mask/source 边界命中证据，接受/拒绝统计分开。
失败条件	1. 一个宽泛 sync 规则接纳所有 mask、source 来自未参与 lane 却强制某个值。 2. 仅有 candidate 文本便宣称会合/实际结果覆盖。
预期产物	1. 完整 manifest 与 source/target/capability/profile/tool/IR hash（计划产物） 2. member_source_boundary_cases.json 3. per_instruction_participant_audit.json 4. lane_reference_results.json

TC-COLL-002：guard/退出单因素负例

TC-COLL-002 项目	执行内容
源与阶段	SIG-COLL-002；P1_GENERATOR_UNIT；CV-COLL-002
配置与 seed	TP-GEN-BASE；fixed_seed=0x534d001d（1397555229）；GEN_BOUNDED
回归与当前状态	DIRECTED, NEGATIVE；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. 发生器/对应 unit harness 和独立审计器已实现；当前缺失，因此本 case 为 NOT_RUN/BLOCKED_MISSING_IMPLEMENTATION。 2. 锁定 PTX9.4 form/target 证据、项目 capability、IR/schema 及数值 profile；缺规则的 variant 为 BLOCKED 而非 PASS。 3. 独立 checker 不调用 generator 或 DUT 的同一合法性、decode、rounding helper；共享常量及独立边界已评审。
参数 1	parameter_status = 下列规模/调度值是待冻结的测试预算建议，不是 SM 时延、native 编码或当前运行结果。
参数 2	seed_strategy = 固定 seed 由主 testplan 赋值；RNG 子流和所有抽样结果写入完整 manifest，边界枚举不依赖随机偶然命中。
参数 3	mutation_dimensions = ["issuer_not_in_mask","one_required_member_missing","one_guard_disagrees","one_uniform_operand_disagrees","legacy_removed_target"]
参数 4	mutations_per_baseline = 1
参数 5	baseline_member_patterns = ["full_live_warp","legal_subset","legal_partial_warp"]
参数 6	injection_boundary = 独立 preflight/全程序合法化；UB 候选绝不进入正常 DUT 数值流。

TC-COLL-002 项目	执行内容
初始化	1. 每个 mutation 都有独立审计合法 baseline，所选违例在当前 form 确实禁止。 2. 已退出与暂不 active 状态明确，拒绝层/规则不是从 generator 预填 expected 值直接信任。
执行步骤	1. 先产生并独立确认合法 collective 程序及目标限制，保存 baseline 原件。 2. 一次只改变一个 member/guard/operand/target 关系，禁止修复器自动补成员后算负例成功。 3. 独立全程序审计定位破坏的规则；语义 UB、target 非法与项目关闭分别分类。 4. 确认非法 stream 未送 DUT，正常 baseline 保持可接受，输出 mutation diff 和具体拒绝证据。
观察点	1. 拒绝原因与单项变异有因果联系，而不是缺 backend/不相关错误。 2. UB 隔离不制造 trap、deadlock 时延或唯一数据 golden。
通过条件	1. 每个已定义的违规维度被正确分类/拒绝，baseline 未误杀，CV-COLL-002 按独立 audit CHECKED 采样。 2. 拒绝、blocked、未运行与 DUT 检查数严格分离。
失败条件	1. 把任何非零 exit 当目标约束命中、缺工具当 PASS 或偷偷修好 mutation。 2. 不一致 collective 被执行并以挂死/某个结果当 ISA 功能 oracle。
预期产物	1. 完整 manifest 与 source/target/capability/profile/tool/IR hash（计划产物） 2. legal_baselines.json 3. one_fault_mutation_diffs.json 4. preflight_rule_and_layer_results.json

TC-COLL-003：会合与归约模型

TC-COLL-003 项目	执行内容
源与阶段	SIG-COLL-003；P2_REFERENCE_PROGRAM；CV-COLL-001
配置与 seed	TP-REF-BASE；fixed_seed=0x534d001e（1397555230）；REFERENCE_BOUNDED
回归与当前状态	DIRECTED, RANDOM；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION

TC-COLL-003 项目	执行内容
前置条件	1. 发生器/对应 unit harness 和独立审计器已实现；当前缺失，因此本 case 为 NOT_RUN/BLOCKED_MISSING_IMPLEMENTATION。 2. 锁定 PTX9.4 form/target 证据、项目 capability、IR/schema 及数值 profile；缺规则的 variant 为 BLOCKED 而非 PASS。 3. 独立 checker 不调用 generator 或 DUT 的同一合法性、decode、rounding helper；共享常量及独立边界已评审。 4. 参考程序解释/编译路线及独立 oracle 均存在并版本锁定；结果只作 P2 证据，不填 SM RTL 覆盖。
参数 1	parameter_status = 下列规模/调度值是待冻结的测试预算建议，不是 SM 时延、native 编码或当前运行结果。
参数 2	seed_strategy = 固定 seed 由主 testplan 赋值；RNG 子流和所有抽样结果写入完整 manifest，边界枚举不依赖随机偶然命中。
参数 3	round_counts = [2,4,8]
参数 4	families = ["shfl","vote","match","redux","elect","activemask"]
参数 5	per_lane_round_tag = round*64+lane，按输入 format 合法表示
参数 6	allowed_schedule_orders = ["ordered_groups","reverse_ready_groups","interleaved_ready_groups"]
参数 7	FP_reduction_cases = 仅启用 tuple 支持时加入 NaN/±0，并按对应允许集合检查。
初始化	1. 每轮 PC 可相同但动态轮次 ID 不同；各 form 成员和退出状态经独立审计。 2. 参考解释器与独立会合模型分别存在，不能共用其内部 rendezvous 表作为 golden。
执行步骤	1. 生成同 PC 多轮 collective 及不同合法 ready-group 次序，输入携带 lane/round 标签。 2. 参考程序按允许调度推进，记录每个动态实例的真实当前 active/live/member 集合。 3. 独立模型逐实例检查交换、ballot/match/归约、elect 确定性和 activemask 当时的集合。 4. 比较每轮结果；允许的调度差异按关系或集合处理，禁止把 activemask 当未来参与者预测。
观察点	1. 相同 membermask 的 elect 按原文确定性选择，而不是每轮任意新 leader。 2. 任何结果 tag 只来自当前实例允许的 lane/round，NaN/signed-zero 采用对应形式规则。
通过条件	1. 多种允许次序下无跨轮混并、漏成员或错误结果，参考输出均满足独立关系。 2. P2 结果不计真实 SM rendezvous 覆盖；不可达/非法组合有显式原因。
失败条件	1. 只以 PC 合并实例、activemask 预测未来 active 线程、采用 host reduction 任意顺序作为唯一 golden。 2. 合法 schedule 差异被误报或共同模型内部表自证。

TC-COLL-003 项目	执行内容
预期产物	1. 完整 manifest 与 source/target/capability/profile/tool/IR hash（计划产物） 2. reference_collective_program.json 3. dynamic_member_round_events.jsonl 4. allowed_outcome_and_lane_tag_checks.json

TC-COLL-004：真实 warp 会合

TC-COLL-004 项目	执行内容
源与阶段	SIG-COLL-004；P3_SM_RTL；CV-COLL-003
配置与 seed	TP-SM-BASE；fixed_seed=0x534d001f（1397555231）；SM_PROGRESS
回归与当前状态	DIRECTED, RANDOM, STRESS；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. 发生器/对应 unit harness 和独立审计器已实现；当前缺失，因此本 case 为 NOT_RUN/BLOCKED_MISSING_IMPLEMENTATION。 2. 锁定 PTX9.4 form/target 证据、项目 capability、IR/schema 及数值 profile；缺规则的 variant 为 BLOCKED 而非 PASS。 3. 独立 checker 不调用 generator 或 DUT 的同一合法性、decode、rounding helper；共享常量及独立边界已评审。 4. 真实 SM RTL、已冻结项目机器入口/IR adapter 及正式驱动协议可用；不使用 PTX 文本字节冒充 native 编码。 5. 实际 admit/writeback/completion 与 lane/fragment/producer monitor 可用，外部服务的公平/延迟合同已配置。
参数 1	parameter_status = 下列规模/调度值是待冻结的测试预算建议，不是 SM 时延、native 编码或当前运行结果。
参数 2	seed_strategy = 固定 seed 由主 testplan 赋值；RNG 子流和所有抽样结果写入完整 manifest，边界枚举不依赖随机偶然命中。
参数 3	warp_count_candidates = [1,2,4]
参数 4	round_counts = [2,4,16]
参数 5	member_patterns = ["full_live","legal_noncontiguous_subset","legal_partial_warp"]
参数 6	planned_service_delay_cycles = [0,1,7,31]
参数 7	stall_method = 只通过合法 memory/执行域返回延迟造成 ready 差异，不篡改 DUT 内部成员或直接强迫非法 lane 事件。

TC-COLL-004 项目	执行内容
初始化	1. 真实 warp/动态实例/generation 监控可关联到源程序轮次；lane/round 输入独特。 2. 所有成员最终在规定公平假设下可到达同一实例，不生成故意缺员 UB。
执行步骤	1. 构造合法 collective 循环，独立审计 mask/guard/参数与前置访存依赖。 2. 经真实 SM 驱动执行，插入允许的有限返回延迟与执行域 stall 形成不同 lane 就绪次序。 3. rendezvous monitor 按 warp+dynamic_instance+generation 收集实际到达/释放/结果，不只看 PC。 4. 独立 lane/allowed-set oracle 检查结果并 drain；同步完成不被当作其他 async 流 full done。
观察点	1. 实例只在本条规定成员满足后完成，不提前释放/重复完成或跨 round 配对。 2. source lane/归约结果与当前 member 集合匹配，异步完成域仍独立。
通过条件	1. 非空真实会合和结果均正确，所有合法成员最终释放且 CV-COLL-003 来自实际 checked 事件。 2. 计数可关联到实际 admit/complete/checked，不以 P1 或 P2 日志补 DUT 证据。
失败条件	1. 漏参与者、跨 warp/round/generation 混并、结果过早 ready 或无限等待合法可到达成员。 2. 测试本身制造缺员 UB 却按功能失败归咎 DUT。
预期产物	1. 完整 manifest 与 source/target/capability/profile/tool/IR hash（计划产物） 2. real_rendezvous_trace.jsonl 3. dynamic_instance_provenance.json 4. lane_results_and_async_domain_checks.json

20.9 GEN-MEM-01：地址空间、别名、普通 LDST 与缓存

TC-MEM-001：空间与地址边界

TC-MEM-001 项目	执行内容
源与阶段	SIG-MEM-001；P1_GENERATOR_UNIT；CV-MEM-001
配置与 seed	TP-GEN-BASE；fixed_seed=0x534d0020（1397555232）；GEN_BOUNDED
回归与当前状态	SMOKE, DIRECTED, NEGATIVE, RANDOM, REPLAY；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. 固定 PTX9.4 清单及 SM-ARCH-S0 来源 hash 通过 preflight；独立形式/资源审计器与本阶段 harness 已实现。 2. 由 manifest 绑定精确 target、capability 及合法 form allowlist；未解析的 target/形式保持 UNRESOLVED 并阻止对应正向分支，不推定数字更大的 target 继承能力。

TC-MEM-001 项目	执行内容
参数 1	target_binding = MANIFEST_EXACT_TARGET_AND_FORM_ALLOWLIST_REQUIRED
参数 2	unresolved_capability_policy = BLOCK_POSITIVE_BRANCH_AND_REPORT_UNRESOLVED
参数 3	spaces = ["global","local","entry_param","func_param","const","shared::cta","generic"]
参数 4	object_bytes = 4096
参数 5	canary_bytes_each_side = 32
参数 6	access_bytes = [1,2,4,8,16]
参数 7	positive_offsets = ["0","natural_alignment","4096-access_bytes"]
参数 8	negative_offsets = ["4096-access_bytes+1"]
参数 9	threads = 32
参数 10	alias_variants = ["disjoint","same_object_overlapping_legal_views"]
初始化	1. 各空间对象分配独立 typed owner 与初始化字节 pattern(i)=i mod251，guard 填充独立 canary。 2. param/const 对象只生成该空间允许的访问；local 按 thread 分别建对象；generic 窗口由 fixture 定义。
执行步骤	1. 先构造每种启用空间/宽度的边界正例，按对应 form 验证对齐、读扩展、mask 与权限。 2. 生成同一对象的合法别名及不同 owner 相同 offset；独立审计从完整 range 而非截位后地址判断。 3. 从已合法 baseline 单独变异越界/对齐/权限；只送 preflight，不将 undefined 程序送数值 DUT。 4. 序列化并重读 IR，确认所有 use 由初始化支配，关闭引用并生成合法退出尾部。
观察点	1. typed 对象边界、初始化与 generic 转换独立检查均成立。 2. 每个负例恰因预定 rule 拒绝；canary/对象写 mask 期望不被地址截断改变。
通过条件	1. 所有启用正例保留原空间语义及完整 byte range；合法别名身份正确。 2. 负例诊断层/原因可重放；未解析空间标 blocked 而非 pass。
失败条件	1. 正例存在越权/未初始化读取，或不同 CTA/local owner 被错误合并。 2. 把编译器缺失、任意非零退出或其他约束错误当作预定边界负例命中。

TC-MEM-001 项目	执行内容
预期产物	1. manifest.json：锁定 2026-11-09 测试计划、源 hash、seed/子流、profile、实际 target/capability 及服务配置。 2. program.ir 与 data_objects.json：保留指令身份、CFG、typed 对象、初始化及 cleanup。 3. resource_audit.json 与 event_obligations.json：独立账本/HB/phase/单位审计。 4. case_result.json 与 coverage_delta.json：区分生成/审计/接收/完成/检查，本阶段真实执行量及 NOT_RUN/BLOCKED 原因。

TC-MEM-002：属性与 cache 指令

TC-MEM-002 项目	执行内容
源与阶段	SIG-MEM-002；P1_GENERATOR_UNIT；CV-MEM-002
配置与 seed	TP-GEN-BASE；fixed_seed=0x534d0021（1397555233）；GEN_BOUNDED
回归与当前状态	DIRECTED, NEGATIVE, REPLAY；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. 固定 PTX9.4 清单及 SM-ARCH-S0 来源 hash 通过 preflight；独立形式/资源审计器与本阶段 harness 已实现。 2. 由 manifest 绑定精确 target、capability 及合法 form allowlist；未解析的 target/形式保持 UNRESOLVED 并阻止对应正向分支，不推定数字更大的 target 继承能力。
参数 1	target_binding = MANIFEST_EXACT_TARGET_AND_FORM_ALLOWLIST_REQUIRED
参数 2	unresolved_capability_policy = BLOCK_POSITIVE_BRANCH_AND_REPORT_UNRESOLVED
参数 3	object_bytes = 256
参数 4	attributes = ["volatile","mmio","readonly","global_nc","prefetch","discard","cache_policy","priority"]
参数 5	readonly_writes_same_kernel = false
参数 6	mmio_access_count_fixture = 4
参数 7	discard_after_read_policy = 不对未重新定义内容设唯一值 oracle；重新初始化并同步后再检查

TC-MEM-002 项目	执行内容
参数 8	negative_trial = {"class":"STATIC_FRONTEND_SCHEMA_NEGATIVE","baseline":"independently approved memory-attribute candidate","single_mutation":"replace exactly one registered attribute symbol with TESTPLAN_UNREGISTERED_MEMORY_ATTR_v1","injection_layer":"SEMANTIC_IR_FRONTEND_SCHEMA","expected_detector_rule":"FRONTEND_SCHEMA_UNREGISTERED_MEMORY_ATTRIBUTE","native_encoding":"none; unknown symbolic token never lowered","DUT_execution_allowed":false}
初始化	1. 建立普通内存、kernel 只读对象及平台提供的 MMIO 寄存器 fixture，三者无别名。 2. 每个属性从独立合法 form 库选择；MMIO 实际地址/权限只由平台 fixture 提供。
执行步骤	1. 分别生成 volatile 与 MMIO 读写意图，保留访问次数/合并约束分类，不能推导 volatile 等于 MMIO。 2. 生成只读/nc/prefetch/policy 操作，确认没有给 nc/texture 添加 generic coherence 或本 kernel 写 readonly。 3. 生成 discard 及有完成义务的 priority 操作；discard 后避免唯一值检查或经合法重新定义后消费。 4. 在合法 baseline 的隔离副本中仅把一个 memory-attribute symbol 改为 TESTPLAN_UNREGISTERED_MEMORY_ATTR_v1；独立前端 schema 须按未注册属性规则拒绝，禁止降低或发送 DUT。 5. 独立分类、roundtrip 并闭合 priority group/对象引用；输出未知属性的拒绝原因。
观察点	1. 属性分类和 proxy/cache 副作用由独立规则重算。 2. MMIO、readonly lifetime 和 completion-bearing priority 不被普通 hint 修复器删除。 3. NEGATIVE trial 恰一项 symbol 变异，指定 FRONTEND_SCHEMA 层/rule 拒绝；baseline 仍合法且不猜 native 保留编码。
通过条件	1. 发生器保留对应访问/完成合同；所有已知未定值均与确定值 oracle 隔离。 2. 无平台 MMIO fixture 时该分支 blocked，不伪造物理 I/O 地址。
失败条件	1. 将 MMIO 可观察访问合并，或从 volatile 推出精确 I/O 次数。 2. discard 后断言未定义数据必须保持旧值，或吞掉带完成责任的 priority 操作。
预期产物	1. manifest.json：锁定 2026-11-09 测试计划、源 hash、seed/子流、profile、实际 target/capability 及服务配置。 2. program.ir 与 data_objects.json：保留指令身份、CFG、typed 对象、初始化及 cleanup。 3. resource_audit.json 与 event_obligations.json：独立账本/HB/phase/单位审计。 4. case_result.json 与 coverage_delta.json：区分生成/审计/接收/完成/检查，本阶段真实执行量及 NOT_RUN/BLOCKED 原因。

TC-MEM-003：memory litmus

TC-MEM-003 项目	执行内容
源与阶段	SIG-MEM-003；P2_REFERENCE_PROGRAM；CV-MEM-001
配置与 seed	TP-REF-BASE；fixed_seed=0x534d0022（1397555234）；REFERENCE_BOUNDED
回归与当前状态	DIRECTED, RANDOM, REPLAY；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. 固定 PTX9.4 清单及 SM-ARCH-S0 来源 hash 通过 preflight；独立形式/资源审计器与本阶段 harness 已实现。 2. 由 manifest 绑定精确 target、capability 及合法 form allowlist；未解析的 target/形式保持 UNRESOLVED 并阻止对应正向分支，不推定数字更大的 target 继承能力。 3. 独立语义/内存/事件模型及其允许结果集合已锁定；生成器 shadow 不能作为唯一 oracle。
参数 1	target_binding = MANIFEST_EXACT_TARGET_AND_FORM_ALLOWLIST_REQUIRED
参数 2	unresolved_capability_policy = BLOCK_POSITIVE_BRANCH_AND_REPORT_UNRESOLVED
参数 3	threads = 2
参数 4	atomic_object = shared::cta u32 x=0
参数 5	atomic_exchange_values = [1,2]
参数 6	allowed_atomic_outcomes = [{"old_t0":0,"old_t1":1,"final_x":2}, {"old_t0":2,"old_t1":0,"final_x":1}]
参数 7	litmus_modes = ["明确 HB 的 singleton 子集","并发 atomic 合法线性化","catalogue 声明的弱内存 litmus"]
参数 8	scope_proxy = 从具体合法 recipe 绑定，不将 warp 当 memory scope
初始化	1. x 及通信 data/flag 初始化为 0，通信变量/原子对象分区且带初始化版本。 2. 允许集合由独立内存/线性化模型产生；不从 host 任意调度产生唯一 gold。
执行步骤	1. 构建具有显式 HB 的确定通信子集，独立证明允许集合为单例后才启用逐值比对。 2. 构建两 thread atomic.exch 的并发程序，枚举两种合法线性化并记录返回值和最终值。 3. 加入源 catalogue 的 weak-memory litmus，探索模型允许顺序，只拒绝禁止结果。 4. 对每个调度完成相应访问与同步，收集结果/返回值后关闭对象引用并退出。
观察点	1. race-free 与 singleton 分类独立；atomic 返回值/最终值满足同一可验证线性化。 2. 模型探索不将所有弱内存行为强制成一个顺序解释器结果。
通过条件	1. singleton 子集严格匹配；atomic 示例只接受给定两组一致结果。 2. 其他 litmus 按其明确模型 allowed set/关系检查，未解析语义阻止关闭。

TC-MEM-003 项目	执行内容
失败条件	1. 仅因程序 race-free 就拒绝另一合法原子顺序，或接受无法统一线性化的返回/最终值。 2. 模型没有执行任何可达结果却报告 allowed-outcome 检查通过。
预期产物	1. manifest.json：锁定 2026-11-09 测试计划、源 hash、seed/子流、profile、实际 target/capability 及服务配置。 2. program.ir 与 data_objects.json：保留指令身份、CFG、typed 对象、初始化及 cleanup。 3. resource_audit.json 与 event_obligations.json：独立账本/HB/phase/单位审计。 4. case_result.json 与 coverage_delta.json：区分生成/审计/接收/完成/检查，本阶段真实执行量及 NOT_RUN/BLOCKED 原因。 5. memory_allowed_outcomes.json 与 linearization_witness.json：保存模型、具体观察和关系见证。

TC-MEM-004：合并与返回

TC-MEM-004 项目	执行内容
源与阶段	SIG-MEM-004；P3_SM_RTL；CV-MEM-003
配置与 seed	TP-SM-BASE；fixed_seed=0x534d0023（1397555235）；SM_PROGRESS
回归与当前状态	DIRECTED, RANDOM, STRESS, REPLAY；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. 固定 PTX9.4 清单及 SM-ARCH-S0 来源 hash 通过 preflight；独立形式/资源审计器与本阶段 harness 已实现。 2. 由 manifest 绑定精确 target、capability 及合法 form allowlist；未解析的 target/形式保持 UNRESOLVED 并阻止对应正向分支，不推定数字更大的 target 继承能力。 3. SM、对应 adapter 和 monitor 真实可用；全部事务的 owner/epoch/返回身份及服务公平合同已配置。
参数 1	target_binding = MANIFEST_EXACT_TARGET_AND_FORM_ALLOWLIST_REQUIRED
参数 2	unresolved_capability_policy = BLOCK_POSITIVE_BRANCH_AND_REPORT_UNRESOLVED
参数 3	threads = 32
参数 4	initialized_data_bytes = 8192
参数 5	element_bytes = [1,2,4,8]
参数 6	address_patterns = ["同地址广播候选","连续 lane","128B stride","不同对象合法别名"]

TC-MEM-004 项目	执行内容
参数 7	logical_response_fragments = [1,2,4]
参数 8	delivery_orders = ["顺序","逆序","合法随机"]
参数 9	mmio_fixture_accesses = 4
初始化	1. data 按 lane/offset 生成唯一可核对 pattern；MMIO fixture 与普通内存分离。 2. 预留整目的寄存器及片段归属，SoC 合法返回/重放身份已注册。
执行步骤	1. 生成不同 lane 地址分布与宽度，先审计空间、范围及每 lane 副作用。 2. 实际驱动 SM，服务模型按合法顺序/逆序/随机返回片段，穿插 4 次已定义 MMIO 访问。 3. 首片返回时检查整目的仍未 ready；全部片段 ack 后才允许后继消费者。 4. 等待全部 ordinary 访问/副作用关闭，核对 canary、MMIO 计数并 drain。
观察点	1. 每 lane/byte 返回对应原指令/owner，合并/replay 不能丢失或重复副作用。 2. MMIO 访问次数与顺序仅依据 fixture 的对应 form 合同检查。
通过条件	1. 结果、mask 及源/目的身份正确；所有完成均恰一次且整值 ready 时点正确。 2. 实际 checked trace 和零泄漏账本存在，不以接口输出文本计执行。
失败条件	1. 首 fragment 即整体 ready、tag 错配，或普通 coalescing 策略合并了不允许的 MMIO 访问。 2. 外部未按服务合同返回时仍无分类地归咎 DUT 或报告 pass。
预期产物	1. manifest.json：锁定 2026-11-09 测试计划、源 hash、seed/子流、profile、实际 target/capability 及服务配置。 2. program.ir 与 data_objects.json：保留指令身份、CFG、typed 对象、初始化及 cleanup。 3. resource_audit.json 与 event_obligations.json：独立账本/HB/phase/单位审计。 4. case_result.json 与 coverage_delta.json：区分生成/审计/接收/完成/检查，本阶段真实执行量及 NOT_RUN/BLOCKED 原因。 5. sm_admit_return_trace.jsonl、lane_byte_scoreboard.json、mmio_observations.json。

20.10 GEN-SMEM-01：SMEM owner、共享服务与 RMW

TC-SMEM-001：多 CTA 与联合预算

TC-SMEM-001 项目	执行内容
源与阶段	SIG-SMEM-001；P1_GENERATOR_UNIT；CV-SMEM-001
配置与 seed	TP-GEN-BASE；fixed_seed=0x534d0024（1397555236）；GEN_BOUNDED

TC-SMEM-001 项目	执行内容
回归与当前状态	SMOKE, DIRECTED, NEGATIVE, REPLAY；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. 固定 PTX9.4 清单及 SM-ARCH-S0 来源 hash 通过 preflight；独立形式/资源审计器与本阶段 harness 已实现。 2. 由 manifest 绑定精确 target、capability 及合法 form allowlist；未解析的 target/形式保持 UNRESOLVED 并阻止对应正向分支，不推定数字更大的 target 继承能力。
参数 1	target_binding = MANIFEST_EXACT_TARGET_AND_FORM_ALLOWLIST_REQUIRED
参数 2	unresolved_capability_policy = BLOCK_POSITIVE_BRANCH_AND_REPORT_UNRESOLVED
参数 3	smem_pool_bytes = 262144
参数 4	cta_slots_limit = 8
参数 5	warp_slots_limit = 64
参数 6	launch_scenarios = [{"ctas":4,"visible_bytes_each":65536,"reserved_bytes_each":0}, {"ctas":5,"visible_bytes_each":65536,"reserved_bytes_each":0}, {"ctas":8,"visible_bytes_each":31744,"reserved_bytes_each":1024}]
参数 7	warps_each_cta = 4
参数 8	RF_budget = R_SM 及编译分配取整后联合检查
参数 9	same_local_offset = 0
初始化	1. 用 S0 资源表建立空 pool；单 CTA target/ABI 上限若小于候选请求，先标该正向变体 unresolved/disabled。 2. visible 与 allocation/reserved 区间分账，8 个 owner generation 不同。
执行步骤	1. 依次实例化 4×64KiB 恰好满、5×64KiB 超预算和 8×(31KiB+1KiBreserved)场景。 2. 独立计算 warp、RF、SMEM 真实 reservation 与 target 单 CTA 限制，未全部获得前不发布 launch。 3. 在获准 owner 的 offset0 写不同值，检查逻辑对象隔离及程序不能访问 reserved。 4. 模拟所有已接受 CTA 正常结束并关闭引用，再回收区间/递增 generation。
观察点	1. 有 CTA slot 不等于通过 shared/RF 预算；拒绝或等待不能留下半 admit owner。 2. 同 offset 不同 owner 不别名；reserved 不与已声明 barrier 重复计算。
通过条件	1. 每个场景的联合预算与独立 interval 模型一致，超预算 CTA 未运行。 2. 所有已接受 owner 最终只回收一次，未偷用任意 target 单 CTA 可见容量。

TC-SMEM-001 项目	执行内容
失败条件	1. 8 槽被当成 8 份 256KiB，或 pool 有空间就无视 target 单 CTA 上限。 2. 失败 launch 泄漏 RF/shared reservation，或用户数据写入 reserved。
预期产物	1. manifest.json：锁定 2026-11-09 测试计划、源 hash、seed/子流、profile、实际 target/capability 及服务配置。 2. program.ir 与 data_objects.json：保留指令身份、CFG、typed 对象、初始化及 cleanup。 3. resource_audit.json 与 event_obligations.json：独立账本/HB/phase/单位审计。 4. case_result.json 与 coverage_delta.json：区分生成/审计/接收/完成/检查，本阶段真实执行量及 NOT_RUN/BLOCKED 原因。

TC-SMEM-002：RMW 与原子能力

TC-SMEM-002 项目	执行内容
源与阶段	SIG-SMEM-002；P1_GENERATOR_UNIT；CV-SMEM-002
配置与 seed	TP-GEN-BASE；fixed_seed=0x534d0025（1397555237）；GEN_BOUNDED
回归与当前状态	DIRECTED, NEGATIVE, REPLAY；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. 固定 PTX9.4 清单及 SM-ARCH-S0 来源 hash 通过 preflight；独立形式/资源审计器与本阶段 harness 已实现。 2. 由 manifest 绑定精确 target、capability 及合法 form allowlist；未解析的 target/形式保持 UNRESOLVED 并阻止对应正向分支，不推定数字更大的 target 继承能力。
参数 1	target_binding = MANIFEST_EXACT_TARGET_AND_FORM_ALLOWLIST_REQUIRED
参数 2	unresolved_capability_policy = BLOCK_POSITIVE_BRANCH_AND_REPORT_UNRESOLVED
参数 3	shared_bytes = 256
参数 4	word_bytes = 4
参数 5	initial_word_hex = 11223344
参数 6	byte_offsets = [0,1,2,3]
参数 7	write_masks = ["单 byte","两个互不重叠 byte"]
参数 8	atomic_variants = ["合法 S32 scalar","合法 packed FP 逐元素"]
参数 9	b128_shared_capability = false
参数 10	negative_b128_request = 只供 preflight 拒绝，不用四 store 替代

TC-SMEM-002 项目	执行内容
初始化	1. 共享 words 和 guard 初始化；合法 atomic 对象按实际宽度对齐且不与 ordinary 并发访问混用。 2. packed 元素初值为可精确表示的小整数，独立模型按每 scalar element 提供原子性。
执行步骤	1. 构建 byte/half masked write 意图并独立计算 little-endian merge，不假设宏有免费 byte 端口。 2. 构建 scalar 与 packed atomic 合法程序，保留各元素/返回义务及必要 HB。 3. 从合法候选变异为关闭的 b128 shared CAS，检查 capability 拒绝且不发出非原子降级序列。 4. 对合法程序完成写回/同步和对象回收，独立审计所有 RMW 责任。
观察点	1. 原子宽度及 packed 逐元素粒度不被扩大或缩窄。 2. b128 关闭分支在指定层拒绝，byte mask 期望不受地址合并影响。
通过条件	1. 合法正例的 mask、数值与义务完整；负例准确报告缺失 capability。 2. 无用普通四 store 模拟 b128 整体原子的假支持。
失败条件	1. RMW 丢写、packed 被强制 whole-vector 原子，或 b128 silently lower 为无互斥 store。 2. 缺少合法 form 来源仍接受任意 atomic op/type。
预期产物	1. manifest.json：锁定 2026-11-09 测试计划、源 hash、seed/子流、profile、实际 target/capability 及服务配置。 2. program.ir 与 data_objects.json：保留指令身份、CFG、typed 对象、初始化及 cleanup。 3. resource_audit.json 与 event_obligations.json：独立账本/HB/phase/单位审计。 4. case_result.json 与 coverage_delta.json：区分生成/审计/接收/完成/检查，本阶段真实执行量及 NOT_RUN/BLOCKED 原因。

TC-SMEM-003：可见性与 collector 版本

TC-SMEM-003 项目	执行内容
源与阶段	SIG-SMEM-003；P2_REFERENCE_PROGRAM；CV-SMEM-001
配置与 seed	TP-REF-BASE；fixed_seed=0x534d0026（1397555238）；REFERENCE_BOUNDED
回归与当前状态	DIRECTED, RANDOM, REPLAY；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. 固定 PTX9.4 清单及 SM-ARCH-S0 来源 hash 通过 preflight；独立形式/资源审计器与本阶段 harness 已实现。 2. 由 manifest 绑定精确 target、capability 及合法 form allowlist；未解析的 target/形式保持 UNRESOLVED 并阻止对应正向分支，不推定数字更大的 target 继承能力。 3. 独立语义/内存/事件模型及其允许结果集合已锁定；生成器 shadow 不能作为唯一 oracle。

TC-SMEM-003 项目	执行内容
参数 1	target_binding = MANIFEST_EXACT_TARGET_AND_FORM_ALLOWLIST_REQUIRED
参数 2	unresolved_capability_policy = BLOCK_POSITIVE_BRANCH_AND_REPORT_UNRESOLVED
参数 3	ctas = 1
参数 4	warps = 4
参数 5	A_view = F16 M128xK16
参数 6	B_view = F16 K16xN64
参数 7	A_versions = ["v0=1","v1=2"]
参数 8	B_value = 1
参数 9	C_value = 0
参数 10	event_orders = ["bankcommit→enqueue→apply→publish","apply 延后且旧 snapshot 继续使用"]
参数 11	collector_policy = future lookup 失效确认与 active immutable snapshot 独立
初始化	1. A/B/C 均初始化，建立 v0 源版本和命令私有 A snapshot；仅使用已解析合法 f16 recipe。 2. proxy/线程交接由模型的 canonical recipe 表示，尚未把 enqueue 当实际失效。
执行步骤	1. 建立 v0 矩阵操作及对应源引用，记录 snapshot 不变量。 2. 在合法 A-read/线程交接后改写原 A 到 v1，保留 B 和 full/D 责任。 3. 分别探索失效入队、实际 apply 与 proxy publish 的允许间隔，未来命令只在发布后读取 v1。 4. 比较旧命令 v0 与新命令 v1 的结果关系，关闭两命令/full 与源引用后回收对象。
观察点	1. 失效 enqueue 不能产生 future-visible ack；publish 只覆盖 captured cutoff。 2. 活动 snapshot 不因原 A 写入被覆盖，B 的可能重读责任未被 A-read 释放。
通过条件	1. 旧命令使用 v0、新命令在合法发布后使用 v1，HB 与版本模型一致。 2. 参考事件所有允许顺序均无提前复用或遗留引用。
失败条件	1. future reuse 仍命中 v0，或原 A 更新销毁 active snapshot。 2. 以 A-read 释放整个 SMEM slice/B/D/full token。

TC-SMEM-003 项目	执行内容
预期产物	1. manifest.json：锁定 2026-11-09 测试计划、源 hash、seed/子流、profile、实际 target/capability 及服务配置。 2. program.ir 与 data_objects.json：保留指令身份、CFG、typed 对象、初始化及 cleanup。 3. resource_audit.json 与 event_obligations.json：独立账本/HB/phase/单位审计。 4. case_result.json 与 coverage_delta.json：区分生成/审计/接收/完成/检查，本阶段真实执行量及 NOT_RUN/BLOCKED 原因。 5. reference_visibility_trace.jsonl、collector_version_witness.json。

TC-SMEM-004：共享竞争及 scrub

TC-SMEM-004 项目	执行内容
源与阶段	SIG-SMEM-004；P3_SM_RTL；CV-SMEM-003
配置与 seed	TP-SM-STRESS；fixed_seed=0x534d0027（1397555239）；SM_PROGRESS
回归与当前状态	DIRECTED, STRESS, REPLAY；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. 固定 PTX9.4 清单及 SM-ARCH-S0 来源 hash 通过 preflight；独立形式/资源审计器与本阶段 harness 已实现。 2. 由 manifest 绑定精确 target、capability 及合法 form allowlist；未解析的 target/形式保持 UNRESOLVED 并阻止对应正向分支，不推定数字更大的 target 继承能力。 3. SM、对应 adapter 和 monitor 真实可用；全部事务的 owner/epoch/返回身份及服务公平合同已配置。
参数 1	target_binding = MANIFEST_EXACT_TARGET_AND_FORM_ALLOWLIST_REQUIRED
参数 2	unresolved_capability_policy = BLOCK_POSITIVE_BRANCH_AND_REPORT_UNRESOLVED
参数 3	clients = ["LSU","TMA","Tensor","peer","barrier"]
参数 4	shared_object_bytes = 4096
参数 5	bank_profile = 显式选择 v1.1 项目 64x32bit1R1W 配置，或绑定实际实现配置
参数 6	hotspot_word_offsets = [0,256]
参数 7	ecc_modes = ["OFF","ON_IF_IMPLEMENTED"]
参数 8	scrub_sequence = ["旧 read 纠正责任","new store","迟到 scrub 入队","锁内重读当前 word"]

TC-SMEM-004 项目	执行内容
初始化	1. 多 owner buffer/canary 和 barrier 对象合法初始化；所有 client 带正确 generation。 2. ECC 可选分支要求真实模型/实现存在，否则该分支 blocked；服务公平和有限回压 schedule 固定。
执行步骤	1. 对共享热点与分散地址混合施加 5 类 client 压力，不给每 client 额外免费峰值。 2. 制造旧 corrected-read 后新 store，再让旧 scrub ticket 迟到入队，验证锁内重读。 3. 压满本类 response 预算并保留 control 完成进度；允许已有返回与 barrier RMW 继续。 4. 停止新增压力、解除有限回压，drain 所有 owner/client/RMW/scrub/visibility 责任后回收。
观察点	1. scrub 没有写回旧 cached corrected 值，owner byte range 与 RMW 原子宽度正确。 2. response/visibility 控制责任不依赖消费者先执行，公平假设实际兑现。
通过条件	1. 当前 word 保持新 store 数据或其当前 ECC 纠正结果，无旧值覆盖或跨 owner 别名。 2. 在服务合同满足下控制/返回前进并最终 drain，所有 ack 恰一次。
失败条件	1. old scrub 覆盖新值、RMW 撕裂或 client 占光控制保留进度形成环等。 2. 将 ECC 未实现/服务永久 stall 当作已运行通过。
预期产物	1. manifest.json：锁定 2026-11-09 测试计划、源 hash、seed/子流、profile、实际 target/capability 及服务配置。 2. program.ir 与 data_objects.json：保留指令身份、CFG、typed 对象、初始化及 cleanup。 3. resource_audit.json 与 event_obligations.json：独立账本/HB/phase/单位审计。 4. case_result.json 与 coverage_delta.json：区分生成/审计/接收/完成/检查，本阶段真实执行量及 NOT_RUN/BLOCKED 原因。 5. owner_rmw_visibility_trace.jsonl、scrub_race_witness.json、wait_for_graph.json。

20.11 GEN-SYNC-01：Barrier、mbarrier 布局/phase 与内存顺序

TC-SYNC-001：mbarrier 生命周期

TC-SYNC-001 项目	执行内容
源与阶段	SIG-SYNC-001；P1_GENERATOR_UNIT；CV-SYNC-001
配置与 seed	TP-GEN-BASE；fixed_seed=0x534d0028（1397555240）；GEN_BOUNDED
回归与当前状态	SMOKE, DIRECTED, NEGATIVE, REPLAY；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION

TC-SYNC-001 项目	执行内容
前置条件	1. 固定 PTX9.4 清单及 SM-ARCH-S0 来源 hash 通过 preflight；独立形式/资源审计器与本阶段 harness 已实现。 2. 由 manifest 绑定精确 target、capability 及合法 form allowlist；未解析的 target/形式保持 UNRESOLVED 并阻止对应正向分支，不推定数字更大的 target 继承能力。
参数 1	target_binding = MANIFEST_EXACT_TARGET_AND_FORM_ALLOWLIST_REQUIRED
参数 2	unresolved_capability_policy = BLOCK_POSITIVE_BRANCH_AND_REPORT_UNRESOLVED
参数 3	barrier_alignment_bytes = 8
参数 4	layouts = ["v0","v1"]
参数 5	arrival_counts_v0 = [1,2,1048575]
参数 6	arrival_counts_v1 = [1,2,510,511]
参数 7	invalid_arrival_v1 = [512]
参数 8	tx_boundary_model_values = [-1048575,0,1048575]
参数 9	counter_boundary_mode = 大计数用符号计数单元模型，不伪造百万驻留 threads
参数 10	phase_count = 3
初始化	1. barrier 独占 shared 对象处于未 init；每变体拥有独立 owner/phase ID。 2. target/layout/操作合法性已解析，typed tx units 按 recipe 绑定。
执行步骤	1. 生成各 layout 合法 init/use 序列，明确 arrival 与 expect/complete 数目及单位。 2. 采用原子 arrive.expect_tx 或先 expect 并保留 arrival 后 arrive，探索 complete 早/晚到。 3. 执行 wait/phase 观察与合法 arrive_drop 变体，再按复用规则 inval；大边界使用有界符号计数。 4. 对 v1=512 或 active 对象普通写/重复 init 作单因素静态负例，不送正常执行。
观察点	1. 独立计数器保留 signed tx 并检查 layout 上限，不按 thread 槽限制符号计数模型。 2. active 对象生命周期、phase 与 inval 前引用准确；无提前 phase 推进。
通过条件	1. 所有合法模板按正确 phase 闭合；负例命中指定 layout/lifetime 规则。 2. 0 执行、无规则或符号模型不覆盖的分支不算通过。
失败条件	1. v1 允许 512、tx units 混淆、active barrier 被普通 store 重置。 2. 先最后 arrival 再 expect 导致 phase 提前结束，或以 parity 单 bit 替代逻辑 phase。

TC-SYNC-001 项目	执行内容
预期产物	1. manifest.json：锁定 2026-11-09 测试计划、源 hash、seed/子流、profile、实际 target/capability 及服务配置。 2. program.ir 与 data_objects.json：保留指令身份、CFG、typed 对象、初始化及 cleanup。 3. resource_audit.json 与 event_obligations.json：独立账本/HB/phase/单位审计。 4. case_result.json 与 coverage_delta.json：区分生成/审计/接收/完成/检查，本阶段真实执行量及 NOT_RUN/BLOCKED 原因。

TC-SYNC-002：phase 与报告

TC-SYNC-002 项目	执行内容
源与阶段	SIG-SYNC-002；P1_GENERATOR_UNIT；CV-SYNC-002
配置与 seed	TP-GEN-BASE；fixed_seed=0x534d0029（1397555241）；GEN_BOUNDED
回归与当前状态	DIRECTED, NEGATIVE, REPLAY；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. 固定 PTX9.4 清单及 SM-ARCH-S0 来源 hash 通过 preflight；独立形式/资源审计器与本阶段 harness 已实现。 2. 由 manifest 绑定精确 target、capability 及合法 form allowlist；未解析的 target/形式保持 UNRESOLVED 并阻止对应正向分支，不推定数字更大的 target 继承能力。
参数 1	target_binding = MANIFEST_EXACT_TARGET_AND_FORM_ALLOWLIST_REQUIRED
参数 2	unresolved_capability_policy = BLOCK_POSITIVE_BRANCH_AND_REPORT_UNRESOLVED
参数 3	layout = v1
参数 4	phase_count = 4
参数 5	logical_phase_ids = [0,1,2,3]
参数 6	observations = ["primary 真 conditional 假","try_wait false","成功项不清旧错误"]
参数 7	report_value = opaque b8，只按已定义 predicate/允许报告关系检查
初始化	1. 每个 phase 有已初始化对象与独立逻辑 ID；错误报告由合法 producer 模型产生。 2. 保留 pending arrival/expect 登记以防 phase 过早结束，不要求 conditional 在失败时最终成功。

TC-SYNC-002 项目	执行内容
执行步骤	1. 构造一个合法报告失败 phase，使 primary 终点到达而 conditional 条件不成立。 2. 插入允许的 try_wait false 观察，不把此时 report 读作有效值。 3. 交错成功/失败 producer，再按合法观察及生命周期推进后续 phase 并覆盖 parity 复用。 4. 在定义的错误处理路径关闭 primary/报告及资源责任，必要复用先 inval，禁止无限等失败 conditional。
观察点	1. primary/conditional 分别检查，逻辑 phase 而非仅 parity 决定本次观察归属。 2. 成功事件不擦除本 phase 已有报告，false 不释放 payload 消费权。
通过条件	1. 报告/phase/false 语义与独立模型一致，失败 conditional 不会制造无解除条件 cleanup。 2. 四 phase 的合法复用和等待身份可重放。
失败条件	1. 把 primary 成功等同 conditional 成功，或比较 false 时未定义 report。 2. 旧 phase 等待误认新 phase，或错误 cleanup 一直等不可能的 conditional。
预期产物	1. manifest.json：锁定 2026-11-09 测试计划、源 hash、seed/子流、profile、实际 target/capability 及服务配置。 2. program.ir 与 data_objects.json：保留指令身份、CFG、typed 对象、初始化及 cleanup。 3. resource_audit.json 与 event_obligations.json：独立账本/HB/phase/单位审计。 4. case_result.json 与 coverage_delta.json：区分生成/审计/接收/完成/检查，本阶段真实执行量及 NOT_RUN/BLOCKED 原因。

TC-SYNC-003：fence 与 HB 模板

TC-SYNC-003 项目	执行内容
源与阶段	SIG-SYNC-003；P2_REFERENCE_PROGRAM；CV-SYNC-001
配置与 seed	TP-REF-BASE；fixed_seed=0x534d002a（1397555242）；REFERENCE_BOUNDED
回归与当前状态	DIRECTED, NEGATIVE, RANDOM, REPLAY；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. 固定 PTX9.4 清单及 SM-ARCH-S0 来源 hash 通过 preflight；独立形式/资源审计器与本阶段 harness 已实现。 2. 由 manifest 绑定精确 target、capability 及合法 form allowlist；未解析的 target/形式保持 UNRESOLVED 并阻止对应正向分支，不推定数字更大的 target 继承能力。 3. 独立语义/内存/事件模型及其允许结果集合已锁定；生成器 shadow 不能作为唯一 oracle。
参数 1	target_binding = MANIFEST_EXACT_TARGET_AND_FORM_ALLOWLIST_REQUIRED

TC-SYNC-003 项目	执行内容
参数 2	unresolved_capability_policy = BLOCK_POSITIVE_BRANCH_AND_REPORT_UNRESOLVED
参数 3	recipes = ["generic→async 共享发布","tensor-map update→发布→TMA","fabric 与本地 generic 交接"]
参数 4	producer_consumer_threads = 2
参数 5	payload_bytes = 256
参数 6	scope_variants = ["具体 recipe 允许的 cta","启用时 cluster/gpu/sys"]
参数 7	negative_mutations = ["删除必要 proxy fence","反转方向","替换为无关 thread barrier"]
初始化	1. payload、控制 flag、descriptor/tensor-map 对象按各自空间初始化且互不混用。 2. 独立内存/proxy 模型包含所用 recipe 与 target 条件。
执行步骤	1. 逐 recipe 生成 producer 写入、正确线程/proxy 发布与 consumer 访问，保持 scope/方向和 cutoff。 2. 探索后发写不延长先前 fence cutoff 的允许顺序。 3. 从合法程序单因素删除/反转必要边，验证 resource/HB 审计拒绝。 4. 在参考模型完成所有责任并 drain 后比较可见性关系，不以全局清队列实现 fence。
观察点	1. consumer 访问被正确 HB/可见性边支配，不以 barrier 或 queue 接收代替。 2. scope 只决定同步范围，不扩大 shared owner 访问权。
通过条件	1. 合法参考程序只产生允许结果，mutant 准确命中缺失方向/HB 规则。 2. 无无关 family 完成被当作该 proxy 发布。
失败条件	1. 普通 fence 被视作所有 proxy 万能发布，或系统 scope 越权访问 shared。 2. oracle 未经模型证明就将弱内存关系强制唯一顺序结果。
预期产物	1. manifest.json：锁定 2026-11-09 测试计划、源 hash、seed/子流、profile、实际 target/capability 及服务配置。 2. program.ir 与 data_objects.json：保留指令身份、CFG、typed 对象、初始化及 cleanup。 3. resource_audit.json 与 event_obligations.json：独立账本/HB/phase/单位审计。 4. case_result.json 与 coverage_delta.json：区分生成/审计/接收/完成/检查，本阶段真实执行量及 NOT_RUN/BLOCKED 原因。 5. proxy_hb_graph.json、cutoff_and_visibility_witness.json。

TC-SYNC-004：对象更新与等待

TC-SYNC-004 项目	执行内容
源与阶段	SIG-SYNC-004；P3_SM_RTL；CV-SYNC-003
配置与 seed	TP-SM-BASE；fixed_seed=0x534d002b（1397555243）；SM_PROGRESS
回归与当前状态	DIRECTED, STRESS, REPLAY；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. 固定 PTX9.4 清单及 SM-ARCH-S0 来源 hash 通过 preflight；独立形式/资源审计器与本阶段 harness 已实现。 2. 由 manifest 绑定精确 target、capability 及合法 form allowlist；未解析的 target/形式保持 UNRESOLVED 并阻止对应正向分支，不推定数字更大的 target 继承能力。 3. SM、对应 adapter 和 monitor 真实可用；全部事务的 owner/epoch/返回身份及服务公平合同已配置。
参数 1	target_binding = MANIFEST_EXACT_TARGET_AND_FORM_ALLOWLIST_REQUIRED
参数 2	unresolved_capability_policy = BLOCK_POSITIVE_BRANCH_AND_REPORT_UNRESOLVED
参数 3	barrier_alignment_bytes = 8
参数 4	expected_arrivals = 1
参数 5	event_schedule = ["event 入队","延后对象 RMW","对象 commit","ack","True acquire 观察"]
参数 6	try_wait_probes = ["对象 commit 前","对象 commit 后"]
初始化	1. 初始化 barrier 及关联 payload；命令/事件引用持有稳定 owner/phase metadata。 2. 服务允许有限延迟 RMW 并保证最终前进，不规定固定周期。
执行步骤	1. 发起可产生真实完成事件的已启用操作，先使 event 可接收而对象 RMW 尚未提交。 2. 在对象更新前观察 wait 与消费者门禁，不能因 event enqueue 即可消费。 3. 让 barrier agent 真正原子更新 SMEM 对象并 ack，消费者完成正确 True acquire 观察。 4. 核对 payload 消费/phase 并 drain 所有事件、对象引用和寄存器返回。
观察点	1. 事件接收、对象 commit、ack、成功观察四事件按合同区分。 2. metadata 在 ack 前稳定，重试不会重复 arrive 或 complete。
通过条件	1. 对象更新前无提前消费，更新和正确观察后数据可用。 2. exactly-once 更新/返回且退出无悬空 owner 引用。
失败条件	1. FIFO 接收触发假完成，或 True/false 判断读错 owner/phase。 2. 元数据提前复用、重复计数或未兑现公平导致无法分类的 hang。

TC-SYNC-004 项目	执行内容
预期产物	1. manifest.json：锁定 2026-11-09 测试计划、源 hash、seed/子流、profile、实际 target/capability 及服务配置。 2. program.ir 与 data_objects.json：保留指令身份、CFG、typed 对象、初始化及 cleanup。 3. resource_audit.json 与 event_obligations.json：独立账本/HB/phase/单位审计。 4. case_result.json 与 coverage_delta.json：区分生成/审计/接收/完成/检查，本阶段真实执行量及 NOT_RUN/BLOCKED 原因。 5. barrier_object_commit_trace.jsonl、wait_observation_scoreboard.json。

TC-SYNC-005：remote 规则与 release 进度

TC-SYNC-005 项目	执行内容
源与阶段	SIG-SYNC-005；P4_SOC；CV-SYNC-003
配置与 seed	TP-SOC-BASE；fixed_seed=0x534d002c（1397555244）；SOC_PROGRESS
回归与当前状态	DIRECTED, NEGATIVE, STRESS, REPLAY；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. 固定 PTX9.4 清单及 SM-ARCH-S0 来源 hash 通过 preflight；独立形式/资源审计器与本阶段 harness 已实现。 2. 由 manifest 绑定精确 target、capability 及合法 form allowlist；未解析的 target/形式保持 UNRESOLVED 并阻止对应正向分支，不推定数字更大的 target 继承能力。 3. 跨 SM/SoC 运行时、对应服务及取消/返回模型可用；未实现系统能力保持 blocked。
参数 1	target_binding = MANIFEST_EXACT_TARGET_AND_FORM_ALLOWLIST_REQUIRED
参数 2	unresolved_capability_policy = BLOCK_POSITIVE_BRANCH_AND_REPORT_UNRESOLVED
参数 3	ctas = 2
参数 4	sms = 2
参数 5	barrier_owner_rank = 1
参数 6	barrier_alignment_bytes = 8
参数 7	allowed_remote_operations = ["source 表允许的 arrive/drop/expect/complete"]
参数 8	consumer_wait_location = rank1 本地 barrier
参数 9	static_mutations = ["remote wait","remote init","remote inval"]

TC-SYNC-005 项目	执行内容
初始化	1. 两个 CTA 已共同 admit 且目标 rank1 存活；barrier 由本地 owner 按合法 layout 初始化。 2. expect/arrival/tx 数目和单位按具体 remote recipe 预先配平。
执行步骤	1. 由 rank0 对 rank1 合法 barrier 更新，保持实际 owner 引用和目标 offset。 2. 将控制更新与大量 DSM 数据返回交错，保留独立控制/返回进度。 3. rank1 只在自己的 local 对象正确等待；单独对 remote wait/init/inval 执行 preflight 负例。 4. 等待全部远端更新/数据/返回，解除 owner pin 后按 cluster 退出合同回收。
观察点	1. remote 更新经真实对象原子 commit 后 ack，消费者未执行非法 remote wait。 2. 退出前远端引用归零，控制信用不被数据反压永久占满。
通过条件	1. 正向 remote/local 协议闭合且计数正确；静态负例无进入正常 DUT 数值检查。 2. 延迟更新不写回已重用 owner。
失败条件	1. 通过裸 SM 号/XOR 构造目标，或提前回收目标 barrier/shared。 2. 将 remote 不支持操作当规范要求硬件 trap 或普通成功。
预期产物	1. manifest.json：锁定 2026-11-09 测试计划、源 hash、seed/子流、profile、实际 target/capability 及服务配置。 2. program.ir 与 data_objects.json：保留指令身份、CFG、typed 对象、初始化及 cleanup。 3. resource_audit.json 与 event_obligations.json：独立账本/HB/phase/单位审计。 4. case_result.json 与 coverage_delta.json：区分生成/审计/接收/完成/检查，本阶段真实执行量及 NOT_RUN/BLOCKED 原因。 5. soc_remote_barrier_trace.jsonl、owner_reference_ledger.json。

20.12 GEN-ASYNC-01：cp.async、TMA 与 bulk/priority 完成

TC-ASYNC-001：group 与方向约束

TC-ASYNC-001 项目	执行内容
源与阶段	SIG-ASYNC-001；P1_GENERATOR_UNIT；CV-ASYNC-001
配置与 seed	TP-GEN-BASE；fixed_seed=0x534d002d（1397555245）；GEN_BOUNDED
回归与当前状态	SMOKE, DIRECTED, NEGATIVE, REPLAY；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION

TC-ASYNC-001 项目	执行内容
前置条件	1. 固定 PTX9.4 清单及 SM-ARCH-S0 来源 hash 通过 preflight；独立形式/资源审计器与本阶段 harness 已实现。 2. 由 manifest 绑定精确 target、capability 及合法 form allowlist；未解析的 target/形式保持 UNRESOLVED 并阻止对应正向分支，不推定数字更大的 target 继承能力。
参数 1	target_binding = MANIFEST_EXACT_TARGET_AND_FORM_ALLOWLIST_REQUIRED
参数 2	unresolved_capability_policy = BLOCK_POSITIVE_BRANCH_AND_REPORT_UNRESOLVED
参数 3	families = ["classic_cp_async","bulk_copy_reduce","async_store_reduce","bulk_priority","st_bulk"]
参数 4	classic_fragment_bytes = [4,8,16]
参数 5	group_counts = [0,1,2]
参数 6	st_bulk_size_bytes = [8,16,64]
参数 7	st_bulk_initval = 0
参数 8	object_bytes = 512
参数 9	direction_binding = 逐 form 合法方向，不自行扩展语法
初始化	1. 各 source/destination 对象初始化且按所选 form 对齐；每 family 建立独立 issuer/group。 2. classic/bulk/TMA 与 priority 能力分开，barrier/group 对象按 recipe 声明。
执行步骤	1. 遍历合法方向和空/非空 group，生成相应 commit/wait 或 barrier 完成链。 2. 对 st.bulk 只生成合法 shared 置零尺寸/值，对 priority 保留 bulk 完成义务。 3. 变异为跨 family wait 或非法方向，独立审计应拒绝而不是自动换成另一 wait。 4. 构造完整 cleanup，保存每条义务的 source 可重用点和目的终点。
观察点	1. classic/bulk/WGMMA/tcgen05 group 互不替代，empty group 按自身规则处理。 2. st.bulk 不被打印成 global memset 或任意 init 值。
通过条件	1. 所有合法 recipe 方向、group 与收尾完整；指定 mutant 准确拒绝。 2. 未解析方向/能力记录 blocked，不静默降级。
失败条件	1. 统一 async_done 清全部 family，或丢弃 priority completion。 2. 负例被修成正例后仍算原规则拒绝覆盖。

TC-ASYNC-001 项目	执行内容
预期产物	1. manifest.json：锁定 2026-11-09 测试计划、源 hash、seed/子流、profile、实际 target/capability 及服务配置。 2. program.ir 与 data_objects.json：保留指令身份、CFG、typed 对象、初始化及 cleanup。 3. resource_audit.json 与 event_obligations.json：独立账本/HB/phase/单位审计。 4. case_result.json 与 coverage_delta.json：区分生成/审计/接收/完成/检查，本阶段真实执行量及 NOT_RUN/BLOCKED 原因。

TC-ASYNC-002：TMA descriptor 与 report

TC-ASYNC-002 项目	执行内容
源与阶段	SIG-ASYNC-002；P1_GENERATOR_UNIT；CV-ASYNC-002
配置与 seed	TP-GEN-BASE；fixed_seed=0x534d002e（1397555246）；GEN_BOUNDED
回归与当前状态	DIRECTED, NEGATIVE, REPLAY；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. 固定 PTX9.4 清单及 SM-ARCH-S0 来源 hash 通过 preflight；独立形式/资源审计器与本阶段 harness 已实现。 2. 由 manifest 绑定精确 target、capability 及合法 form allowlist；未解析的 target/形式保持 UNRESOLVED 并阻止对应正向分支，不推定数字更大的 target 继承能力。
参数 1	target_binding = MANIFEST_EXACT_TARGET_AND_FORM_ALLOWLIST_REQUIRED
参数 2	unresolved_capability_policy = BLOCK_POSITIVE_BRANCH_AND_REPORT_UNRESOLVED
参数 3	tensor_ranks = [1,2,3,4,5]
参数 4	2d_example_elements = [32,16]
参数 5	element_type = F16 或所选 report 形式允许类型
参数 6	report_modes = ["disabled","per16bytes_if_legal","per_element_if_legal"]
参数 7	oob_variants = ["无 OOB","一个边界元素 OOB"]
参数 8	ignore_oob = 仅合法 global→shared::cta 且 report disabled
参数 9	multicast_destination_sets = [[0],[0,1]]
初始化	1. tensor-map 由锁定 fixture/合法构建器产生；不用 driver enum 直接猜 PTX 字段。 2. shared/barrier 对象、owner 与 payload 按目的分别初始化，report enabled 分支用合法 v1。

TC-ASYNC-002 项目	执行内容
执行步骤	1. 生成各 rank/layout/override 的已证合法 form，再组合 report/OOB/multicast 约束。 2. 初始化能被规定采样位置检测的 pattern 及仅非采样位置命中的对照；不把 pattern 报告当通用 ECC。 3. 逐一变异 layout/report/barrier/ignore_oob/mask，独立 checker 核对目标规则。 4. 为合法程序生成 copy、观察、consumer 和对象 cleanup，保存目的布局期望。
观察点	1. tensor map 和 SMEM matrix descriptor/TMEM 地址类型不互换。 2. report-enabled/v1 与 ignore_oob/report-disabled 组合符合具体 form。
通过条件	1. 合法 fixture 可独立重构布局/采样；负例命中预定 tuple 规则。 2. 缺合法 rank/override 来源时对应分支 unresolved 且不关闭覆盖。
失败条件	1. 随机填 native descriptor 字段、给不支持方向启用 ignore_oob。 2. 未采样 pattern 不报告就被当作通用数据完整性失败。
预期产物	1. manifest.json：锁定 2026-11-09 测试计划、源 hash、seed/子流、profile、实际 target/capability 及服务配置。 2. program.ir 与 data_objects.json：保留指令身份、CFG、typed 对象、初始化及 cleanup。 3. resource_audit.json 与 event_obligations.json：独立账本/HB/phase/单位审计。 4. case_result.json 与 coverage_delta.json：区分生成/审计/接收/完成/检查，本阶段真实执行量及 NOT_RUN/BLOCKED 原因。 5. tensor_map_fixture.json、layout_and_report_sampling_audit.json。

TC-ASYNC-003：partial/full 边界

TC-ASYNC-003 项目	执行内容
源与阶段	SIG-ASYNC-003；P2_REFERENCE_PROGRAM；CV-ASYNC-001
配置与 seed	TP-REF-BASE；fixed_seed=0x534d002f（1397555247）；REFERENCE_BOUNDED
回归与当前状态	DIRECTED, RANDOM, REPLAY；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. 固定 PTX9.4 清单及 SM-ARCH-S0 来源 hash 通过 preflight；独立形式/资源审计器与本阶段 harness 已实现。 2. 由 manifest 绑定精确 target、capability 及合法 form allowlist；未解析的 target/形式保持 UNRESOLVED 并阻止对应正向分支，不推定数字更大的 target 继承能力。 3. 独立语义/内存/事件模型及其允许结果集合已锁定；生成器 shadow 不能作为唯一 oracle。
参数 1	target_binding = MANIFEST_EXACT_TARGET_AND_FORM_ALLOWLIST_REQUIRED

TC-ASYNC-003 项目	执行内容
参数 2	unresolved_capability_policy = BLOCK_POSITIVE_BRANCH_AND_REPORT_UNRESOLVED
参数 3	payload_bytes = 512
参数 4	logical_check_partition_bytes = 128
参数 5	partition_note = 仅参考 checker 分区，不规定 native 传输 beat/端口
参数 6	orders = ["顺序","逆序","合法随机 fragment","source 读终点早于 destination full"]
参数 7	operations = ["合法 SMEM→global bulk","带完成义务的 bulk priority"]
初始化	1. source 有确定 pattern，destination/canary 初始化；bulk issuer/group 与 source 引用独立。 2. 对每个 form 建立真实语义的 read-partial/full 义务。
执行步骤	1. 生成含多个逻辑检查片段的 bulk 程序及对应 commit/read-wait/full-wait。 2. 参考模型探索允许 fragment 乱序，先完成 source 读并观察 read-wait。 3. 允许安全重用已获准 source，同时禁止 consumer 把未 full destination 当可用。 4. 推进剩余 fragment/priority full，观察正确终点并关闭全部对象/tx 义务。
观察点	1. read-only 终点只释放对应 source/tensor-map 读取，不直接清目的责任。 2. 每段及整条计数恰一次，不凭 checker 分区假设 native 片段数。
通过条件	1. 所有允许顺序中目的消费都有 full/visibility 保护。 2. reference trace 保存每个 partial/full 归属，最终无孤儿义务。
失败条件	1. read-wait 后立即允许目的消费，或 priority 被当 no-op 而漏 group 完成。 2. 同一 fragment 重试重复计数导致早完成。
预期产物	1. manifest.json：锁定 2026-11-09 测试计划、源 hash、seed/子流、profile、实际 target/capability 及服务配置。 2. program.ir 与 data_objects.json：保留指令身份、CFG、typed 对象、初始化及 cleanup。 3. resource_audit.json 与 event_obligations.json：独立账本/HB/phase/单位审计。 4. case_result.json 与 coverage_delta.json：区分生成/审计/接收/完成/检查，本阶段真实执行量及 NOT_RUN/BLOCKED 原因。 5. reference_bulk_partial_full_trace.jsonl、typed_completion_ledger.json。

TC-ASYNC-004：真实 TMA 竞争与报告

TC-ASYNC-004 项目	执行内容
源与阶段	SIG-ASYNC-004；P3_SM_RTL；CV-ASYNC-003
配置与 seed	TP-SM-BASE；fixed_seed=0x534d0030（1397555248）；SM_PROGRESS
回归与当前状态	DIRECTED, RANDOM, STRESS, REPLAY；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. 固定 PTX9.4 清单及 SM-ARCH-S0 来源 hash 通过 preflight；独立形式/资源审计器与本阶段 harness 已实现。 2. 由 manifest 绑定精确 target、capability 及合法 form allowlist；未解析的 target/形式保持 UNRESOLVED 并阻止对应正向分支，不推定数字更大的 target 继承能力。 3. SM、对应 adapter 和 monitor 真实可用；全部事务的 owner/epoch/返回身份及服务公平合同已配置。
参数 1	target_binding = MANIFEST_EXACT_TARGET_AND_FORM_ALLOWLIST_REQUIRED
参数 2	unresolved_capability_policy = BLOCK_POSITIVE_BRANCH_AND_REPORT_UNRESOLVED
参数 3	tile_example_elements = [32,16]
参数 4	element_bytes = 2
参数 5	im2col_fixture = {"H":8,"W":8,"C":8,"window":[3,3]}
参数 6	legal_pattern_positions = ["规定采样 element","仅未采样 element","无 pattern"]
参数 7	return_orders = ["顺序","合法乱序","尾片延后"]
参数 8	report_modes = ["disabled","enabled_if_capable"]
参数 9	peer_boundary = P3 使用声明的服务模型；跨 SM 端到端关闭留 P4
初始化	1. global 激活/矩阵、合法 tensor map 和 shared 目标/canary 初始化。 2. barrier 使用相容 layout，expect 登记与 arrival 保证不会提前 phase 结束；im2col fixture 需精确 target recipe。
执行步骤	1. 驱动已启用 tile/im2col copy，分别注入规定 pattern 而非任意 ECC 错误。 2. 服务模型扰动 L2/SMEM 返回及尾片，实际 monitor 跟踪所有目的对象/tx 责任。 3. 等待对应终点并比较独立布局和报告 oracle，partial 时不得消费目的。 4. 完成所有目的、报告及 consumer 后 drain source/destination/barrier 引用。
观察点	1. 最后 payload 尚未落地时不能先减完 tx 或报告整操作完成。 2. report 按规定采样，multi-destination 每 owner/phase 独立闭合。

TC-ASYNC-004 项目	执行内容
通过条件	1. 数据位置、zero/OOB 规则和报告关系正确；真实 event 达到 form 规定终点。 2. P3 模型边界明确，不把未实现跨 SM 服务计入 P4 执行覆盖。
失败条件	1. 早 complete_tx、错误 swizzle/owner、将非采样数据作为必报错误。 2. 报告或 source/full 终点错配，而 checked trace 缺实际观察。
预期产物	1. manifest.json：锁定 2026-11-09 测试计划、源 hash、seed/子流、profile、实际 target/capability 及服务配置。 2. program.ir 与 data_objects.json：保留指令身份、CFG、typed 对象、初始化及 cleanup。 3. resource_audit.json 与 event_obligations.json：独立账本/HB/phase/单位审计。 4. case_result.json 与 coverage_delta.json：区分生成/审计/接收/完成/检查，本阶段真实执行量及 NOT_RUN/BLOCKED 原因。 5. tma_smem_barrier_trace.jsonl、layout_report_scoreboard.json。

TC-ASYNC-005：多目的 bulk 系统

TC-ASYNC-005 项目	执行内容
源与阶段	SIG-ASYNC-005；P4_SOC；CV-ASYNC-003
配置与 seed	TP-SOC-BASE；fixed_seed=0x534d0031（1397555249）；SOC_PROGRESS
回归与当前状态	DIRECTED, RANDOM, STRESS, REPLAY；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. 固定 PTX9.4 清单及 SM-ARCH-S0 来源 hash 通过 preflight；独立形式/资源审计器与本阶段 harness 已实现。 2. 由 manifest 绑定精确 target、capability 及合法 form allowlist；未解析的 target/形式保持 UNRESOLVED 并阻止对应正向分支，不推定数字更大的 target 继承能力。 3. 跨 SM/SoC 运行时、对应服务及取消/返回模型可用；未实现系统能力保持 blocked。
参数 1	target_binding = MANIFEST_EXACT_TARGET_AND_FORM_ALLOWLIST_REQUIRED
参数 2	unresolved_capability_policy = BLOCK_POSITIVE_BRANCH_AND_REPORT_UNRESOLVED
参数 3	destinations = [2,4]
参数 4	payload_bytes = 512
参数 5	source_rank = 0
参数 6	destination_schedule = ["一个目的最早 full","一个目的尾片延后","其余合法乱序"]

TC-ASYNC-005 项目	执行内容
参数 7	variants = ["capable multicast","DSM 配合的多目标程序","capable multimem bulk"]
初始化	1. 各目的 CTA 共同 admit 且各有独立 shared/barrier owner；目的集由 legal recipe 编码。 2. 源/目标 payload 和计数按实际方向/单位分别初始化。
执行步骤	1. 发起一个可明确关联多目的责任的合法 bulk 系统事务。 2. 先使一个目的完成并允许该目的合法 consumer，保留其他目的/源仍需责任。 3. 延后远端尾片与 barrier 更新，检查整个 parent 不能因单目的 full 而退休。 4. 全部目的真正闭合后按各方向 wait/report 与 cluster 退出规则 drain。
观察点	1. parent 完成由全部 required 目的及事件终点共同决定。 2. 延迟目的 owner 仍 pin 住，multicast 通知与 payload 不混淆。
通过条件	1. 每目的数据/计数/报告正确且完整 parent 只完成一次。 2. 所有本地/远端引用关闭后才复用 owner。
失败条件	1. 单目的完成导致全事务 free，或向同 offset 错误套本端 base。 2. 跨目的 tx 单位重复/少算，或者迟到远端写入新 generation。
预期产物	1. manifest.json：锁定 2026-11-09 测试计划、源 hash、seed/子流、profile、实际 target/capability 及服务配置。 2. program.ir 与 data_objects.json：保留指令身份、CFG、typed 对象、初始化及 cleanup。 3. resource_audit.json 与 event_obligations.json：独立账本/HB/phase/单位审计。 4. case_result.json 与 coverage_delta.json：区分生成/审计/接收/完成/检查，本阶段真实执行量及 NOT_RUN/BLOCKED 原因。 5. soc_multidestination_trace.jsonl、parent_child_completion_witness.json。

20.13 GEN-TMEM-01：TMEM 分配、访问、搬移和释放全生命周期

TC-TMEM-001：分配规则与 permit

TC-TMEM-001 项目	执行内容
源与阶段	SIG-TMEM-001；P1_GENERATOR_UNIT；CV-TMEM-001
配置与 seed	TP-GEN-BASE；fixed_seed=0x534d0032（1397555250）；GEN_BOUNDED
回归与当前状态	SMOKE, DIRECTED, NEGATIVE, RANDOM, REPLAY；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION

TC-TMEM-001 项目	执行内容
前置条件	1. 固定 PTX9.4 清单及 SM-ARCH-S0 来源 hash 通过 preflight；独立形式/资源审计器与本阶段 harness 已实现。 2. 由 manifest 绑定精确 target、capability 及合法 form allowlist；未解析的 target/形式保持 UNRESOLVED 并阻止对应正向分支，不推定数字更大的 target 继承能力。
参数 1	target_binding = MANIFEST_EXACT_TARGET_AND_FORM_ALLOWLIST_REQUIRED
参数 2	unresolved_capability_policy = BLOCK_POSITIVE_BRANCH_AND_REPORT_UNRESOLVED
参数 3	profiles_columns = [512,576]
参数 4	nonexclusive_sequences = [[256,128,64,32,32],[32,32], [32,32,32,32,32,32,32,32,32,32,32,32,32,32,32,32,32,32,32,32]]
参数 5	exclusive_sizes_columns = [32,96,512,576]
参数 6	profile_note = 18x32 及 576 exclusive 仅在已解析 576 profile；不强行用于 512 profile
参数 7	negative_variants = ["同 CTA 连续 32 后 64","exclusive 非 32 倍数","relinquish 后再 alloc","不匹配 dealloc"]
初始化	1. TMEM 为空且 permit 有效；各 kernel 固定 cta_group，exclusive 能力与 MAX_COLS 均来自精确 target。 2. alloc 结果写入的 SMEM 对象预分配，address 发布顺序被保留。
执行步骤	1. 实例化非递增含相等边界的 allocation 序列，按 profile 预算校验 base/columns/owner。 2. 在独立场景生成 exclusive 大小及等待其他 owner 安全释放后取得容量，不把合法等待当非法。 3. relinquish 后继续使用并匹配释放既有 allocation，拒绝后续 alloc；负例只做静态审计。 4. 为每笔 live allocation 生成访问结束与匹配 dealloc，发布结果并正常退出。
观察点	1. permit 与 allocation 状态分离，exclusive 属性和 dealloc 大小/group 匹配。 2. 512/576 能力、连续申请顺序及无重叠不越权均独立检查。
通过条件	1. 全部合法生命周期闭合，包括 32→32 相等边界；未知 576 能力阻止正向分支。 2. 每个 mutant 按对应 permit/顺序/属性规则拒绝。
失败条件	1. 把 permit 放弃当 free、给 512 profile 开放 576，或合法 alloc 等待被误判必错。 2. 同一 allocation 重复 free、半发布地址或忽略有权范围。

TC-TMEM-001 项目	执行内容
预期产物	1. manifest.json：锁定 2026-11-09 测试计划、源 hash、seed/子流、profile、实际 target/capability 及服务配置。 2. program.ir 与 data_objects.json：保留指令身份、CFG、typed 对象、初始化及 cleanup。 3. resource_audit.json 与 event_obligations.json：独立账本/HB/phase/单位审计。 4. case_result.json 与 coverage_delta.json：区分生成/审计/接收/完成/检查，本阶段真实执行量及 NOT_RUN/BLOCKED 原因。

TC-TMEM-002：参与与地址类型

TC-TMEM-002 项目	执行内容
源与阶段	SIG-TMEM-002；P1_GENERATOR_UNIT；CV-TMEM-002
配置与 seed	TP-GEN-BASE；fixed_seed=0x534d0033（1397555251）；GEN_BOUNDED
回归与当前状态	DIRECTED, NEGATIVE, REPLAY；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. 固定 PTX9.4 清单及 SM-ARCH-S0 来源 hash 通过 preflight；独立形式/资源审计器与本阶段 harness 已实现。 2. 由 manifest 绑定精确 target、capability 及合法 form allowlist；未解析的 target/形式保持 UNRESOLVED 并阻止对应正向分支，不推定数字更大的 target 继承能力。
参数 1	target_binding = MANIFEST_EXACT_TARGET_AND_FORM_ALLOWLIST_REQUIRED
参数 2	unresolved_capability_policy = BLOCK_POSITIVE_BRANCH_AND_REPORT_UNRESOLVED
参数 3	group1 = {"ctas":1,"warps":4,"threads_per_warp":32}
参数 4	group2 = {"ctas":2,"sms":2,"allocator_warps_each_cta":1}
参数 5	tmem_views = ["quarter0","quarter1","quarter2","quarter3"]
参数 6	base_operand = 同一 warp 统一 symbolic TMEM base
参数 7	negative_variants = ["某 lane 不同 base","成员缺失","TMEM 误作 byte 地址","错误 pair rank"]
初始化	1. 为每 quarter 建立合法 allocation 及对应 warp 身份；group1/group2 分别使用独立 kernel。 2. CTA pair 已符合 S0 两个不同可互通 endpoint 和 joint admission。

TC-TMEM-002 项目	执行内容
执行步骤	1. 生成 warp ld/st/wait 统一 base 与各 quarter 合法片段，独立验证 lane/column 坐标。 2. 生成两 CTA 各一 warp 的 allocation collective 及正确单 thread MCS 发令意图，防重复 parent。 3. 单因素变异 base/成员/地址类型/rank，只交参与/类型审计器拒绝。 4. 对合法 IR 完成结果发布、相应 wait 及双边匹配释放。
观察点	1. collective 按动态实例与成员匹配，ld/st 一个 warp 不能获得全部 TMEM lanes。 2. TMEM、SMEM byte 地址和 alloc 的 SMEM dst 是不同 typed 对象。
通过条件	1. 正确参与者/地址模板通过独立审计，非法变体在指定规则拒绝。 2. 没有从 M/N 逻辑 shape 推物理 lane 数量或端口。
失败条件	1. 单 thread 替代 warp collective、两端重复发相同 MCS parent。 2. 地址 cast 靠裸整数巧合通过，或同 PC 不同轮实例被合并。
预期产物	1. manifest.json：锁定 2026-11-09 测试计划、源 hash、seed/子流、profile、实际 target/capability 及服务配置。 2. program.ir 与 data_objects.json：保留指令身份、CFG、typed 对象、初始化及 cleanup。 3. resource_audit.json 与 event_obligations.json：独立账本/HB/phase/单位审计。 4. case_result.json 与 coverage_delta.json：区分生成/审计/接收/完成/检查，本阶段真实执行量及 NOT_RUN/BLOCKED 原因。

TC-TMEM-003：LD/ST 布局与扩展

TC-TMEM-003 项目	执行内容
源与阶段	SIG-TMEM-003；P2_REFERENCE_PROGRAM；CV-TMEM-001
配置与 seed	TP-REF-BASE；fixed_seed=0x534d0034（1397555252）；REFERENCE_BOUNDED
回归与当前状态	DIRECTED, RANDOM, REPLAY；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. 固定 PTX9.4 清单及 SM-ARCH-S0 来源 hash 通过 preflight；独立形式/资源审计器与本阶段 harness 已实现。 2. 由 manifest 绑定精确 target、capability 及合法 form allowlist；未解析的 target/形式保持 UNRESOLVED 并阻止对应正向分支，不推定数字更大的 target 继承能力。 3. 独立语义/内存/事件模型及其允许结果集合已锁定；生成器 shadow 不能作为唯一 oracle。
参数 1	target_binding = MANIFEST_EXACT_TARGET_AND_FORM_ALLOWLIST_REQUIRED
参数 2	unresolved_capability_policy = BLOCK_POSITIVE_BRANCH_AND_REPORT_UNRESOLVED

TC-TMEM-003 项目	执行内容
参数 3	shape_candidates = ["32x32b","16x64b","16x128b","16x256b","16x32bx2"]
参数 4	repeat_candidates = [1,4,128]
参数 5	selection_policy = 仅保留该 form/shape/target 合法 repeat
参数 6	data_types = ["合法 F16 pack/unpack","F32/S32/U32 red","F32 spcompress2:4 b2"]
参数 7	F32_groups = [[1,2,3,4],[0.0,-0.0,1,1],["NaN",1,2,3]]
参数 8	spcompress_repeat_candidates = [4,128]
初始化	1. 实际访问范围由 layout recipe 求出后分配足够 columns，不将所有 repeat 硬塞固定小 allocation。 2. TMEM 内容与预期 register/meta 片段由独立 layout 模型初始化。
执行步骤	1. 对每个合法 shape/repeat 构造 st/ld、pack/unpack 的参考程序并建立寄存器片段身份。 2. 分别生成 red、spcompress、red.spcompress 的合法类型/target 及特殊值组。 3. 独立计算 lane/column 映射、压缩允许选择集合与 metadata，区分 reduction NaN 控制。 4. 按 ld/st 完成观察全部返回并关闭 allocation/源寄存器引用后释放。
观察点	1. 宽返回全部片段才 ready，不以一个结果 word 代表整 warp tuple。 2. tie/NaN 处理按允许集合，spcompress metadata 不未经适配冒充任意 mma.sp。
通过条件	1. 数据/packing/meta/reduction 满足独立模型或允许集合。 2. target/shape/repeat 未解析的组合 blocked，不靠邻近形式补猜。
失败条件	1. pack 改变数值或 padding 精度、归约当整数序替代浮点 NaN 规则。 2. 越过 allocation 范围，或把 spcompress 限定符/metadata 适用范围扩大。
预期产物	1. manifest.json：锁定 2026-11-09 测试计划、源 hash、seed/子流、profile、实际 target/capability 及服务配置。 2. program.ir 与 data_objects.json：保留指令身份、CFG、typed 对象、初始化及 cleanup。 3. resource_audit.json 与 event_obligations.json：独立账本/HB/phase/单位审计。 4. case_result.json 与 coverage_delta.json：区分生成/审计/接收/完成/检查，本阶段真实执行量及 NOT_RUN/BLOCKED 原因。 5. reference_tmem_fragment_map.json、compression_allowed_selection.json。

TC-TMEM-004：CP/shift 与多片 wait

TC-TMEM-004 项目	执行内容
源与阶段	SIG-TMEM-004；P3_SM_RTL；CV-TMEM-003
配置与 seed	TP-SM-BASE；fixed_seed=0x534d0035（1397555253）；SM_PROGRESS
回归与当前状态	DIRECTED, RANDOM, STRESS, REPLAY；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. 固定 PTX9.4 清单及 SM-ARCH-S0 来源 hash 通过 preflight；独立形式/资源审计器与本阶段 harness 已实现。 2. 由 manifest 绑定精确 target、capability 及合法 form allowlist；未解析的 target/形式保持 UNRESOLVED 并阻止对应正向分支，不推定数字更大的 target 继承能力。 3. SM、对应 adapter 和 monitor 真实可用；全部事务的 owner/epoch/返回身份及服务公平合同已配置。
参数 1	target_binding = MANIFEST_EXACT_TARGET_AND_FORM_ALLOWLIST_REQUIRED
参数 2	unresolved_capability_policy = BLOCK_POSITIVE_BRANCH_AND_REPORT_UNRESOLVED
参数 3	ctas = 1
参数 4	warps = 4
参数 5	allocation_columns = 128
参数 6	operations = ["SMEM→TMEM cp","合法 TMEM shift","非冲突 TMEM ld/st"]
参数 7	source_bytes = 512
参数 8	shift_binding = 固定规范 recipe 的区域/方向，不自定义字段
参数 9	return_variants = ["多片顺序","多片乱序","最后片延后"]
初始化	1. 源/目的 TMEM 区域按 recipe 分区并初始化 canary，alias shift 仅使用指令定义的原地语义。 2. SMEM 源已正确发布到 async proxy，MCS issuer 与 ld/st warp streams 分别建账。
执行步骤	1. 在资源/HB 允许下交错 cp、shift 和独立 ld/st，冲突对象按规范显式排序。 2. 先后观察 MCS full、ld/st 片段与 wait，允许彼此独立进度而不混用终点。 3. 延后最后 RF 片段/写 ack，检查消费者/后续覆盖仍被保护。 4. 全部 MCS 与 ld/st 责任闭合后读取结果/canary 并 dealloc 退出。
观察点	1. cp/shift 数据真正提交 TMEM 才计对应 full，RF 需全片段 ready。 2. 相互重叠 shift 保留原数据语义；不是从未知固定周期推出完成。

TC-TMEM-004 项目	执行内容
通过条件	1. 数据移动/片段/顺序与独立模型相符，normal commit 和 wait::ld/st 不互代。 2. 所有引用和完成 exactly-once 后资源归还。
失败条件	1. cp 被当 TMA 或普通 byte cast、shift 覆盖尚未捕获源。 2. MCS full 清掉仍未完成 ld/st stream，或首片触发整体 ready。
预期产物	1. manifest.json：锁定 2026-11-09 测试计划、源 hash、seed/子流、profile、实际 target/capability 及服务配置。 2. program.ir 与 data_objects.json：保留指令身份、CFG、typed 对象、初始化及 cleanup。 3. resource_audit.json 与 event_obligations.json：独立账本/HB/phase/单位审计。 4. case_result.json 与 coverage_delta.json：区分生成/审计/接收/完成/检查，本阶段真实执行量及 NOT_RUN/BLOCKED 原因。 5. move_tmem_rf_trace.jsonl、fragment_completion_scoreboard.json。

TC-TMEM-005：释放与退出竞争

TC-TMEM-005 项目	执行内容
源与阶段	SIG-TMEM-005；P3_SM_RTL；CV-TMEM-003
配置与 seed	TP-SM-BASE；fixed_seed=0x534d0036（1397555254）；SM_PROGRESS
回归与当前状态	DIRECTED, NEGATIVE, STRESS, REPLAY；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. 固定 PTX9.4 清单及 SM-ARCH-S0 来源 hash 通过 preflight；独立形式/资源审计器与本阶段 harness 已实现。 2. 由 manifest 绑定精确 target、capability 及合法 form allowlist；未解析的 target/形式保持 UNRESOLVED 并阻止对应正向分支，不推定数字更大的 target 继承能力。 3. SM、对应 adapter 和 monitor 真实可用；全部事务的 owner/epoch/返回身份及服务公平合同已配置。
参数 1	target_binding = MANIFEST_EXACT_TARGET_AND_FORM_ALLOWLIST_REQUIRED
参数 2	unresolved_capability_policy = BLOCK_POSITIVE_BRANCH_AND_REPORT_UNRESOLVED
参数 3	allocation_columns = 128
参数 4	reuse_rounds = 2
参数 5	schedules = ["MCS full 先于 ld 返回","源读结束但目的 writeack 未到","合法最终全部关闭"]

TC-TMEM-005 项目	执行内容
参数 6	positive_release = 显式观察对应 full 及 ld/st 后才 dealloc
参数 7	negative_release = 独立静态 mutant 或已定义 NONISA robustness 路线，不默认运行 UB
初始化	1. TMEM、barrier、register 目的范围和 generation 映射均有效且已初始化。 2. 已发起操作持有引用，释放请求尚未具备程序 HB 条件。
执行步骤	1. 生成一个有限延迟的 MCS 及相关 ld/st，保持真实 full/片段责任。 2. 在正向程序中等待每个需要的终点和线程交接，再执行匹配 dealloc。 3. 另建提前 release mutation，由资源审计拒绝；只在项目保护合同明确时走隔离 robustness。 4. 新一轮重用相同可分配范围并改变 generation，检查迟到旧响应隔离及新数据。
观察点	1. 同一 full 事件不能替代未完 ld/st；A-read 不能替代 D/full。 2. 复用发生在全部旧访问关闭或正式取消确认之后，旧消息不得改变新 owner。
通过条件	1. 正向释放前排除全部并发访问；每轮 allocation 只回收一次。 2. 负例拒绝层正确，旧 generation 注入若属 robustness 则单列 NONISA 证据。
失败条件	1. 以 dealloc 恰好阻塞替代程序同步，或旧 writeack 使新 allocation 提前完成。 2. undefined 提前 free 被当正常合法程序比较数值结果。
预期产物	1. manifest.json：锁定 2026-11-09 测试计划、源 hash、seed/子流、profile、实际 target/capability 及服务配置。 2. program.ir 与 data_objects.json：保留指令身份、CFG、typed 对象、初始化及 cleanup。 3. resource_audit.json 与 event_obligations.json：独立账本/HB/phase/单位审计。 4. case_result.json 与 coverage_delta.json：区分生成/审计/接收/完成/检查，本阶段真实执行量及 NOT_RUN/BLOCKED 原因。 5. tmem_release_lifetime_trace.jsonl、generation_reuse_witness.json。

TC-TMEM-006：CTA-pair 双端资源

TC-TMEM-006 项目	执行内容
源与阶段	SIG-TMEM-006；P4_SOC；CV-TMEM-003
配置与 seed	TP-SOC-BASE；fixed_seed=0x534d0037（1397555255）；SOC_PROGRESS
回归与当前状态	DIRECTED, RANDOM, STRESS, REPLAY；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION

TC-TMEM-006 项目	执行内容
前置条件	1. 固定 PTX9.4 清单及 SM-ARCH-S0 来源 hash 通过 preflight；独立形式/资源审计器与本阶段 harness 已实现。 2. 由 manifest 绑定精确 target、capability 及合法 form allowlist；未解析的 target/形式保持 UNRESOLVED 并阻止对应正向分支，不推定数字更大的 target 继承能力。 3. 跨 SM/SoC 运行时、对应服务及取消/返回模型可用；未实现系统能力保持 blocked。
参数 1	target_binding = MANIFEST_EXACT_TARGET_AND_FORM_ALLOWLIST_REQUIRED
参数 2	unresolved_capability_policy = BLOCK_POSITIVE_BRANCH_AND_REPORT_UNRESOLVED
参数 3	ctas = 2
参数 4	sms = 2
参数 5	pair_ranks = [0,1]
参数 6	cta_group = 2
参数 7	example_mma_shape = M256xN128xK16 F16 合法 recipe
参数 8	columns_each_endpoint = 128
参数 9	allocator_warps_each_cta = 1
参数 10	parent_issuer_threads = 1
参数 11	schedule_variants = ["peer prepare 延迟","peer 数据尾片延迟","双边正常释放"]
参数 12	independent_CTA1_exclusive = 仅在另一个 group1 kernel 场景验证，不同 kernel 混组
初始化	1. 设备 joint admission 保证两个不同可互通 endpoint；两 CTA owner/barrier/SMEM 源均合法初始化。 2. 整个 kernel 的 cta_group 固定 2，双边 allocation 属性和 source 布局由 group2 recipe 确定。
执行步骤	1. 两端 collective 申请共同合法范围并发布地址，不在单边 prepare 后消费。 2. 由一个合法 thread 发起一次 parent 操作，peer 仅保存对应 child/双边义务。 3. 交错双端访问并延迟一边完成，观察 A-read/full masks 不被单边清空。 4. 双边访问与数据/事件 ack 全关闭后执行匹配 dealloc，再允许两 CTA 退出。
观察点	1. allocation 与释放无半边发布/回收，parent/child 引用稳定且 full 需要 required 两端。 2. S0 放置、rank 配对和 kernel group 一致性独立审计。
通过条件	1. 双端结果/计数正确，peer 有限延迟不会丢事件或提前 free。 2. group1 独立 exclusive 只在独立 kernel 验证，不错误扩大成全 pair 锁。

TC-TMEM-006 项目	执行内容
失败条件	1. 任意两 CTA 被配成 peer、同 SM 未经定义适配，或两端重复 parent。 2. 单边完成释放全 parent，或者 group2 与 group1 在同 kernel 随意切换。
预期产物	1. manifest.json：锁定 2026-11-09 测试计划、源 hash、seed/子流、profile、实际 target/capability 及服务配置。 2. program.ir 与 data_objects.json：保留指令身份、CFG、typed 对象、初始化及 cleanup。 3. resource_audit.json 与 event_obligations.json：独立账本/HB/phase/单位审计。 4. case_result.json 与 coverage_delta.json：区分生成/审计/接收/完成/检查，本阶段真实执行量及 NOT_RUN/BLOCKED 原因。 5. soc_pair_parent_child_trace.jsonl、paired_allocation_and_free_witness.json。

20.14 GEN-MATRIX-01：各代矩阵、scale/BLOCK 与 dot 接口

TC-MATRIX-001：格式/scale/block 独立约束

TC-MATRIX-001 项目	执行内容
源与阶段	SIG-MATRIX-001；P1_GENERATOR_UNIT；CV-MATRIX-001
配置与 seed	TP-GEN-BASE；fixed_seed=0x534d0038（1397555256）；GEN_BOUNDED
回归与当前状态	SMOKE, DIRECTED, NEGATIVE, RANDOM, REPLAY；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. 固定 PTX9.4 清单及 SM-ARCH-S0 来源 hash 通过 preflight；独立形式/资源审计器与本阶段 harness 已实现。 2. 由 manifest 绑定精确 target、capability 及合法 form allowlist；未解析的 target/形式保持 UNRESOLVED 并阻止对应正向分支，不推定数字更大的 target 继承能力。
参数 1	target_binding = MANIFEST_EXACT_TARGET_AND_FORM_ALLOWLIST_REQUIRED
参数 2	unresolved_capability_policy = BLOCK_POSITIVE_BRANCH_AND_REPORT_UNRESOLVED
参数 3	tuple_families = ["f16","tf32","i8","ti16","f8f6f4","mxf8f6f4","mxf4","mxf4nvf4"]
参数 4	native_scale_rules = ["mxf8f6f4: UE8M0+BLOCK32","mxf4:E2M1+UE8M0+BLOCK32","mxf4nvf4:E2M1+合法 UE8M0/UE4M3/UE5M3 和 BLOCK16/32；附精确 target 条件"]
参数 5	project_offsets_BLOCK16 = [0,1,15]
参数 6	project_offsets_BLOCK32 = [0,1,31]

TC-MATRIX-001 项目	执行内容
参数 7	raw_Ti16 = false
参数 8	decoded_Ti16_range = [-2047,2047]
参数 9	native_AB_scale_format = 共同字段；项目接口独立字段不意味着 native 可不同
参数 10	project_MALU_input_format_candidates = ["FMT_F16","FMT_BF16","FMT_TF32","FMT_F32","FMT_F64","FMT_E4M3","FMT_E5M2","FMT_E2M3","FMT_E3M2","FMT_E2M1","FMT_S8","FMT_U8","FMT_S4","FMT_U4","FMT_TI16","FMT_B1"]
参数 11	project_result_format_policy = {"floating_C_D_candidates": ["FMT_F16","FMT_BF16","FMT_F32","FMT_F64"],"integer_C_D":"FMT_S32","FP64":"positive only with ENABLE_FP64 and required adapter/backend; disabled variant checks capability rejection","native_difference":"Project F32/F64/INT4/B1 dot support does not imply corresponding tcgen05 kind or native encoding."}
参数 12	project_Ti16_transport = {"a_fmt":"FMT_TI16","b_fmt":"FMT_TI16","ti16_decoded":true,"raw_capability":false,"value_range":[-2047,2047],"note":"Decoded transport is a separate control, not an invented native or enum format."}
初始化	1. 所有矩阵/scale/meta 对象由已审核 shape/layout recipe 初始化；项目和 native 入口分别建 fixture。 2. UE5M3 及 UE4M3+BLOCK32 等 target 条件明确解析；缺证据组合不开启。
执行步骤	1. 从合法 tuple 表选择 kind/shape/格式/source/scale/block，不做全字段随机笛卡尔积。 2. 分别生成独立项目 offset 与 native 布局，检查项目 adapter 而不虚构 PTX 总线约束。 3. 单因素变异 scale fmt/block/target 或请求 raw Ti16；验证对应 cap/tuple 拒绝。 4. 合法 IR 生成完整矩阵完成与 cleanup，decoded Ti16 单列项目扩展且范围审计。
观察点	1. native AB 共用 scale 格式与项目独立字段映射正确，Cshift 不混入 scale。 2. raw Ti16 保持关闭，decoded/S16 不被标为原生 s1z4m11 编码。 3. 原生 i8/ti16 与 project INT8/INT4/B1/decoded Ti16、F32/F64 扩展分别登记，不能只枚举浮点 kind；raw Ti16 仍关闭。
通过条件	1. 所有已解析正例合法、负例规则精确；target 未解析分支明确 blocked。 2. source phase/CV 与 native/project 身份在 manifest 中保持分离。
失败条件	1. 任意 FP4 均允许所有 scale/BLOCK，或从 sm 数字大小自动继承能力。 2. 把 decoded Ti16 通过记为 raw native 正向覆盖。

TC-MATRIX-001 项目	执行内容
预期产物	1. manifest.json：锁定 2026-11-09 测试计划、源 hash、seed/子流、profile、实际 target/capability 及服务配置。 2. program.ir 与 data_objects.json：保留指令身份、CFG、typed 对象、初始化及 cleanup。 3. resource_audit.json 与 event_obligations.json：独立账本/HB/phase/单位审计。 4. case_result.json 与 coverage_delta.json：区分生成/审计/接收/完成/检查，本阶段真实执行量及 NOT_RUN/BLOCKED 原因。

TC-MATRIX-002：Cseed 和数学 K

TC-MATRIX-002 项目	执行内容
源与阶段	SIG-MATRIX-002；P1_GENERATOR_UNIT；CV-MATRIX-002
配置与 seed	TP-GEN-BASE；fixed_seed=0x534d0039（1397555257）；GEN_BOUNDED
回归与当前状态	DIRECTED, RANDOM, REPLAY；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. 固定 PTX9.4 清单及 SM-ARCH-S0 来源 hash 通过 preflight；独立形式/资源审计器与本阶段 harness 已实现。 2. 由 manifest 绑定精确 target、capability 及合法 form allowlist；未解析的 target/形式保持 UNRESOLVED 并阻止对应正向分支，不推定数字更大的 target 继承能力。
参数 1	target_binding = MANIFEST_EXACT_TARGET_AND_FORM_ALLOWLIST_REQUIRED
参数 2	unresolved_capability_policy = BLOCK_POSITIVE_BRANCH_AND_REPORT_UNRESOLVED
参数 3	native_example = 合法 f16 M128xN64xK16；Cshift 仅规定 f16/tf32 kind
参数 4	project_A_B_formats = ["F16/F16","BF16/BF16","TF32/TF32"]
参数 5	project_logical_K = [1,4,15,16,17,31,32,33]
参数 6	C_value = 8
参数 7	Cshift = [0,1,3,15]
参数 8	enable_input_d = [false,true]
参数 9	disabled_old_D = 目标格式 qNaN 或未定义旧 D；不允许任何读取依赖

TC-MATRIX-002 项目	执行内容
参数 10	named_project_adapter_witness = {"id":"PROJECT_F16_K4_CSHIFT3","entry":"PROJECT_MALU_ADAPTER; not an inferred raw-native tuple","profile":"PROJECT_EXACT","a_fmt":"FMT_F16","b_fmt":"FMT_F16","c_fmt":"FMT_F32","d_fmt":"FMT_F32","A_constant_raw_u16":"0x3c00","B_constant_raw_u16":"0x3c00","logical_K":4,"block_scale_en":false,"C_true_value":8,"C_true_raw_F32":"0x41000000","c_scale_shift":3,"enable_input_d_true_expected_value":5,"enable_input_d_true_expected_F32":"0x40a00000","enable_input_d_false_old_D_F32":"0x7fc00000","enable_input_d_false_C_seed":0,"enable_input_d_false_expected_value":4,"enable_input_d_false_expected_F32":"0x40800000","false_old_D_read_obligations":0,"native_gate":"Any mapping to a native matrix operation requires a separately approved kind/shape/target recipe."}
初始化	1. 合法浮点矩阵 A/B 初始化为 1，正式 C 为 8；禁用 D 变体另给旧 D qNaN/未定义标记。 2. native K 由选定 shape 固定，项目 K 候选不能未经合法化直接印为 native shape。
执行步骤	1. 分别生成 enable-input-d=true 和 false 的 dot/matrix 意图，保留 logical/packed/compressed K 计数。 2. true 传未缩放 C 及独立 shift，false 以正式零 seed 替代且不形成旧 D 读取请求。 3. 改变 beat 切分/合法 loop 分组，确认 C 每 dot 只加一次并用独立精确/allowed-profile 模型求值。 4. 完成正确 full/结果观察与源释放，保存 Cshift 和旧 D 读取义务审计。
观察点	1. s=0/1/15 不被预量化或双缩放，C 不按 beat 重复累加。 2. false 分支没有旧 D 读/NaN 传播；raw dtype/shape/profile 仍合法。 3. 显式执行 PROJECT_F16_K4_CSHIFT3：scale 关闭，true 为 5/false 为 4；不把该见证与低位 block scale 组合。
通过条件	1. 所选 profile 数值关系正确，逻辑 K 与源消耗/scale 索引一致。 2. native 及项目 Cshift 允许范围分别通过独立规则审计。
失败条件	1. false 仍读旧 D，或先对 C 量化再重复 shift 改变结果。 2. 把任意 kind 的 0..15 shift 都默认合法，或把项目 K 印成非法 native K。

TC-MATRIX-002 项目	执行内容
预期产物	1. manifest.json：锁定 2026-11-09 测试计划、源 hash、seed/子流、profile、实际 target/capability 及服务配置。 2. program.ir 与 data_objects.json：保留指令身份、CFG、typed 对象、初始化及 cleanup。 3. resource_audit.json 与 event_obligations.json：独立账本/HB/phase/单位审计。 4. case_result.json 与 coverage_delta.json：区分生成/审计/接收/完成/检查，本阶段真实执行量及 NOT_RUN/BLOCKED 原因。 5. C_seed_shift_and_K_audit.json、reference_expected_results.json。

TC-MATRIX-003：数值与稀疏变换

TC-MATRIX-003 项目	执行内容
源与阶段	SIG-MATRIX-003；P2_REFERENCE_PROGRAM；CV-MATRIX-001
配置与 seed	TP-REF-BASE；fixed_seed=0x534d003a（1397555258）；REFERENCE_BOUNDED
回归与当前状态	DIRECTED, RANDOM, REPLAY；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. 固定 PTX9.4 清单及 SM-ARCH-S0 来源 hash 通过 preflight；独立形式/资源审计器与本阶段 harness 已实现。 2. 由 manifest 绑定精确 target、capability 及合法 form allowlist；未解析的 target/形式保持 UNRESOLVED 并阻止对应正向分支，不推定数字更大的 target 继承能力。 3. 独立语义/内存/事件模型及其允许结果集合已锁定；生成器 shadow 不能作为唯一 oracle。
参数 1	target_binding = MANIFEST_EXACT_TARGET_AND_FORM_ALLOWLIST_REQUIRED
参数 2	unresolved_capability_policy = BLOCK_POSITIVE_BRANCH_AND_REPORT_UNRESOLVED
参数 3	profiles = ["PROJECT_EXACT","已解析 native 允许精度 profile"]
参数 4	project_K = [4,8,32]
参数 5	value_classes = ["有限精确相消","+0/-0","次正规","该格式合法 NaN/Inf"]
参数 6	structural_zero_Inf = PROJECT_EXACT 展开基线检查；不冒充全部 native 硬件唯一规则
参数 7	metadata_LUT = 按 kind/shape/source layout 分别生成
参数 8	sparse_pattern_by_kind = {"tf32":"1:2, use exact approved target metadata encodings","f16_i8_ti16_f8f6f4_mxf8f6f4":"2:4, logical positions and exact target/version metadata separately validated","mxf4_mxf4nvf4":"paired4:8: selected element pairs remain intact; not arbitrary4-of8"}

TC-MATRIX-003 项目	执行内容
参数 9	native_metadata_policy = Do not infer native legality from only the six canonical mathematical 2-of-4 position pairs; use exact approved per-kind/target/version metadata tables, including clarified special cases.
参数 10	project_integer_modes = {"AB_candidates": ["S8","U8","S4","U4","TI16_DECODED","B1"],"C_D":"S32","block_scale_en":false,"c_scale_shift":0,"overflow":"full mathematical integer result then allowed S32 wrap or final saturation","saturation":"only combinations explicitly allowed by project cfg; TI16 and B1 saturation off","B1_modes":["AND-popcount","XOR-popcount"],"same_width_and_binary_flags":"independently audit according to project interface; never imply native compatibility"}
初始化	1. 矩阵、scale、合法 metadata 与 LUT 分开初始化；FP4/FP6 等无 Inf 格式不构造假 Inf 编码。 2. oracle 独立解码并记录 native 保证/allowed set，缺数值公开规则分支 blocked。
执行步骤	1. 按所选 kind 分别生成 TF32 1:2、其他批准 2:4 或 MX/NV FP4 paired4:8 的逻辑位置及原生 target/version metadata；project 整数/B1 扩展另建合法向量，不套任意 native 编码。 2. 在适用 profile 加入结构性零和对应 B 的 Inf/NaN，检查项目展开语义不被跳项消失。 3. 独立改变 metadata 位置/LUT 索引与布局，计算位置和数值允许结果。 4. 按完整矩阵事务完成并关闭资源，保存 profile 分离的数值见证。
观察点	1. Fraction/整数模型不调用 DUT decode；native 不被强制等于 PROJECT_EXACT。 2. metadata、LUT、scale cursor 分别对应 logical/packed K 和目标 kind。 3. sparsity 按 kind 分型，FP4 成对 4:8 保持 pair，TF32 不用 2:4 模板；整数/B1 不得注入浮点 NaN/Inf，结果 C/D 和溢出判据正确。
通过条件	1. 精确 profile 逐位或规定舍入正确，native 只接受其规范/目标允许结果。 2. 无 Inf 编码的格式没有被错误当 IEEE all-ones 特例。
失败条件	1. 省略项跳过导致项目 0xInf/NaN 错误，或元数据仅位宽相同就通用。 2. 用单一 exact oracle 给无依据 native 精度声明 bitfit。
预期产物	1. manifest.json：锁定 2026-11-09 测试计划、源 hash、seed/子流、profile、实际 target/capability 及服务配置。 2. program.ir 与 data_objects.json：保留指令身份、CFG、typed 对象、初始化及 cleanup。 3. resource_audit.json 与 event_obligations.json：独立账本/HB/phase/单位审计。 4. case_result.json 与 coverage_delta.json：区分生成/审计/接收/完成/检查，本阶段真实执行量及 NOT_RUN/BLOCKED 原因。 5. independent_numeric_and_sparse_witness.json、profile_allowed_results.json。

TC-MATRIX-004：同 MALU 后端多前端

TC-MATRIX-004 项目	执行内容
源与阶段	SIG-MATRIX-004；P3_SM_RTL；CV-MATRIX-003
配置与 seed	TP-SM-BASE；fixed_seed=0x534d003b（1397555259）；SM_PROGRESS
回归与当前状态	DIRECTED, RANDOM, STRESS, REPLAY；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. 固定 PTX9.4 清单及 SM-ARCH-S0 来源 hash 通过 preflight；独立形式/资源审计器与本阶段 harness 已实现。 2. 由 manifest 绑定精确 target、capability 及合法 form allowlist；未解析的 target/形式保持 UNRESOLVED 并阻止对应正向分支，不推定数字更大的 target 继承能力。 3. SM、对应 adapter 和 monitor 真实可用；全部事务的 owner/epoch/返回身份及服务公平合同已配置。
参数 1	target_binding = MANIFEST_EXACT_TARGET_AND_FORM_ALLOWLIST_REQUIRED
参数 2	unresolved_capability_policy = BLOCK_POSITIVE_BRANCH_AND_REPORT_UNRESOLVED
参数 3	frontend_campaigns = ["wmma/mma 合法 target 单独程序","wgmma sm90a 兼容候选单独程序","tcgen05 合法 target 程序"]
参数 4	mix_policy = 单 native kernel 不得混互不兼容 target；单后端混合仅显式项目兼容 IR 且能力已实现
参数 5	backend = 共享 dot-only 数值服务
参数 6	default_wgmma_compatibility = false
参数 7	matrix_batches = 2
初始化	1. 每个 frontend 按自己的参与者/fragment/descriptor 和 profile 初始化对象。 2. 兼容扩展与 native 入口标识不同；关闭路径只执行 preflight 拒绝而不发真实未实现命令。
执行步骤	1. 先运行每个已启用 frontend 的闭合 baseline，核对参与/寄存器或 TMEM 结果位置。 2. 在支持的项目多前端路线交错两 batch，观察共享数值后端的源保护、mode/返回身份。 3. 分别执行 warp 同步、wgmma group 及 tcgen05 commit/wait，拒绝跨 family 替代 mutation。 4. 所有结果返回/实际存储写入完成后，逐 frontend 清理并退出。
观察点	1. 没有给 sm107 等目标无依据开启 wgmma sm90a native 语法。 2. MALU 仅 dot+C，不因共享后端承担标量 FMA/SFU。

TC-MATRIX-004 项目	执行内容
通过条件	1. 每代前端结果 relation、成员和完成协议均正确；关闭分支有独立拒绝证据。 2. 共享后端切换不串 tag、不破坏已接受源快照/完成进度。
失败条件	1. 同一 native kernel 混入 target 互斥形式，或以换 opcode 替代参与协议。 2. wgmma wait、warp mma 与 tcgen05 commit 互相代等。
预期产物	1. manifest.json：锁定 2026-11-09 测试计划、源 hash、seed/子流、profile、实际 target/capability 及服务配置。 2. program.ir 与 data_objects.json：保留指令身份、CFG、typed 对象、初始化及 cleanup。 3. resource_audit.json 与 event_obligations.json：独立账本/HB/phase/单位审计。 4. case_result.json 与 coverage_delta.json：区分生成/审计/接收/完成/检查，本阶段真实执行量及 NOT_RUN/BLOCKED 原因。 5. frontend_completion_relation_trace.jsonl、shared_backend_identity_scoreboard.json。

TC-MATRIX-005：Aread/full 和 collector

TC-MATRIX-005 项目	执行内容
源与阶段	SIG-MATRIX-005；P3_SM_RTL；CV-MATRIX-003
配置与 seed	TP-SM-BASE；fixed_seed=0x534d003c（1397555260）；SM_PROGRESS
回归与当前状态	DIRECTED, RANDOM, STRESS, REPLAY；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. 固定 PTX9.4 清单及 SM-ARCH-S0 来源 hash 通过 preflight；独立形式/资源审计器与本阶段 harness 已实现。 2. 由 manifest 绑定精确 target、capability 及合法 form allowlist；未解析的 target/形式保持 UNRESOLVED 并阻止对应正向分支，不推定数字更大的 target 继承能力。 3. SM、对应 adapter 和 monitor 真实可用；全部事务的 owner/epoch/返回身份及服务公平合同已配置。
参数 1	target_binding = MANIFEST_EXACT_TARGET_AND_FORM_ALLOWLIST_REQUIRED
参数 2	unresolved_capability_policy = BLOCK_POSITIVE_BRANCH_AND_REPORT_UNRESOLVED
参数 3	example = 合法 SMEM-A f16 M128xN64xK16
参数 4	restricted_Aread_capability = 必须已解析支持该修饰符的精确 target
参数 5	A_versions = [1,2]

TC-MATRIX-005 项目	执行内容
参数 6	B_value = 1
参数 7	C_seed = 0
参数 8	schedules = ["A-read 观察早于 full","full 观察可早于受限通知交付","collector miss 重读"]
参数 9	barriers = restricted/full 分别使用正确 phase 与 typed 义务
初始化	1. 原 A/B 初始化并发布，旧命令保存正确 source 版本；D 具有合法 seed。 2. Aread 和 normal 完整责任分开，B 可重读与 D lease 未结束。
执行步骤	1. 同一合法 issuer 发 MMA 并登记 restricted/normal commit 及相应观察。 2. 仅在 A-read 正确观察并完成必要线程/proxy 交接后覆写获准原 A；不覆盖 B 或 D。 3. 故意让 collector miss/未来 reuse 读取新版本，同时旧命令保有不可变 snapshot。 4. 等待 normal full 及全部结果/事件 ack 再释放 B/D/allocation/owner。
观察点	1. A-read 不提前清 full token、B 读取义务或整个 shared slice。 2. future lookup 失效实际生效后才 ack，通知交付允许顺序不假设等于物理里程碑顺序。
通过条件	1. 旧命令仍按正确版本计算，新命令只在合法发布后取新版本。 2. source/full 账本与真实事件匹配且最终零孤儿引用。
失败条件	1. 覆盖 A 同时释放 B/D、或 cache 命中被当作早退源许可。 2. 对合法 full 先观察/partial 通知晚交付的次序作错误断言。
预期产物	1. manifest.json：锁定 2026-11-09 测试计划、源 hash、seed/子流、profile、实际 target/capability 及服务配置。 2. program.ir 与 data_objects.json：保留指令身份、CFG、typed 对象、初始化及 cleanup。 3. resource_audit.json 与 event_obligations.json：独立账本/HB/phase/单位审计。 4. case_result.json 与 coverage_delta.json：区分生成/审计/接收/完成/检查，本阶段真实执行量及 NOT_RUN/BLOCKED 原因。 5. Aread_full_and_collector_trace.jsonl、source_version_and_ownership_witness.json。

TC-MATRIX-006：BLOCK 边界与 dot beat 压力

TC-MATRIX-006 项目	执行内容
源与阶段	SIG-MATRIX-006；P3_SM_RTL；CV-MATRIX-003
配置与 seed	TP-SM-BASE；fixed_seed=0x534d003d（1397555261）；SM_PROGRESS

TC-MATRIX-006 项目	执行内容
回归与当前状态	DIRECTED, RANDOM, STRESS, REPLAY；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. 固定 PTX9.4 清单及 SM-ARCH-S0 来源 hash 通过 preflight；独立形式/资源审计器与本阶段 harness 已实现。 2. 由 manifest 绑定精确 target、capability 及合法 form allowlist；未解析的 target/形式保持 UNRESOLVED 并阻止对应正向分支，不推定数字更大的 target 继承能力。 3. SM、对应 adapter 和 monitor 真实可用；全部事务的 owner/epoch/返回身份及服务公平合同已配置。
参数 1	target_binding = MANIFEST_EXACT_TARGET_AND_FORM_ALLOWLIST_REQUIRED
参数 2	unresolved_capability_policy = BLOCK_POSITIVE_BRANCH_AND_REPORT_UNRESOLVED
参数 3	interface_LANES = [1,4]
参数 4	K = [1,15,16,17,31,32,33,64,65]
参数 5	BLOCK16_offsets = [0,1,15]
参数 6	BLOCK32_offsets = [0,1,15,31]
参数 7	AB_offsets = 两侧独立，仅 project adapter 范围；native 先经合法 layout 解包
参数 8	scale_math_values = [0.5,1,2]
参数 9	backpressure = 有限随机握手停顿，无固定 native 周期
参数 10	project_profile = PROJECT_EXACT

TC-MATRIX-006 项目	执行内容
参数 11	named_project_adapter_witness = {"id":"PROJECT_E4M3_K64_BLOCK32_OFFSET15_31","entry":"PROJECT_MALU_ADAPTER; independent from Cshift witness","profile":"PROJECT_EXACT","a_fmt":"FMT_E4M3","b_fmt":"FMT_E4M3","c_fmt":"FMT_F32","d_fmt":"FMT_F32","A_constant_raw_u8":"0x38","B_constant_raw_u8":"0x38","logical_K":64,"block_scale_en":true,"block_size":32,"a_scale_fmt":"SCALE_UE8M0","b_scale_fmt":"SCALE_UE8M0","a_block_offset":15,"b_block_offset":31,"scale_A_payloads_u8":["0x7f","0x7f","0x7f"],"scale_B_payloads_u8":["0x7f","0x7f","0x7f"],"A_block_element_counts":[17,32,15],"B_block_element_counts":[1,32,31],"C_value":0,"C_raw_F32":"0x00000000","c_scale_shift":0,"LANES":4,"expected_beat_counts":[1,4,4,4,4,4,4,4,4,4,4,4,4,4,4,4,3],"expected_result_value":64,"expected_result_F32":"0x42800000","native_gate":"Do not encode arbitrary project offsets/K as an unapproved native tuple."}
初始化	1. 有效 A/B/C 及每逻辑块 scale payload 初始化，格式取项目实际合法组合。 2. 每 dot 锁存 cfg/tag；按两侧最早块边界和 LANES 算合法 beat 计划。
执行步骤	1. 对每 K/block/独立 offset 生成多个合法 beat 切分，C 仅请求一次。 2. 施加 input/output 有限随机 stall，保持 valid 且未 ready 时 payload 稳定。 3. 检查每 beat 不跨任一边界、同块重复 scale 一致、累计 op_count 与 last 恰好等于 K。 4. 消费单一 response 并完成 TMEM 写 ack/parent 责任后，才发下一事务并 cleanup。
观察点	1. K 和块游标不因 stall/beat 重置，C 不重复加入。 2. 同逻辑 dot 换切分保持 PROJECT_EXACT 结果；协议 mutation 单列 NONISA。 3. 命名 BLOCK32 见证 A/B 各需 3 项 scale，分别覆盖 17/32/15 与 1/32/31 元素；17 拍 count=[1,4×15,3]，总数 64 且结果 64。
通过条件	1. 所有合法切分数值一致、身份/长度/scale 契约满足。 2. adapter 与完整 SM 缺失时仍 blocked，不因既有 MALU 回归标该 case 已运行。
失败条件	1. 跨边界 beat、块内 payload 改变、early/late last 或输出 tag 串事务。 2. 用 LANES 缓存槽当每周期并行乘法资源或硬编码完成周期。

TC-MATRIX-006 项目	执行内容
预期产物	1. manifest.json：锁定 2026-11-09 测试计划、源 hash、seed/子流、profile、实际 target/capability 及服务配置。 2. program.ir 与 data_objects.json：保留指令身份、CFG、typed 对象、初始化及 cleanup。 3. resource_audit.json 与 event_obligations.json：独立账本/HB/phase/单位审计。 4. case_result.json 与 coverage_delta.json：区分生成/审计/接收/完成/检查，本阶段真实执行量及 NOT_RUN/BLOCKED 原因。 5. dot_beat_plan.json、adapter_handshake_trace.jsonl、beat_invariance_results.json。

20.15 GEN-CLUSTER-01：Cluster/DSM/CTA-pair 与 grid 控制

TC-CLUSTER-001：cluster 资源映射

TC-CLUSTER-001 项目	执行内容
源与阶段	SIG-CLUSTER-001；P1_GENERATOR_UNIT；CV-CLUSTER-001
配置与 seed	TP-GEN-BASE；fixed_seed=0x534d003e（1397555262）；GEN_BOUNDED
回归与当前状态	SMOKE, DIRECTED, NEGATIVE, REPLAY；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. 固定 PTX9.4 清单及 SM-ARCH-S0 来源 hash 通过 preflight；独立形式/资源审计器与本阶段 harness 已实现。 2. 由 manifest 绑定精确 target、capability 及合法 form allowlist；未解析的 target/形式保持 UNRESOLVED 并阻止对应正向分支，不推定数字更大的 target 继承能力。
参数 1	target_binding = MANIFEST_EXACT_TARGET_AND_FORM_ALLOWLIST_REQUIRED
参数 2	unresolved_capability_policy = BLOCK_POSITIVE_BRANCH_AND_REPORT_UNRESOLVED
参数 3	cluster_shapes = [[2,1,1],[4,1,1]]
参数 4	warps_each_CTA = 4
参数 5	shared_visible_bytes_each = 16384
参数 6	placement = 按 target 允许 cluster 和 S0 pair 约束解析
参数 7	negative_variants = ["group2 pair 同 SM","缺一个 peer 资源","rank 映射错误","shared 总预算不足"]

TC-CLUSTER-001 项目	执行内容
初始化	1. cluster membership/rank→SM-owner 表及 R_SM/SMEM reservation 为空且可核对。 2. 确切 target 的 cluster 维度/共同驻留条件未解析时不接纳正向 launch。
执行步骤	1. 构造两 CTA 和四 CTA 合法 cluster 候选，独立联合计算所有参与 SM 的资源。 2. 检查 even/odd pair 放到两个可互通不同 endpoint，所有 owner 一起成功才发布 launch。 3. 针对放置/资源/rank 作单因素 mutation，确认没有部分 cluster 开始执行。 4. 合法场景建立各 owner shared 引用并模拟全部远端责任关闭后退出回收。
观察点	1. cluster 范围与 pair 范围不同；任意 DSM 目标不是当前 Tensor peer。 2. 失败 admission 不泄漏单边资源或留下运行 CTA。
通过条件	1. 合法映射/预算正确，负例准确在项目放置/资源层拒绝。 2. 不同 owner 同 offset 不产生错误 physical 别名。
失败条件	1. 随机凑 peer 或先启动 CTA 再等基础资源导致无解除者。 2. 把 S0 同 SM group2 限制误称 PTX 对所有硬件的通用规则。
预期产物	1. manifest.json：锁定 2026-11-09 测试计划、源 hash、seed/子流、profile、实际 target/capability 及服务配置。 2. program.ir 与 data_objects.json：保留指令身份、CFG、typed 对象、初始化及 cleanup。 3. resource_audit.json 与 event_obligations.json：独立账本/HB/phase/单位审计。 4. case_result.json 与 coverage_delta.json：区分生成/审计/接收/完成/检查，本阶段真实执行量及 NOT_RUN/BLOCKED 原因。

TC-CLUSTER-002：cancel 与 grid 模板

TC-CLUSTER-002 项目	执行内容
源与阶段	SIG-CLUSTER-002；P1_GENERATOR_UNIT；CV-CLUSTER-002
配置与 seed	TP-GEN-BASE；fixed_seed=0x534d003f（1397555263）；GEN_BOUNDED
回归与当前状态	DIRECTED, NEGATIVE, REPLAY；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. 固定 PTX9.4 清单及 SM-ARCH-S0 来源 hash 通过 preflight；独立形式/资源审计器与本阶段 harness 已实现。 2. 由 manifest 绑定精确 target、capability 及合法 form allowlist；未解析的 target/形式保持 UNRESOLVED 并阻止对应正向分支，不推定数字更大的 target 继承能力。
参数 1	target_binding = MANIFEST_EXACT_TARGET_AND_FORM_ALLOWLIST_REQUIRED

TC-CLUSTER-002 项目	执行内容
参数 2	unresolved_capability_policy = BLOCK_POSITIVE_BRANCH_AND_REPORT_UNRESOLVED
参数 3	runtime_work_ids = [0,1,2,3]
参数 4	initial_running_ids = [0]
参数 5	initial_pending_ids = [1,2,3]
参数 6	cancel_response_bytes = 16
参数 7	response_storage_alignment_bytes = 256
参数 8	response_and_barrier = 独立合法 shared 对象；barrier layout/tx 按 CLC recipe
参数 9	branches = ["cancel 成功","cancel 失败后不再 try_cancel"]
参数 10	grid_dependency = launch_dependents 与 wait 分别建模
初始化	1. 工作队列、唯一 ID 与 pending/running 状态由 runtime fixture 建立。 2. response bytes 未定义，barrier 已合法初始化并正确登记 16B 完成责任。
执行步骤	1. 生成对尚未启动 cluster 的合法 try_cancel 意图及对应响应/等待程序。 2. 独立模型分别返回成功 ID 或失败，只有完成/正确查询后才使用 opaque response。 3. 成功分支接管唯一工作；失败观察后禁止再 try_cancel，并保留该规则负例。 4. 生成 grid 依赖通知/等待模板，完成 response/barrier 及工作引用后 cleanup。
观察点	1. try_cancel 不是 kill running cluster，成功 ID 唯一且不是猜 response 内部位编码。 2. launch_dependents 不代表当前 grid 完成；wait 按 prerequisite 语义。
通过条件	1. 所有分支状态/义务完整，失败后非法重试被指定审计器拒绝。 2. response 未完成前不读其 ID/成功值。
失败条件	1. 取消正在运行 CTA、重复领取成功 ID，或查询前把 16B 响应当普通整数。 2. 缺少 runtime 服务而伪造本地 dummy done。
预期产物	1. manifest.json：锁定 2026-11-09 测试计划、源 hash、seed/子流、profile、实际 target/capability 及服务配置。 2. program.ir 与 data_objects.json：保留指令身份、CFG、typed 对象、初始化及 cleanup。 3. resource_audit.json 与 event_obligations.json：独立账本/HB/phase/单位审计。 4. case_result.json 与 coverage_delta.json：区分生成/审计/接收/完成/检查，本阶段真实执行量及 NOT_RUN/BLOCKED 原因。 5. reference_runtime_cancel_trace.jsonl、opaque_response_lifetime_audit.json。

TC-CLUSTER-003：DSM 与可见性

TC-CLUSTER-003 项目	执行内容
源与阶段	SIG-CLUSTER-003；P2_REFERENCE_PROGRAM；CV-CLUSTER-001
配置与 seed	TP-REF-BASE；fixed_seed=0x534d0040（1397555264）；REFERENCE_BOUNDED
回归与当前状态	DIRECTED, RANDOM, REPLAY；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. 固定 PTX9.4 清单及 SM-ARCH-S0 来源 hash 通过 preflight；独立形式/资源审计器与本阶段 harness 已实现。 2. 由 manifest 绑定精确 target、capability 及合法 form allowlist；未解析的 target/形式保持 UNRESOLVED 并阻止对应正向分支，不推定数字更大的 target 继承能力。 3. 独立语义/内存/事件模型及其允许结果集合已锁定；生成器 shadow 不能作为唯一 oracle。
参数 1	target_binding = MANIFEST_EXACT_TARGET_AND_FORM_ALLOWLIST_REQUIRED
参数 2	unresolved_capability_policy = BLOCK_POSITIVE_BRANCH_AND_REPORT_UNRESOLVED
参数 3	ctas = 2
参数 4	sms = 2
参数 5	shared_bytes_each = 1024
参数 6	same_offset = 0
参数 7	initial_values_each = [17,34]
参数 8	DSM_payload_bytes = 64
参数 9	orders = ["远端写后正确同步再读","远端尾响应晚到但目标保持 live"]
参数 10	target_exit = remote 责任关闭后的合法 cluster 顺序
初始化	1. 两 CTA 共同 admit 并使用不同 generation/shared base，本地 offset0 内容不同。 2. mapa/getctarank 采用 typed 转换，remote view 映射到真实目标 owner。
执行步骤	1. 构造本地/remote shared 访问及正确 release/acquire 或 cluster 同步 recipe。 2. 模型探索远端写/响应及 consumer 观察顺序，target 在所有引用结束前不退出。 3. 核对同 offset 不同 owner 隔离和 remote 消费 HB；删除关键同步/提前 exit 作为独立 mutant。 4. 双方关闭 remote 返回、barrier 与 owner pin 后执行合法退出。
观察点	1. 不同 owner 字节对象与 rank 映射准确，不能裸异或 SM 编号造地址。 2. remote 对象 live 与可见性是两个条件，任何一个缺失都不允许消费。

TC-CLUSTER-003 项目	执行内容
通过条件	1. 所有允许合法次序满足模型关系，mutant 因缺失 HB/lifetime 拒绝。 2. owner 复用前引用为零，旧 generation 不会映射到新对象。
失败条件	1. 只因目标存在就忽略同步，或只因同步就允许已退出 owner。 2. 参考模型硬编码一条网络延迟导致漏掉合法长尾。
预期产物	1. manifest.json：锁定 2026-11-09 测试计划、源 hash、seed/子流、profile、实际 target/capability 及服务配置。 2. program.ir 与 data_objects.json：保留指令身份、CFG、typed 对象、初始化及 cleanup。 3. resource_audit.json 与 event_obligations.json：独立账本/HB/phase/单位审计。 4. case_result.json 与 coverage_delta.json：区分生成/审计/接收/完成/检查，本阶段真实执行量及 NOT_RUN/BLOCKED 原因。 5. reference_DSM_HB_and_owner_trace.jsonl、rank_mapping_witness.json。

TC-CLUSTER-004：真实 cluster 与工作服务

TC-CLUSTER-004 项目	执行内容
源与阶段	SIG-CLUSTER-004；P4_SOC；CV-CLUSTER-003
配置与 seed	TP-SOC-BASE；fixed_seed=0x534d0041（1397555265）；SOC_PROGRESS
回归与当前状态	DIRECTED, RANDOM, STRESS, REPLAY；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. 固定 PTX9.4 清单及 SM-ARCH-S0 来源 hash 通过 preflight；独立形式/资源审计器与本阶段 harness 已实现。 2. 由 manifest 绑定精确 target、capability 及合法 form allowlist；未解析的 target/形式保持 UNRESOLVED 并阻止对应正向分支，不推定数字更大的 target 继承能力。 3. 跨 SM/SoC 运行时、对应服务及取消/返回模型可用；未实现系统能力保持 blocked。
参数 1	target_binding = MANIFEST_EXACT_TARGET_AND_FORM_ALLOWLIST_REQUIRED
参数 2	unresolved_capability_policy = BLOCK_POSITIVE_BRANCH_AND_REPORT_UNRESOLVED
参数 3	cluster_ctas = 2
参数 4	sms = 2
参数 5	queued_clusters = 3
参数 6	runtime_variants = ["取消成功","取消失败","dependent grid 可早 launch 但 wait 未满足"]

TC-CLUSTER-004 项目	执行内容
参数 7	service_orders = ["本地早/远端尾","合法乱序","有限控制反压"]
参数 8	work_data_bytes = 256
初始化	1. 工作 ID/rank/依赖图、初始 payload 和所有 owner/barrier 由 SoC fixture 建立。 2. source/phase/tx 登记和 cancel 成功/失败的定义合同锁定，不猜 runtime 内部编码。
执行步骤	1. 实际运行多 SM cluster 通信并注入有限网络/返回扰动。 2. 分别取消可取消 queued 工作及观察失败，跟踪唯一 ID 与响应完整性。 3. 允许 dependent grid 早启动独立工作，但在读取前置结果前执行规定 wait。 4. 等待 prerequisite/远端数据/控制完成，drain 所有 cluster 引用后回收 owner。
观察点	1. cancel 不杀运行 CTA；成功 ID 不重复，失败后不非法重试。 2. dependent launch 不是数据可见，读取由真正 wait/HB 保护。
通过条件	1. 真实 SoC/DSM/runtime 证据与独立模型一致，所有退出责任结清。 2. 公平服务下长尾最终前进，未用 dummy 回应提前清计数。
失败条件	1. ID 重复、response/phase 错配、dependent 提前读取未完成数据。 2. CTA 退出忽略远端引用或故意服务永久 stall 仍按正常 pass。
预期产物	1. manifest.json：锁定 2026-11-09 测试计划、源 hash、seed/子流、profile、实际 target/capability 及服务配置。 2. program.ir 与 data_objects.json：保留指令身份、CFG、typed 对象、初始化及 cleanup。 3. resource_audit.json 与 event_obligations.json：独立账本/HB/phase/单位审计。 4. case_result.json 与 coverage_delta.json：区分生成/审计/接收/完成/检查，本阶段真实执行量及 NOT_RUN/BLOCKED 原因。 5. soc_cluster_runtime_trace.jsonl、grid_dependency_visibility_witness.json。

20.16 GEN-FABRIC-01：Fabric/CFT 与 multimem

TC-FABRIC-001：句柄和单位约束

TC-FABRIC-001 项目	执行内容
源与阶段	SIG-FABRIC-001；P1_GENERATOR_UNIT；CV-FABRIC-001
配置与 seed	TP-GEN-BASE；fixed_seed=0x534d0042（1397555266）；GEN_BOUNDED
回归与当前状态	SMOKE, DIRECTED, NEGATIVE, REPLAY；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION

TC-FABRIC-001 项目	执行内容
前置条件	1. 固定 PTX9.4 清单及 SM-ARCH-S0 来源 hash 通过 preflight；独立形式/资源审计器与本阶段 harness 已实现。 2. 由 manifest 绑定精确 target、capability 及合法 form allowlist；未解析的 target/形式保持 UNRESOLVED 并阻止对应正向分支，不推定数字更大的 target 继承能力。
参数 1	target_binding = MANIFEST_EXACT_TARGET_AND_FORM_ALLOWLIST_REQUIRED
参数 2	unresolved_capability_policy = BLOCK_POSITIVE_BRANCH_AND_REPORT_UNRESOLVED
参数 3	CFT_carriers = {"leId":"u32","offset":"u64"}
参数 4	transfer_bytes = [16,32,256]
参数 5	get_pullred_tx_units = [16,32,256]
参数 6	put_red_tx_units = [1,2,16]
参数 7	counter_bytes = 8
参数 8	counter_alignment_bytes = 256
参数 9	valid_counter_offset = 256
参数 10	data_offset = 0
参数 11	data_bytes = 256
参数 12	mutations = ["counter 仅 8B 对齐","counter 与 data 重叠","layout_v0","tx 单位错误","普通 ld/st 访问 multimemVA"]
初始化	1. host/runtime fixture 提供合法 CFT endpoint 绑定和 resource extent，服务未实现不声称存在真实连接。 2. 本地 shared 源/目标与 layoutv1 barrier 独立初始化，counter 位于不重叠合法 range。
执行步骤	1. 生成各操作/size 的 handle、单位和 barrier 约束模板，独立重算 tx 责任。 2. 单因素变异 carrier/endpoint 范围/counter 对齐/overlap/layout/units。 3. 核查 ordinary VA、multimem VA 与 CFT 类型不能裸 cast 互用，关闭能力仅走 preflight。 4. 正向 IR 保留 submit/观察/drain 尾部，负例停在预期审计层。
观察点	1. get/pullred 按 bytes，put/red 按 16B units；counter 按最终目的 bytes 另计。 2. allwarp pullred 参与约束来自具体 form，不套一般 subset 规则。
通过条件	1. 合法 tuple 及 typed 义务完整；所有 mutant 精确拒绝。 2. exact target/CFT 支持未解析即阻止正向分支。

TC-FABRIC-001 项目	执行内容
失败条件	1. counter 只按 8B 自然对齐即接受，或 units 被统一成 bytes。 2. 无系统能力还生成可执行成功 Fabric 命令。
预期产物	1. manifest.json：锁定 2026-11-09 测试计划、源 hash、seed/子流、profile、实际 target/capability 及服务配置。 2. program.ir 与 data_objects.json：保留指令身份、CFG、typed 对象、初始化及 cleanup。 3. resource_audit.json 与 event_obligations.json：独立账本/HB/phase/单位审计。 4. case_result.json 与 coverage_delta.json：区分生成/审计/接收/完成/检查，本阶段真实执行量及 NOT_RUN/BLOCKED 原因。

TC-FABRIC-002：提交/等待程序模板

TC-FABRIC-002 项目	执行内容
源与阶段	SIG-FABRIC-002；P1_GENERATOR_UNIT；CV-FABRIC-002
配置与 seed	TP-GEN-BASE；fixed_seed=0x534d0043（1397555267）；GEN_BOUNDED
回归与当前状态	DIRECTED, NEGATIVE, REPLAY；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. 固定 PTX9.4 清单及 SM-ARCH-S0 来源 hash 通过 preflight；独立形式/资源审计器与本阶段 harness 已实现。 2. 由 manifest 绑定精确 target、capability 及合法 form allowlist；未解析的 target/形式保持 UNRESOLVED 并阻止对应正向分支，不推定数字更大的 target 继承能力。
参数 1	target_binding = MANIFEST_EXACT_TARGET_AND_FORM_ALLOWLIST_REQUIRED
参数 2	unresolved_capability_policy = BLOCK_POSITIVE_BRANCH_AND_REPORT_UNRESOLVED
参数 3	logical_operations = ["get","put","red","pullred_if_legal"]
参数 4	transfer_bytes_each = 256
参数 5	completion_units = {"get":256,"put":16,"red":16,"pullred":256}
参数 6	barrier_layout = v1
参数 7	arrival_registration = ["原子 arrive.expect_tx","先 expect_tx 并保留 pending arrival 后 arrive"]
参数 8	schedules = ["complete 早于登记但 arrival 保留","complete 晚于登记"]
参数 9	pullred_participants = 完整未退出 warp 且规定全 membermask、相同 operand

TC-FABRIC-002 项目	执行内容
参数 10	negative_trial = {"class":"ISA_UB_AUDIT_ONLY_QUARANTINE","baseline":"fully audited issue-submit-atomic-expect/arrive-primary-wait-report program","single_mutation":"remove exactly one mandatory fabric.submit from the issuing-thread prefix before the tracked phase can advance","injection_layer":"PROGRAM_LEGALIZER_HB_AUDIT_ONLY","expected_detector_rule":"MISSING_FABRIC_SUBMIT_BEFORE_TRACKED_PHASE_ADVANCE","DUT_execution_allowed":false,"numeric_or_trap_oracle":"none for the invalid copy"}
初始化	1. CFT fixture、本地 payload、opaque barrier 及对应 typed 单位明确初始化。 2. issuing thread/group 身份确定，pending arrival 在最后登记之前不会提前降至零。
执行步骤	1. 发起合法操作并由对应 thread submit，fetching 限定只包括 get/pullred。 2. 分别采用原子 arrive.expect_tx 或先 expect 并保留 arrival 后 arrive，探索 complete 早/晚的允许顺序。 3. 观察 primary 完成后检查合法 report；fabric.wait 仅用于 local shared 源读安全复用。 4. 在审计专用副本中仅删除跟踪 phase 可能推进前发射 thread 的一次必需 fabric.submit；独立 program/HB 审计按 MISSING_FABRIC_SUBMIT_BEFORE_TRACKED_PHASE_ADVANCE 拒绝，副本隔离且不执行设备行为。 5. 等待最终所有责任并关闭 source/目的/报告，再生成 grid 退出尾部。
观察点	1. submit 先于其 phase 推进，最后 arrival 不在 tx0 时先释放再登记期望。 2. Fabric local-read 终点不清 try_get 目的或远端 full 责任。 3. NEGATIVE 副本仅缺一次必需 submit，拒绝层为 PROGRAM_LEGALIZER/HB audit；合法 baseline 不变，非法副本不期待数值/自动 trap。
通过条件	1. 所有模板在允许顺序下均无提前 phase/错误复用；typed 计数恰好配平。 2. 完整 warp pullred 只计其规范逻辑操作责任，不随日志 lane 数膨胀完成计数。
失败条件	1. 误把 arrive/expect 斜杠实现为先最后 arrive 后 expect，或 submit 遗漏。 2. 只用 fabric.wait 就结束 grid/消费 get 目的，或成功清旧报告。
预期产物	1. manifest.json：锁定 2026-11-09 测试计划、源 hash、seed/子流、profile、实际 target/capability 及服务配置。 2. program.ir 与 data_objects.json：保留指令身份、CFG、typed 对象、初始化及 cleanup。 3. resource_audit.json 与 event_obligations.json：独立账本/HB/phase/单位审计。 4. case_result.json 与 coverage_delta.json：区分生成/审计/接收/完成/检查，本阶段真实执行量及 NOT_RUN/BLOCKED 原因。 5. fabric_program_and_phase_audit.json、source_full_obligation_witness.json。

TC-FABRIC-003：报告和 counter 模型

TC-FABRIC-003 项目	执行内容
源与阶段	SIG-FABRIC-003；P2_REFERENCE_PROGRAM；CV-FABRIC-001
配置与 seed	TP-REF-BASE；fixed_seed=0x534d0044（1397555268）；REFERENCE_BOUNDED
回归与当前状态	DIRECTED, RANDOM, REPLAY；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. 固定 PTX9.4 清单及 SM-ARCH-S0 来源 hash 通过 preflight；独立形式/资源审计器与本阶段 harness 已实现。 2. 由 manifest 绑定精确 target、capability 及合法 form allowlist；未解析的 target/形式保持 UNRESOLVED 并阻止对应正向分支，不推定数字更大的 target 继承能力。 3. 独立语义/内存/事件模型及其允许结果集合已锁定；生成器 shadow 不能作为唯一 oracle。
参数 1	target_binding = MANIFEST_EXACT_TARGET_AND_FORM_ALLOWLIST_REQUIRED
参数 2	unresolved_capability_policy = BLOCK_POSITIVE_BRANCH_AND_REPORT_UNRESOLVED
参数 3	layout = v1
参数 4	same_phase_operations = 3
参数 5	report_schedules = ["失败→成功","成功→失败","多个失败允许报告集合"]
参数 6	successful_counter_expected_bytes = 256
参数 7	successful_counter_partials = [16,64,128,256]
参数 8	multimem_destinations = 2
参数 9	failed_transfer_data_counter = 仅按规定错误/允许集合检查，不断言失败仍写出完整正常 payload
初始化	1. 成功 counter 场景 counter=0 且 256B 对齐；失败报告场景单独 fixture，不混用不兼容 barrier 报告机制。 2. 各目的独立 data/counter/phase 责任与允许错误集合建立，opaque reportValue 不猜固定错误编号。
执行步骤	1. 参考模型生成同 phase 多成功/失败事件，分别探索其可允许的完成顺序。 2. 成功传输场景让 counter 分段增长，仅最终对应目的 bytes 满足终点后允许完成。 3. 失败场景按 primary/report 结算，保留可能的部分副作用，不无条件重试 reduce。 4. 逐目的收齐合法终点/错误并 drain source/目的/报告引用。
观察点	1. 成功事件不擦除同 phase 已有错误；report 只在有效观察时消费。 2. counter 中间增长不当 full，multimem 各目的 counter 仅覆盖自身访问。

TC-FABRIC-003 项目	执行内容
通过条件	1. 报告关系、成功最终 counter 与多目的责任符合独立模型。 2. 失败通过定义错误路径结清而非伪造正常零数据或强制完整成功 counter。
失败条件	1. partial counter 提前释放 parent，或失败操作被盲重试导致重复副作用。 2. 错误 oracle 按固定自造 report 编码或把不同目的计数合并。
预期产物	1. manifest.json：锁定 2026-11-09 测试计划、源 hash、seed/子流、profile、实际 target/capability 及服务配置。 2. program.ir 与 data_objects.json：保留指令身份、CFG、typed 对象、初始化及 cleanup。 3. resource_audit.json 与 event_obligations.json：独立账本/HB/phase/单位审计。 4. case_result.json 与 coverage_delta.json：区分生成/审计/接收/完成/检查，本阶段真实执行量及 NOT_RUN/BLOCKED 原因。 5. reference_fabric_report_counter_trace.jsonl、allowed_error_and_side_effects.json。

TC-FABRIC-004：系统传输失败与退出

TC-FABRIC-004 项目	执行内容
源与阶段	SIG-FABRIC-004；P4_SOC；CV-FABRIC-003
配置与 seed	TP-SOC-FAULT；fixed_seed=0x534d0045（1397555269）；SOC_PROGRESS
回归与当前状态	DIRECTED, STRESS, REPLAY；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. 固定 PTX9.4 清单及 SM-ARCH-S0 来源 hash 通过 preflight；独立形式/资源审计器与本阶段 harness 已实现。 2. 由 manifest 绑定精确 target、capability 及合法 form allowlist；未解析的 target/形式保持 UNRESOLVED 并阻止对应正向分支，不推定数字更大的 target 继承能力。 3. 跨 SM/SoC 运行时、对应服务及取消/返回模型可用；未实现系统能力保持 blocked。
参数 1	target_binding = MANIFEST_EXACT_TARGET_AND_FORM_ALLOWLIST_REQUIRED
参数 2	unresolved_capability_policy = BLOCK_POSITIVE_BRANCH_AND_REPORT_UNRESOLVED
参数 3	legal_faults = ["可报告 transport 访问失败","一个目的合法长尾"]
参数 4	logical_operations = ["get","put","red_if_capable"]
参数 5	transfer_bytes = 256
参数 6	proxy_domains = {"CFT":"fabric","local_shared":"async","barrier":"generic"}

TC-FABRIC-004 项目	执行内容
参数 7	retry_policy = 仅应用已定义且证明幂等/明确完成的分支允许重试；无自动重试 reduce
初始化	1. 真实或锁定 SoC 服务 fixture 提供 CFT 与 layoutv1 对象；故障注入类型受定义服务合同限制。 2. 初始化 payload/canary、报告和每操作 typed units，记录已承诺外部副作用。
执行步骤	1. 实际提交合法 Fabric 程序，注入可报告故障及有限延迟，而非构造 UB 指令。 2. 分别观察 source-read、目的/远端 full 及 primary/report，保持 proxy 顺序。 3. 按程序定义错误分支停止/收尾或有证明的重试，记录部分外部副作用与取消确认。 4. grid 退出前 drain 所有 Fabric 责任；失败不被改写成正常零数据结果。
观察点	1. 失败操作仍 exactly-once 结算并保持 owner 到相应终点。 2. proxy/domain/单位与真实报告匹配，不能仅 FIFO 接收即结束。
通过条件	1. 数据成功或定义报告失败均按各自 oracle 正确分类，外部副作用无重复。 2. 所有已接受事务完成或正式取消，退出不泄漏系统引用。
失败条件	1. 将合法服务错误当 UB/trap，或吞错为正常值。 2. 对不明部分成功 reduce 自动重发，或过滤旧响应冒称撤销远端写。
预期产物	1. manifest.json：锁定 2026-11-09 测试计划、源 hash、seed/子流、profile、实际 target/capability 及服务配置。 2. program.ir 与 data_objects.json：保留指令身份、CFG、typed 对象、初始化及 cleanup。 3. resource_audit.json 与 event_obligations.json：独立账本/HB/phase/单位审计。 4. case_result.json 与 coverage_delta.json：区分生成/审计/接收/完成/检查，本阶段真实执行量及 NOT_RUN/BLOCKED 原因。 5. soc_fabric_transport_trace.jsonl、report_side_effect_cancel_ledger.json、grid_drain_witness.json。

20.17 GEN-TEX-01：Texture/surface 关闭与可选服务

TC-TEX-001：默认关闭门禁

TC-TEX-001 项目	执行内容
源与阶段	SIG-TEX-001；P1_GENERATOR_UNIT；CV-TEX-001
配置与 seed	TP-GEN-BASE；fixed_seed=0x534d0046（1397555270）；GEN_BOUNDED
回归与当前状态	SMOKE, DIRECTED, NEGATIVE；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION

TC-TEX-001 项目	执行内容
前置条件	1. 固定并核对源 vPlan/testpoint、PTX9.4 inventory、target/capability、规则及模型版本 hash；fixed_seed 由主 campaign 在运行前赋值。 2. 发生器、unit harness 和独立合法性/结构审计器必须可用；共享 source 身份不等于共享待测约束函数。 3. 执行本 case 前冻结 timeout 预算；缺预算、模型、工具或服务时保持 BLOCKED，不以空执行关闭 case。
参数 1	timeout_budget_ref = campaign 配置中与 timeout_policy 对应的有限预算；不在本 case 猜测 SM 固定周期。
参数 2	texture_surface_capability = DISABLED
参数 3	candidate_source = independently legal per-target texture/surface recipes
参数 4	single_factor = same candidate with only project enable bit changed in auditor-only comparison
参数 5	expected_layer = GENERATOR_LEGALIZER
参数 6	expected_reason = CAPABILITY_DISABLED
初始化	1. default 计算型 profile 关闭 texture/surface，任何 functional issue 队列为空。 2. 合法对象 recipe 只用于候选审计，不假设真实 sampler 存在。
执行步骤	1. 独立选取规范 target 合法的 texture/surface 候选并保存其对象/维度/类型证据。 2. 保持 profile disabled 提交发生器/合法化入口，记录 CAPABILITY_DISABLED 及 source identity。 3. 在隔离 auditor 对照中只改变 enable bit 以验证拒绝由 project capability 而非错误句柄/target 引起；不因此启动未实现服务。 4. 核对 native 输出和 DUT issue 计数为零，将拒绝证据归入负例/门禁覆盖而非功能执行覆盖。
观察点	1. 候选规范合法而 project 关闭，两种状态不混。 2. 拒绝发生在规定入口，无 no-op/伪支持/实际服务请求。
通过条件	1. 门禁精确拒绝关闭候选，合法性对照能区分 target 与 capability 原因。 2. 实际执行量、检查量及所需覆盖非空，所有必需检查满足；本 case 当前 NOT_RUN 不因计划/静态检查而变为 PASS。
失败条件	1. 关闭时发出 sampler/surface 事务，或把入口拒绝算正向服务 PASS。 2. 用本来就非法的对象掩盖能力门禁缺陷。

TC-TEX-001 项目	执行内容
预期产物	1. manifest.json 2. case_result.json 3. coverage_delta.json 4. texture_gate_verdicts.json 5. candidate_object_manifest.json 6. issue_counter_snapshot.json

TC-TEX-002：可选对象合法化

TC-TEX-002 项目	执行内容
源与阶段	SIG-TEX-002；P1_GENERATOR_UNIT；CV-TEX-002
配置与 seed	TP-GEN-BASE；fixed_seed=0x534d0047（1397555271）；GEN_BOUNDED
回归与当前状态	DIRECTED, NEGATIVE；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. 固定并核对源 vPlan/testpoint、PTX9.4 inventory、target/capability、规则及模型版本 hash；fixed_seed 由主 campaign 在运行前赋值。 2. 发生器、unit harness 和独立合法性/结构审计器必须可用；共享 source 身份不等于共享待测约束函数。 3. 执行本 case 前冻结 timeout 预算；缺预算、模型、工具或服务时保持 BLOCKED，不以空执行关闭 case。
参数 1	timeout_budget_ref = campaign 配置中与 timeout_policy 对应的有限预算；不在本 case 猜测 SM 固定周期。
参数 2	profile_enable = isolated future capability snapshot only
参数 3	axes = ["texture_or_surface","dimension","channel_type","pixel_vs_byte_coordinate","LOD_or_boundary_mode"]
参数 4	mutation_cardinality = 1
参数 5	negative_change = replace one opaque handle/object-kind reference with incompatible registry kind
参数 6	expected_layer = OBJECT_LEGALIZER
初始化	1. 启用对照仅用于独立合法化，不宣称当前 SoC 服务实现。 2. 对象 registry 给出 exact kind、格式、维度与 source-target recipe；opaque handle 不当普通地址。

TC-TEX-002 项目	执行内容
执行步骤	1. 在隔离启用 snapshot 登记一组合法 texture 和 surface 对象，按各自指令规则生成 dim/channel/坐标及 LOD/boundary 参数。 2. 用独立 registry 和 target 表审计每个候选，明确 surface byte/element 地址语义与 texture 采样 address mode。 3. 从合法候选只变更一个对象 kind/handle 引用，保持其余 operand 和 recipe 不变，送 object legalizer。 4. 核对基线接受、mutant 在 object-kind 规则拒绝，保存字段差分；未冻结/未支持组合标 BLOCKED 或 disabled，不填造 native payload。
观察点	1. 合法对象属性与指令期待相符，surface/texture 坐标边界语义分开。 2. 单因素对象变异被精确定位，未改变 target/format 掩盖错误。
通过条件	1. 所有已登记合法候选通过，单对象变异被拒绝；未知组合不被静默合法化。 2. 实际执行量、检查量及所需覆盖非空，所有必需检查满足；本 case 当前 NOT_RUN 不因计划/静态检查而变为 PASS。
失败条件	1. 句柄当普通 pointer 处理，或把 surface boundary 与 texture address mode 混用。 2. 候选同时违反多个约束，无法归因拒绝。
预期产物	1. manifest.json 2. case_result.json 3. coverage_delta.json 4. object_registry_snapshot.json 5. object_recipe_audit.json 6. handle_kind_diff.json

TC-TEX-003：独立采样模型

TC-TEX-003 项目	执行内容
源与阶段	SIG-TEX-003；P2_REFERENCE_PROGRAM；CV-TEX-001
配置与 seed	TP-REF-BASE；fixed_seed=0x534d0048（1397555272）；REFERENCE_BOUNDED
回归与当前状态	DIRECTED；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. 固定并核对源 vPlan/testpoint、PTX9.4 inventory、target/capability、规则及模型版本 hash；fixed_seed 由主 campaign 在运行前赋值。 2. 所用独立参考解释器/允许结果模型及 lowering 或服务 reference route 必须可用；P2 结果不计真实 SM 执行。 3. 执行本 case 前冻结 timeout 预算；缺预算、模型、工具或服务时保持 BLOCKED，不以空执行关闭 case。

TC-TEX-003 项目	执行内容
参数 1	timeout_budget_ref = campaign 配置中与 timeout_policy 对应的有限预算；不在本 case 猜测 SM 固定周期。
参数 2	image_extent = [4,4]
参数 3	pattern = P(x,y,c)=16*x+4*y+c; component type/normalization from validated object recipe
参数 4	operations = ["nearest","filter","gather","query","surface_load_store_reduce where legal"]
参数 5	coordinate_classes = ["texel_centers","interior_midpoints","documented_boundaries"]
参数 6	oracle = independent sampler/channel-conversion/boundary model; target-allowed precision sets
初始化	1. 独立 sampler 与 surface reference 模型和对象规范已实现；当前 DUT 能力不因此开启。 2. 静态纹理 pattern 已初始化；surface 读写阶段采用本对象服务明确的完成/顺序合同，不附加 generic coherence。
执行步骤	1. 用可解析 4x4 pattern 构建合法对象，分别登记 channel conversion、normalized coordinate、address/filter/LOD 规则。 2. 生成 texel center 与内部 midpoint 样本，对 nearest/filter/gather/query 逐操作运行 reference，按允许精度关系比较。 3. 在独立 surface 对象上执行合法 load/store/reduce 并按已声明服务顺序读取结果，检查 pixel/byte 坐标和 channel 值。 4. 对文档边界样本应用各对象自己的 address/boundary 模式，不用普通 global load 或 generic fence 假定替代对象语义。
观察点	1. 中心/中点/gather/query 结果符合独立对象模型和允许数值误差，不按任意精确 bit 值误判 filter。 2. surface 副作用及边界处理遵循本对象合同，texture/surface 未被加上普通 generic 内存模型保证。
通过条件	1. 每个启用 reference 操作有非空独立结果检查，定义域与目标允许集合都满足。 2. 实际执行量、检查量及所需覆盖非空，所有必需检查满足；本 case 当前 NOT_RUN 不因计划/静态检查而变为 PASS。
失败条件	1. 用同一 sampler 实现给自己做 oracle，或普通 global load 冒充 filter/gather/channel 转换。 2. 凭 generic store/fence 推断 texture 可见，或错误复用 surface/texture 边界模式。

TC-TEX-003 项目	执行内容
预期产物	1. manifest.json 2. case_result.json 3. coverage_delta.json 4. image_pattern.json 5. sampler_reference_results.json 6. surface_reference_events.jsonl 7. allowed_precision_checks.json

TC-TEX-004：实际可选服务

TC-TEX-004 项目	执行内容
源与阶段	SIG-TEX-004；P4_SOC；CV-TEX-003
配置与 seed	TP-SOC-BASE；fixed_seed=0x534d0049（1397555273）；SOC_PROGRESS
回归与当前状态	DIRECTED, STRESS；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. 固定并核对源 vPlan/testpoint、PTX9.4 inventory、target/capability、规则及模型版本 hash；fixed_seed 由主 campaign 在运行前赋值。 2. 真实 SoC/SM adapter 与本 case 启用的对象/服务、端到端 monitor 必须可用；stub 结果不能替代 P4 执行。 3. 执行本 case 前冻结 timeout 预算；缺预算、模型、工具或服务时保持 BLOCKED，不以空执行关闭 case。
参数 1	timeout_budget_ref = campaign 配置中与 timeout_policy 对应的有限预算；不在本 case 猜测 SM 固定周期。
参数 2	required_capability = IMPLEMENTED_AND_ENABLED_TEXTURE_SURFACE_SERVICE
参数 3	request_mix = ["texture_sample","query","surface_memory_operation"]
参数 4	response_schedule = ["bounded_return_stall","legal_out_of_order_between_independent_requests","drain"]
参数 5	no_aliasing_assumption = independent initialized objects unless explicit service ordering recipe
初始化	1. TP-SOC-BASE 中真实服务已实现并显式启用；否则本 case 保持 BLOCKED，不转换成关闭门禁 PASS。 2. 请求 ID/owner/generation、对象元数据和 sampler oracle 已登记。

TC-TEX-004 项目	执行内容
执行步骤	1. 生成已审计合法的静态纹理及独立 surface 对象请求，先记录每请求参考值与允许精度/副作用。 2. 提交真实 SoC 服务，施加有限返回 stall 并交错独立对象请求，保持各自 payload/handle/owner 身份。 3. 按服务明确允许的边界/返回顺序监控 sample、query 和 surface 事务，禁止自行假定跨对象 coherence。 4. drain 全部已接受请求，核对真实返回值/副作用及对象回收条件；关闭 profile 对照只验证不会进入此正向执行。
观察点	1. 接受/返回/检查数量按真实事件对应，元数据和数据在 stall/乱序间不串对象。 2. 所有已接受服务责任最终结算，关闭 capability 从不执行本路径。
通过条件	1. 真实启用服务有非空端到端检查，数值/对象/边界/回收合同满足。 2. 实际执行量、检查量及所需覆盖非空，所有必需检查满足；本 case 当前 NOT_RUN 不因计划/静态检查而变为 PASS。
失败条件	1. 返回与错误对象/owner 匹配、stall 导致丢失/重复副作用，或未完成即回收对象。 2. 只有 stub/文本输出或关闭门禁证据却报告 P4 采样服务通过。
预期产物	1. manifest.json 2. case_result.json 3. coverage_delta.json 4. soc_object_manifest.json 5. sampler_service_trace.jsonl 6. surface_side_effects.json 7. soc_scoreboard.json

20.18 GEN-SYS-01：Stack/ABI、特殊寄存器与系统指令

TC-SYS-001：sreg 和元数据清单

TC-SYS-001 项目	执行内容
源与阶段	SIG-SYS-001；P1_GENERATOR_UNIT；CV-SYS-001
配置与 seed	TP-GEN-BASE；fixed_seed=0x534d004a（1397555274）；GEN_BOUNDED
回归与当前状态	SMOKE, DIRECTED, NEGATIVE；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION

TC-SYS-001 项目	执行内容
前置条件	1. 固定并核对源 vPlan/testpoint、PTX9.4 inventory、target/capability、规则及模型版本 hash；fixed_seed 由主 campaign 在运行前赋值。 2. 发生器、unit harness 和独立合法性/结构审计器必须可用；共享 source 身份不等于共享待测约束函数。 3. 执行本 case 前冻结 timeout 预算；缺预算、模型、工具或服务时保持 BLOCKED，不以空执行关闭 case。
参数 1	timeout_budget_ref = campaign 配置中与 timeout_policy 对应的有限预算；不在本 case 猜测 SM 固定周期。
参数 2	special_register_sections = 35
参数 3	directive_description_leaves = 34
参数 4	read_access = only source-legal mov/cvt forms, widths/components per target
参数 5	classification = ["launch_constant","reschedule_sensitive","driver_defined","undefined_or_target_specific"]
参数 6	negative_change = one attempted ordinary write to read-only special-register AST
参数 7	expected_layer = SOURCE_IR_LEGALIZER
初始化	1. 35 节 sreg 与 34 节 directive 源索引只读，目标/driver 状态提供方已声明。 2. 未展开 range 标签为猜测的物理 CSR 计数；元数据与执行指令账本分开。
执行步骤	1. 逐节读取 sreg 预定义声明和 target notes，生成各自合法 mov/cvt 读取的 typed IR 与状态提供方绑定。 2. 遍历 version/target/entry/cluster/ABI/debug 等 directive，验证只进入编译/launch 元数据，不占执行 opcode。 3. 按稳定性分类配置 sreg 参考策略；对 pm/target-specific 读禁止固定通用 golden。 4. 从一合法读 AST 仅改为普通写 sreg 的隔离副本，检查 source/IR legalizer 拒绝，合法读集合仍可用。
观察点	1. 35 与 34 说明身份/名称/来源一致，sreg 通过合法访问读取且 directive 不变成执行操作。 2. 只读写入变异被拒，类型/分量/target 无静默放宽。
通过条件	1. 全部说明节有真实生成/审计记录且分类正确，普通 sreg 写被 legalizer 拦截。 2. 实际执行量、检查量及所需覆盖非空，所有必需检查满足；本 case 当前 NOT_RUN 不因计划/静态检查而变为 PASS。
失败条件	1. 把全部 sreg 缓存为 CTA 常量、猜 PM 固定值或把 directive 计入 opcode。 2. 产生普通写 special register 而无诊断。

TC-SYS-001 项目	执行内容
预期产物	1. manifest.json 2. case_result.json 3. coverage_delta.json 4. sreg_access_matrix.json 5. directive_metadata_index.json 6. sreg_stability_classes.json 7. readonly_negative.json

TC-SYS-002：栈与配额约束

TC-SYS-002 项目	执行内容
源与阶段	SIG-SYS-002；P1_GENERATOR_UNIT；CV-SYS-002
配置与 seed	TP-GEN-BASE；fixed_seed=0x534d004b（1397555275）；GEN_BOUNDED
回归与当前状态	DIRECTED, NEGATIVE；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. 固定并核对源 vPlan/testpoint、PTX9.4 inventory、target/capability、规则及模型版本 hash；fixed_seed 由主 campaign 在运行前赋值。 2. 发生器、unit harness 和独立合法性/结构审计器必须可用；共享 source 身份不等于共享待测约束函数。 3. 执行本 case 前冻结 timeout 预算；缺预算、模型、工具或服务时保持 BLOCKED，不以空执行关闭 case。
参数 1	timeout_budget_ref = campaign 配置中与 timeout_policy 对应的有限预算；不在本 case 猜测 SM 固定周期。
参数 2	thread_contexts_min = 2
参数 3	stack_actions = ["stacksave","alloca","nested_call_return","stackrestore"]
参数 4	quota_actions = ["setmaxnreg.dec","setmaxnreg.inc"]
参数 5	quota_values = select legal aligned values from exact target recipe, not arbitrary counts
参数 6	resource_supply = reserve legal future dec/release path before accepting blocking inc
参数 7	negative_change = one use of an object invalidated by stackrestore; frontend-only
初始化	1. 每 thread 独立 stack/frame、live register 及 CC/predicate 状态；CTA RF 池预算来自已声明 profile。 2. 未分配/新获得寄存器均不可当已定义源读取，warpgroup 参与规则按 selected target。

TC-SYS-002 项目	执行内容
执行步骤	1. 为两个 thread 构造不同 stack 对象和分歧 call/return 路径，记录各自 saved SP 及对象 generation。 2. 执行 symbolic alloca/调用/restore，独立检查对齐、frame 生命周期和返回关系；restore 后旧对象失效。 3. 按合法 recipe 安排 dec 释放及 inc 请求，预留可执行供给者和同步，确认 inc 不能在无供给时无限等待；新增寄存器先定义后使用。 4. 仅在隔离副本把一次后续引用改为失效 stack 对象，要求生命周期 legalizer 拒绝；不执行该 UB 副本。
观察点	1. stack 不是 warp 共用一份返回/对象状态，restore 不影响其他 thread 或保留过期对象。 2. RF quota 按规定请求/释放/参与合同处理，不因指令描述为 hint 就忽略架构约束；有界供给可进展。
通过条件	1. 合法 stack/ABI 和 quota 序列通过独立状态模型，过期引用变异被阻止。 2. 实际执行量、检查量及所需覆盖非空，所有必需检查满足；本 case 当前 NOT_RUN 不因计划/静态检查而变为 PASS。
失败条件	1. 失效对象仍可用、线程返回状态串扰，或读未初始化新寄存器。 2. inc 生成无可执行供给/收尾路径，或把 quota 变化当无语义 no-op。
预期产物	1. manifest.json 2. case_result.json 3. coverage_delta.json 4. per_thread_stack_ledger.json 5. call_return_trace.jsonl 6. rf_quota_schedule.json 7. expired_object_reject.json

TC-SYS-003：驱动和不确定值

TC-SYS-003 项目	执行内容
源与阶段	SIG-SYS-003；P2_REFERENCE_PROGRAM；CV-SYS-001
配置与 seed	TP-REF-BASE；fixed_seed=0x534d004c（1397555276）；REFERENCE_BOUNDED
回归与当前状态	DIRECTED；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION

TC-SYS-003 项目	执行内容
前置条件	1. 固定并核对源 vPlan/testpoint、PTX9.4 inventory、target/capability、规则及模型版本 hash；fixed_seed 由主 campaign 在运行前赋值。 2. 所用独立参考解释器/允许结果模型及 lowering 或服务 reference route 必须可用；P2 结果不计真实 SM 执行。 3. 执行本 case 前冻结 timeout 预算；缺预算、模型、工具或服务时保持 BLOCKED，不以空执行关闭 case。
参数 1	timeout_budget_ref = campaign 配置中与 timeout_policy 对应的有限预算；不在本 case 猜测 SM 固定周期。
参数 2	driver_profiles = ["perCTA allocation unsupported: size=0","supported: legal size multiple1024 and >=minperctamemory"]
参数 3	sreg_relations = ["smid/warpid may change after declared reschedule","nsmid is ID bound, not physical count","pm values undefined","globaltimer target-specific"]
参数 4	nanosleep = source/target allowed timing relation only; no fence assumption
初始化	1. driver/launch 模型明确.minperctamemory 及 slice 地址/pitch，shared/local/TMEM 对象独立。 2. 调度模型可在合法点更新 smid/warpid，工具值没有默认黄金常量。
执行步骤	1. 分别加载 driver 不支持与支持 per-CTA global 分配的环境 snapshot；仅在有.minperctamemory 的 entry 读取相关 sreg。 2. 对支持环境检查 size 为合法 1024B 倍数和 slice 地址来源；对不支持环境记录 size=0，不将地址译到 SMEM。 3. 在声明的重调度点前后读取 smid/warpid，按 ID 范围/上下文关系检查而非要求值不变；PM 读只检合法访问与允许值类别。 4. 对 nanosleep/clock/globaltimer reference 事件应用已注册 target/环境关系，拒绝添加精确唤醒时刻、跨设备统一时钟或隐含 fence 断言。
观察点	1. perCTA global 由 driver slice 模型解释，size/driver 关系正确，不与其他地址空间混用。 2. 可变/未规定值使用关系或允许集合判据；没有把它们误分类成 UB。
通过条件	1. 所有已定义关系成立，未规定/target-specific 值不被固定 bit 比较误判。 2. 实际执行量、检查量及所需覆盖非空，所有必需检查满足；本 case 当前 NOT_RUN 不因计划/静态检查而变为 PASS。
失败条件	1. nsmid 当实际 SM 计数、smid/warpid 强制不变，或 PM/globaltimer 被伪造通用精确 oracle。 2. 无.minperctamemory 时仍当合法普通读取，或 size=0 被当成功分配。

TC-SYS-003 项目	执行内容
预期产物	1. manifest.json 2. case_result.json 3. coverage_delta.json 4. driver_environment_snapshots.json 5. special_value_classification.json 6. sreg_relation_checks.json 7. timing_relation.json

TC-SYS-004：系统事件与调度

TC-SYS-004 项目	执行内容
源与阶段	SIG-SYS-004；P3_SM_RTL；CV-SYS-003
配置与 seed	TP-SM-BASE；fixed_seed=0x534d004d（1397555277）；SM_PROGRESS
回归与当前状态	DIRECTED, STRESS；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. 固定并核对源 vPlan/testpoint、PTX9.4 inventory、target/capability、规则及模型版本 hash；fixed_seed 由主 campaign 在运行前赋值。 2. 项目接口版本、SM RTL/driver/monitor 及本 case 启用服务必须可用；native 编码未冻结时不得发送伪二进制。 3. 执行本 case 前冻结 timeout 预算；缺预算、模型、工具或服务时保持 BLOCKED，不以空执行关闭 case。
参数 1	timeout_budget_ref = campaign 配置中与 timeout_policy 对应的有限预算；不在本 case 猜测 SM 固定周期。
参数 2	system_trials = ["debug_breakpoint","trap_expected_termination","nanosleep_with_legal_context","runtime_rf_quota"]
参数 3	trial_isolation = separate reset/context per terminating event
参数 4	expected_terminal_class = from frozen project debug/trap service contract
参数 5	no_exact_latency = only configured progress/fairness and source-allowed timing relations
初始化	1. 真实 debug/trap/RF 服务和 monitor 可用且明确哪些事件终止 kernel/context。 2. 每 trial 有独立初始上下文，预期终止与普通算术错误分不同结果类。

TC-SYS-004 项目	执行内容
执行步骤	1. 先运行无终止的合法系统 baseline 并采样 driver/sreg 关系，确认观测路径可用。 2. 在隔离上下文发起 brkpt 或 trap，记录真实接受、调试/终止事件及责任清理；不继续等待正常算术 writeback。 3. 另起上下文运行合法 nanosleep 和 RF quota 序列，施加有界压力但不假定精确延迟或 fence。 4. 按事件分类 monitor 和 ABI/relation checker 检查各 trial 终态，再执行规定 reset/恢复并验证无旧 generation 事件污染。
观察点	1. 预期 trap/debug 不被错误记为普通算术 FAIL 或丢指令，事件类与服务合同匹配。 2. 非终止 trial 的 ABI/配额/driver 关系正确，恢复后无旧响应污染。
通过条件	1. 真实系统事件及恢复证据非空，终止、调试和正常完成分类符合合同。 2. 实际执行量、检查量及所需覆盖非空，所有必需检查满足；本 case 当前 NOT_RUN 不因计划/静态检查而变为 PASS。
失败条件	1. trap 被吞成无副作用 branch，或预期终止被算正常 completed。 2. 借精确 nanosleep 周期或 generic fence 假设判定通过，或恢复遗留旧责任。
预期产物	1. manifest.json 2. case_result.json 3. coverage_delta.json 4. system_event_trace.jsonl 5. debug_trap_contract.json 6. abi_relation_checks.json 7. post_reset_generation_checks.json

20.19 GEN-NEG-01：负例、UB 与错误分类隔离

TC-NEG-001：单因素变异

TC-NEG-001 项目	执行内容
源与阶段	SIG-NEG-001；P1_GENERATOR_UNIT；CV-NEG-001
配置与 seed	TP-GEN-NEGATIVE；fixed_seed=0x534d004e（1397555278）；GEN_BOUNDED
回归与当前状态	SMOKE, DIRECTED, NEGATIVE；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION

TC-NEG-001 项目	执行内容
前置条件	1. 固定并核对源 vPlan/testpoint、PTX9.4 inventory、target/capability、规则及模型版本 hash；fixed_seed 由主 campaign 在运行前赋值。 2. 发生器、unit harness 和独立合法性/结构审计器必须可用；共享 source 身份不等于共享待测约束函数。 3. 执行本 case 前冻结 timeout 预算；缺预算、模型、工具或服务时保持 BLOCKED，不以空执行关闭 case。
参数 1	timeout_budget_ref = campaign 配置中与 timeout_policy 对应的有限预算；不在本 case 猜测 SM 固定周期。
参数 2	baseline_required = independent positive verdict with immutable source/target/IR hashes
参数 3	mutation_cardinality = 1
参数 4	mutation_fields = ["one tuple field","one participant relation","one lifetime reference"]
参数 5	expected_layer_rule = declared before mutant submission
参数 6	silent_repair = forbidden
初始化	1. baseline 已独立合法化并保留正向可接受记录；mutator 不修改原 baseline。 2. 每 trial 指定单一 field/relation、预期 rule 及拒绝层，其他约束保持合法。
执行步骤	1. 选择一个已验证 baseline 并保存完整结构与 source/target 证据。 2. 按预登记 mutation 只改变一个字段/关系，例如把一次引用改为已失效 owner，而不同时改 type/mask/target。 3. 在执行前用独立差分确认单因素，并按规则预测拒绝层；若额外约束被连带破坏则丢弃该 trial 并记录原因。 4. 向指定前端/合法化层提交 mutant，核对规则与层；重新提交原 baseline 确认仍可接受，不允许静默修复后宣称负例命中。
观察点	1. baseline 保持合法，mutant 差分与预登记规则一一对应。 2. 实际拒绝层/原因匹配，修复/重采样有显式新身份而不冒充原负例。
通过条件	1. 单因素负例有明确拒绝证据且对应 baseline 实际通过合法性检查。 2. 实际执行量、检查量及所需覆盖非空，所有必需检查满足；本 case 当前 NOT_RUN 不因计划/静态检查而变为 PASS。
失败条件	1. 多个变化混入或 baseline 本来非法，拒绝原因被其他约束遮蔽。 2. 自动修复、替换 seed 或全拒绝器被报预期命中。

TC-NEG-001 项目	执行内容
预期产物	1. manifest.json 2. case_result.json 3. coverage_delta.json 4. baseline_positive_verdict.json 5. mutation_manifest.json 6. single_factor_diff.json 7. rejection_layer_check.json

TC-NEG-002：UB 隔离

TC-NEG-002 项目	执行内容
源与阶段	SIG-NEG-002；P1_GENERATOR_UNIT；CV-NEG-002
配置与 seed	TP-GEN-NEGATIVE；fixed_seed=0x534d004f（1397555279）；GEN_BOUNDED
回归与当前状态	DIRECTED, NEGATIVE；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. 固定并核对源 vPlan/testpoint、PTX9.4 inventory、target/capability、规则及模型版本 hash；fixed_seed 由主 campaign 在运行前赋值。 2. 发生器、unit harness 和独立合法性/结构审计器必须可用；共享 source 身份不等于共享待测约束函数。 3. 执行本 case 前冻结 timeout 预算；缺预算、模型、工具或服务时保持 BLOCKED，不以空执行关闭 case。
参数 1	timeout_budget_ref = campaign 配置中与 timeout_policy 对应的有限预算；不在本 case 猜测 SM 固定周期。
参数 2	classes = ["defined","implementation_specific_or_underspecified","source_undefined_behavior","unresolved_source_rule"]
参数 3	ub_examples = ["out_of_range where source explicitly UB","inconsistent collective participation where source UB","use_after_lifetime where source UB"]
参数 4	ub_sink = quarantine only; no normal backend issue
参数 5	non_ub_example = spdecompress index>num_target is implementation-specific, not automatically UB
初始化	1. 正常 functional stream 与 quarantine 出口物理/逻辑分开，coverage 分母账本为空。 2. 每候选的 source classification 与证据已登记；未知规则不默认 UB 或合法。

TC-NEG-002 项目	执行内容
执行步骤	1. 从合法 baseline 分别单改一个条件构造 source 明确 UB 候选，并记录该条规范证据。 2. 另构造 implementation-specific/允许集合候选和未解决规则候选，用独立分类器确认它们不被合并成 UB。 3. 将 UB 送 quarantine 并尝试普通 functional 发布，要求路由拒入且无 backend/DUT 事件；项目另有明确 robustness 合同的试验使用不同 ID/stream。 4. 核对 coverage registry 只按规则登记可达合法项，不把 UB 或缺规则自动当通过/已覆盖；保留隔离和诊断 artifact。
观察点	1. source UB 与 implementation-specific/允许集合、unresolved 分开，分类具有原文出处。 2. UB 在普通 stream 的 issue/completed/checked 均零，无预期硬件 trap 假设。
通过条件	1. UB 隔离生效，普通功能分母/执行统计不被负例污染，非 UB 边界不会被粗暴归类。 2. 实际执行量、检查量及所需覆盖非空，所有必需检查满足；本 case 当前 NOT_RUN 不因计划/静态检查而变为 PASS。
失败条件	1. 执行 UB 并把崩溃/任意值当 PASS，或让 quarantine 流混入普通覆盖。 2. 把 implementation-specific 值强制为唯一 golden，或把 unknown rule 自动 waive。
预期产物	1. manifest.json 2. case_result.json 3. coverage_delta.json 4. behavior_taxonomy.json 5. quarantine_manifest.json 6. functional_stream_audit.json 7. coverage_denominator_diff.json

TC-NEG-003：错误层 relation

TC-NEG-003 项目	执行内容
源与阶段	SIG-NEG-003；P2_REFERENCE_PROGRAM；CV-NEG-003
配置与 seed	TP-REF-BASE；fixed_seed=0x534d0050（1397555280）；REFERENCE_BOUNDED
回归与当前状态	DIRECTED, NEGATIVE；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION

TC-NEG-003 项目	执行内容
前置条件	1. 固定并核对源 vPlan/testpoint、PTX9.4 inventory、target/capability、规则及模型版本 hash；fixed_seed 由主 campaign 在运行前赋值。 2. 所用独立参考解释器/允许结果模型及 lowering 或服务 reference route 必须可用；P2 结果不计真实 SM 执行。 3. 执行本 case 前冻结 timeout 预算；缺预算、模型、工具或服务时保持 BLOCKED，不以空执行关闭 case。
参数 1	timeout_budget_ref = campaign 配置中与 timeout_policy 对应的有限预算；不在本 case 猜测 SM 固定周期。
参数 2	result_layers = ["PRE_FLIGHT_REJECT","COMPILE_OR_TOOL_BLOCKED","BACKEND_UNSUPPORTED","PROJECT_PROTOCOL_ERROR"]
参数 3	expected_rule_layer = declared separately per trial
参数 4	missing_dependency_policy = non-injected missing tool/service makes affected trial and aggregate required case BLOCKED, not PASS
参数 5	taxonomy_fixture_policy = explicitly SIMULATED taxonomy records may test classifier only; they do not count compile/SM execution
初始化	1. 独立错误 taxonomy 和 reference/harness 结果接口已实现，真实观测与 SIMULATED 分类 fixture 分开。 2. 至少一个实际运行的参考/前端 trial 可用，不能以全缺工具关闭本 case。
执行步骤	1. 为每类结果声明触发层、错误 rule、证据来源和所需依赖，准备合法 baseline 及各自单因素 trial。 2. 运行可用的 preflight/reference/backend route，采集原始结果、工具版本、事件与退出分类；缺依赖立即记录 BLOCKED 而非预期失败 PASS。 3. 对项目协议类只使用已声明 reference 模型或真实合同事件并记录 provenance；若使用 taxonomy fixture 则明确 SIMULATED 且排除执行计数。 4. 由独立 classifier 比对实际层/rule 与预期，检查非零 exit 不是统一通过条件，汇总时保留必需 trial 的 BLOCKED 状态。
观察点	1. 拒绝、unsupported、缺工具和协议故障保留不同层及证据，实际执行与模拟分类分开。 2. 任一必需依赖缺失不会因错误码非零就关闭该 trial 或整个 case。
通过条件	1. 实际可运行 trial 的分类与预期匹配，所有必需依赖满足后才能关闭 case；fixture 单独不构成执行证据。 2. 实际执行量、检查量及所需覆盖非空，所有必需检查满足；本 case 当前 NOT_RUN 不因计划/静态检查而变为 PASS。

TC-NEG-003 项目	执行内容
失败条件	1. 所有非零 exit 被当负例 PASS，或缺工具被误记编译/执行成功。 2. reference 协议结果冒充真实 SM 事件，或模拟 taxonomy 覆盖替代实际 case 证据。
预期产物	1. manifest.json 2. case_result.json 3. coverage_delta.json 4. raw_outcomes.json 5. error_layer_relation.json 6. dependency_gate_report.json 7. provenance_manifest.json

TC-NEG-004：项目 robustness 注入

TC-NEG-004 项目	执行内容
源与阶段	SIG-NEG-004；P3_SM_RTL；CV-NEG-003
配置与 seed	TP-SM-BASE；fixed_seed=0x534d0051（1397555281）；SM_PROGRESS
回归与当前状态	DIRECTED, NEGATIVE, STRESS；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. 固定并核对源 vPlan/testpoint、PTX9.4 inventory、target/capability、规则及模型版本 hash；fixed_seed 由主 campaign 在运行前赋值。 2. 项目接口版本、SM RTL/driver/monitor 及本 case 启用服务必须可用；native 编码未冻结时不得发送伪二进制。 3. 执行本 case 前冻结 timeout 预算；缺预算、模型、工具或服务时保持 BLOCKED，不以空执行关闭 case。
参数 1	timeout_budget_ref = campaign 配置中与 timeout_policy 对应的有限预算；不在本 case 猜测 SM 固定周期。
参数 2	classification = NONISA_ROBUSTNESS
参数 3	selected_fault = one documented project adapter illegal-command token via fault API
参数 4	encoding = symbolic fault ID mapped only by frozen backend contract; no invented reserved bits
参数 5	fault_cardinality = 1
参数 6	optional_protocol_trial = valid-held mutation only if monitor responsibility explicitly defined; no assumed SM recovery
参数 7	not_in_scope = PTX UB behavioral consistency

TC-NEG-004 项目	执行内容
初始化	1. 项目 fault 注入点、expected detector/ack/cleanup 已冻结；没有合同的 fault 类保持 BLOCKED。 2. 真实 adapter/monitor 可用，合法 baseline 路径已确认；fault 注入 stream 与普通随机正向分开。
执行步骤	1. 执行合法 baseline 并记录接收/完成/资源闭合，建立本 fault trial 的非故障对照。 2. 通过已批准项目 fault API 只改变一次命令 token 或合同定义错误字段，其他 payload/握手/target 保持合法。 3. 观察规定检测层、错误 ack、禁止的副作用和资源 cleanup；若选 valid 协议违例，严格按合同只检查指定 monitor 而不额外要求 SM 恢复。 4. 执行合同规定 drain/abort/reset 并检查旧 generation 无后写，使用 NONISA_ROBUSTNESS 结果类归档，不计 PTX UB 兼容性。
观察点	1. 注入恰一次且属于书面项目合同，检测/ack/责任归属明确。 2. 错误处置和清理满足合同，无越权写/重复结算；结果未混入普通 PTX 数值覆盖。
通过条件	1. 真实 fault 检测及规定恢复/终止证据完整，并与合法 baseline 对照一致。 2. 实际执行量、检查量及所需覆盖非空，所有必需检查满足；本 case 当前 NOT_RUN 不因计划/静态检查而变为 PASS。
失败条件	1. 无合同就要求硬件 trap/恢复，或把项目协议试验称 PTX UB 规范通过。 2. fault 导致未授权写入、双 ack、旧 generation 污染或资源泄漏。
预期产物	1. manifest.json 2. case_result.json 3. coverage_delta.json 4. project_fault_contract.json 5. fault_injection_trace.jsonl 6. error_cleanup_scoreboard.json 7. nonisa_classification.json

20.20 GEN-ENV-01：Launch 环境、服务延迟与 fault 控制

TC-ENV-001：launch 组合边界

TC-ENV-001 项目	执行内容
源与阶段	SIG-ENV-001；P1_GENERATOR_UNIT；CV-ENV-001
配置与 seed	TP-GEN-BASE；fixed_seed=0x534d0052（1397555282）；GEN_BOUNDED

TC-ENV-001 项目	执行内容
回归与当前状态	SMOKE, DIRECTED, NEGATIVE, RANDOM；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. Generator/independent checker not implemented; reference or unit harness required. 2. 锁定本用例的 vPlan/source/capability/schema 及独立 checker 版本；所需具体 target/recipe 未闭合则 BLOCKED。 3. 以下数值是项目测试预算或 fixture，不是 NVIDIA 物理参数或本轮实际执行记录。
参数 1	threads_per_CTA = [1,31,32,33]
参数 2	CTA_count = [1,8,9]
参数 3	S0_SMEM_pool_bytes = 262144
参数 4	shared_budget_witnesses = [{"CTA_count":8,"bytes_per_CTA":32768,"reserved_bytes_per_CTA":0,"expected":"pool_exactly_full_if_target_and_RF_allow"}, {"CTA_count":8,"bytes_per_CTA":65536,"expected":"reject_shared_overcommit"}, {"CTA_count":9,"threads_each":32,"expected":"reject_CTA_slots"}]
参数 5	RF_budget_edges = ["R_SM-resolved_allocation_unit","R_SM","R_SM+resolved_allocation_unit"]
参数 6	resource_constants_status = S0 project configuration; target/ABI, R_SM and allocation units must be independently resolved
初始化	1. 初始化独立 launch 预算表与各 CTA 私有 owner，total reservation 为 0。 2. 不把尾 warp 的 padding lane 加入 live mask；每 CTA 静态/动态/reserved 对象分别计数。
执行步骤	1. 枚举 thread 边界和 CTA/shared witness，以 ceil(threads/32)计算 warp reservation 并声明目标单 CTA limits。 2. 使用独立预算模型依次判断 CTA/warp/RF/SMEM 以及 target/ABI；未知 R_SM 或 granularity 保持 unresolved。 3. 让发生器构造相同 launch 包，比较 admit/reject 分类和每项 reservation，不启动任何 SM。 4. 撤回被拒候选，检查账本恢复；把正常/拒绝记录及 gate 状态写入 seed manifest。
观察点	1. 8×32KiB 仅在其余资源与 target 也允许时可 admit；不能只因等于 pool 就强制通过。 2. 9CTA 和 8×64KiB witness 分别因槽/容量拒绝，拒绝不得分配部分 owner 或计 DUT 事件。
通过条件	1. 所有已解析组合与独立预算判定一致，padding lane 无副作用，拒绝后资源守恒。

TC-ENV-001 项目	执行内容
失败条件	1. 将 SMEM 池容量当单 CTA 保证；R_SM 未配置仍声称成功；部分 launch 占基本资源后等待。
预期产物	1. launch_matrix.json 2. joint_budget_audit.json 3. candidate_rollback_log.json 4. manifest.json

TC-ENV-002：服务 schedule 记录

TC-ENV-002 项目	执行内容
源与阶段	SIG-ENV-002；P1_GENERATOR_UNIT；CV-ENV-002
配置与 seed	TP-GEN-BASE；fixed_seed=0x534d0053（1397555283）；GEN_BOUNDED
回归与当前状态	DIRECTED, RANDOM, REPLAY；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. Generator/independent checker not implemented; reference or unit harness required. 2. 锁定本用例的 vPlan/source/capability/schema 及独立 checker 版本；所需具体 target/recipe 未闭合则 BLOCKED。 3. 以下数值是项目测试预算或 fixture，不是 NVIDIA 物理参数或本轮实际执行记录。
参数 1	event_fixture_count = 16
参数 2	RNG_streams = ["opcode","operand","CFG","resource","timing","fault"]
参数 3	schedule_variants = ["READY_ALWAYS","STALL_ON_ODD_ACCEPT_OPPORTUNITIES","PERMITTED_INDEPENDENT_RETURNS_REVERSED","DELAYED_OLD_EPOCH"]
参数 4	schedule_unit = logical events; not fixed hardware cycles
参数 5	record_all_sampler_retries = true
初始化	1. unit-harness 创建 16 个明确类型和 owner/generation 的 SYNTHETIC 事务；异步依赖图来自审计 fixture。 2. 每条随机子流由 run seed 和 stream name 确定，环境调度完全物化为 JSON。

TC-ENV-002 项目	执行内容
执行步骤	1. 按同一 manifest 生成两份环境 schedule，再按不同 worker 顺序重新生成。 2. 独立 schedule validator 检查 return 仅对应已发起事务，反序只作用于允许并发的独立返回。 3. 对晚到旧 epoch 和 backpressure 场景验证预期过滤/阻塞事件，不模拟未知 DUT 成功响应。 4. 比较各流 hash 及完整 schedule；改一个 timing 子流时检查未声明受影响的 operand 等子流不变。
观察点	1. schedule 记录每个 logical_event_id、source transaction、ready/return/fault 及 fairness contract。 2. 同 case 同 manifest 不受 worker 遍历顺序影响；仅输入子流变化的影响范围符合合同。
通过条件	1. schedule 字节/语义按确定性合同可重现且所有事件可追踪；不增加 DUT 运行计数。
失败条件	1. 用 wall clock 播种或依赖全局 worker RNG；凭空返回成功；重排违反依赖/接口合同。
预期产物	1. environment_schedule.json 2. stream_digests.json 3. schedule_audit.json 4. manifest.json

TC-ENV-003：公平与 deadlock 模板

TC-ENV-003 项目	执行内容
源与阶段	SIG-ENV-003；P2_REFERENCE_PROGRAM；CV-ENV-001
配置与 seed	TP-REF-BASE；fixed_seed=0x534d0054（1397555284）；REFERENCE_BOUNDED
回归与当前状态	DIRECTED, STRESS；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. Generator/independent checker not implemented; reference or unit harness required. 2. 锁定本用例的 vPlan/source/capability/schema 及独立 checker 版本；所需具体 target/recipe 未闭合则 BLOCKED。 3. 以下数值是项目测试预算或 fixture，不是 NVIDIA 物理参数或本轮实际执行记录。
参数 1	reference_fixture_kind = SYNTHETIC_REFERENCE
参数 2	wait_graph_scenarios = ["FAIR_CHAIN_DRAINS","EXTERNAL_SERVICE_WITHHOLDS_RETURN","INJECTED_INTERNAL_CREDIT_CYCLE"]
参数 3	logical_progress_watch_window_events = 64

TC-ENV-003 项目	执行内容
参数 4	service_contract = explicit eventual-service assumption; no universal per-instruction cycle limit
初始化	1. 为三种 fixture 分别记录相同有效程序的资源图和可见事件；内部 cycle 为 reference fault injection，不是合法程序主动制造 ISA UB。 2. 独立 reference scheduler 初始化所有 owner 与待服务节点，关闭真实 DUT coverage sink。
执行步骤	1. 在公平服务 fixture 推进事件直到全部义务关闭，记录最小进度链。 2. 在外部服务不回应 fixture 保留未履行 assumption，不把停滞自动标 DUT 死锁。 3. 在注入内部 credit 环的 reference 模型中导出 wait-for strongly-connected components 与责任归属。 4. 由独立 diagnostic checker 比较三种分类；timeout 只终止实验，不能代替根因证据。
观察点	1. FAIR 与 STARVATION 的区别来自服务 assumption 被满足/违反，不靠同一固定周期阈值。 2. 内部 fault 产生的 cycle 有可重放 wait graph；负例不进入正常合法 DUT 程序。
通过条件	1. 三 fixture 被分别归类为完成、外部阻塞、模型内部环等待，并保留完整事件/假设证据。
失败条件	1. 全部非完成都标同一 deadlock；无限等待；合成 reference 日志填入真实 SM coverage。
预期产物	1. wait_graph.json 2. assumption_status.json 3. progress_classification.json 4. reference_events.jsonl 5. manifest.json

TC-ENV-004：reset 与迟到返回

TC-ENV-004 项目	执行内容
源与阶段	SIG-ENV-004；P3_SM_RTL；CV-ENV-003
配置与 seed	TP-SM-BASE；fixed_seed=0x534d0055（1397555285）；SM_PROGRESS
回归与当前状态	DIRECTED, NEGATIVE, STRESS；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. Generator/independent checker not implemented; SM RTL and corresponding service/adapter required. 2. 锁定本用例的 vPlan/source/capability/schema 及独立 checker 版本；所需具体 target/recipe 未闭合则 BLOCKED。 3. 以下数值是项目测试预算或 fixture，不是 NVIDIA 物理参数或本轮实际执行记录。
参数 1	actions = ["SAFE_DRAIN","PROJECT_ABORT","RESET_EPOCH"]

TC-ENV-004 项目	执行内容
参数 2	action_cutpoints = ["AFTER_ADMIT","AFTER_SOURCE_CAPTURE","BEFORE_RESPONSE","AFTER_EXTERNAL_STORE_COMMIT"]
参数 3	old_new_generations = [10,11]
参数 4	late_response_repetitions = [1,2]
参数 5	reset_semantics = requires explicit project/SOC cancellation contract; no arbitrary-cycle preemption promise
初始化	1. 初始化旧 epoch/owner 与新 epoch 可复用资源，创建带明确合法源/目的的一组在途事务。 2. 独立 ledger 记录哪些外部 store 已 commit、哪些仍可取消，以及返回引用。
执行步骤	1. 以真实 adapter 提交合法程序并等 monitor 确认指定 cutpoint 后发出选定 project action。 2. SAFE_DRAIN 停止新 body 但保留 commit/wait/load/free 等必要收尾；ABORT/RESET 按已冻结取消/隔离协议执行。 3. 在新 epoch 建立后送达旧 generation 迟到返回/重发 fixture，监测是否写入新 owner。 4. 等待所有旧责任 drain 或取得正式 cancel ack；核对已 commit 外部副作用保持真实，再结束新 context。
观察点	1. 旧包过滤与外部副作用撤销是不同事实，必须独立记录 commit/cancel。 2. 新 owner 字节、RF 目的和 barrier phase 不被旧 response 污染；安全 drain 路径仍有进度。
通过条件	1. 所有 project 动作按其合同结束，旧责任无孤儿，新 context 无污染，外部已 commit 状态正确。
失败条件	1. 无取消确认就回收 owner；过滤旧包即声称撤销 store；一刀切阻断必要 cleanup。
预期产物	1. epoch_owner_events.jsonl 2. external_side_effect_ledger.json 3. cancel_ack_log.json 4. scoreboard.json 5. manifest.json

20.21 GEN-LIFE-01：有状态资源、cleanup 与活性

TC-LIFE-001：符号资源程序合成

TC-LIFE-001 项目	执行内容
源与阶段	SIG-LIFE-001；P1_GENERATOR_UNIT；CV-LIFE-001
配置与 seed	TP-GEN-BASE；fixed_seed=0x534d0056（1397555286）；GEN_BOUNDED
回归与当前状态	SMOKE, DIRECTED, RANDOM, REPLAY；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. 固定 PTX9.4 清单及 SM-ARCH-S0 来源 hash 通过 preflight；独立形式/资源审计器与本阶段 harness 已实现。 2. 由 manifest 绑定精确 target、capability 及合法 form allowlist；未解析的 target/形式保持 UNRESOLVED 并阻止对应正向分支，不推定数字更大的 target 继承能力。
参数 1	target_binding = MANIFEST_EXACT_TARGET_AND_FORM_ALLOWLIST_REQUIRED
参数 2	unresolved_capability_policy = BLOCK_POSITIVE_BRANCH_AND_REPORT_UNRESOLVED
参数 3	semantic_IR_node_budget = 128
参数 4	initial_cleanup_reserve_nodes = 48
参数 5	body_node_limit_initial = 80
参数 6	max_loop_iterations = 3
参数 7	max_live_memory_objects = 4
参数 8	CFG_shapes = ["直线","diamond","有界 loop"]
参数 9	motifs = ["TMEM 生命周期","合法 barrier/async","独立对象通信"]
参数 10	budget_rule = 每次加入 recipe 重算实际 cleanup 需求；48 不是所有程序充分的固定保证
初始化	1. 空 planned 账本与独立审计器初始状态一致，各对象先定义而非直接伪造 live。 2. 已解析 recipe 声明 pre/post、参与者、资源/phase、允许 partial/full 及 cleanup cost。
执行步骤	1. 按 dependency DAG 和资源可行集合选择 recipe，预留创建/发布/等待/释放执行者。 2. 在 diamond/loop 分别维护 path predicate、定义集合及 loop-carried allocation/phase 不变式。 3. 独立重走全 CFG 与允许 completion 前缀，确认 use 由创建/发布支配、reuse/free 有 HB。 4. 达预算前停止 body，生成完整 cleanup 并重新序列化审计到合法终止。

TC-LIFE-001 项目	执行内容
观察点	1. planned/issued/observed 分账，不能靠生成到第 N 节点猜测完成。 2. CFG merge 只保留所有路径成立事实，单路径 allocation 不能 merge 后无条件释放。
通过条件	1. 非空程序所有路径资源闭合且无依赖环；实际 cleanup 成本不超预算。 2. 独立审计与 generator 不会调用同一约束函数自证。
失败条件	1. 全局清 shadow 掩盖泄漏、缺路径 cleanup，或 source 未初始化支配 use。 2. 全部候选 UNSAT/空程序仍报告 whole-program pass。
预期产物	1. manifest.json：锁定 2026-11-09 测试计划、源 hash、seed/子流、profile、实际 target/capability 及服务配置。 2. program.ir 与 data_objects.json：保留指令身份、CFG、typed 对象、初始化及 cleanup。 3. resource_audit.json 与 event_obligations.json：独立账本/HB/phase/单位审计。 4. case_result.json 与 coverage_delta.json：区分生成/审计/接收/完成/检查，本阶段真实执行量及 NOT_RUN/BLOCKED 原因。 5. CFG_and_obligation_DAG.json、independent_prefix_audit.json。

TC-LIFE-002：尾部预算与失败回滚

TC-LIFE-002 项目	执行内容
源与阶段	SIG-LIFE-002；P1_GENERATOR_UNIT；CV-LIFE-002
配置与 seed	TP-GEN-BASE；fixed_seed=0x534d0057（1397555287）；GEN_BOUNDED
回归与当前状态	DIRECTED, NEGATIVE, STRESS, REPLAY；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. 固定 PTX9.4 清单及 SM-ARCH-S0 来源 hash 通过 preflight；独立形式/资源审计器与本阶段 harness 已实现。 2. 由 manifest 绑定精确 target、capability 及合法 form allowlist；未解析的 target/形式保持 UNRESOLVED 并阻止对应正向分支，不推定数字更大的 target 继承能力。
参数 1	target_binding = MANIFEST_EXACT_TARGET_AND_FORM_ALLOWLIST_REQUIRED
参数 2	unresolved_capability_policy = BLOCK_POSITIVE_BRANCH_AND_REPORT_UNRESOLVED
参数 3	IR_node_budgets = [32,64,128]
参数 4	candidate_retry_limits = [0,1,4]

TC-LIFE-002 项目	执行内容
参数 5	stop_positions = ["cleanup 余量刚好","再加一个 recipe 即不足","分支 merge 前","loop 迭代边界"]
参数 6	injected_candidate_failures = ["合法化拒绝","资源 UNSAT","序列化候选失败"]
初始化	1. 已发布 prefix/manifest hash 固定，候选 reservation 尚未进入提交账本。 2. 每 recipe 有有界 cleanup 代价与所需执行者，不把 tail 预算作为事后截断长度。
执行步骤	1. 在不同预算和 stop 位置填 body，独立计算剩余 tail 成本。 2. 注入有限候选失败/UNSAT 并回滚仅未发布 reservation，核对已提交 trace 不变。 3. 预算将不足时停止扩展并产出完整尾部；零重试配置立即给定义诊断而非忙等。 4. 重新审计所有终止路径与程序 hash，保存拒绝/回滚与最终账本。
观察点	1. 无资源/随机子流/已发布 prefix 被非法清空或偷偷改写。 2. 实际输出包含必要 wait/phase/dealloc，不通过截断 IR 伪装预算满足。
通过条件	1. 每次成功输出均在预算内闭合；UNSAT/预算不足都有非 PASS 原因及有限终止。 2. 同 manifest 可重放生成/拒绝路径。
失败条件	1. 无限重试、候选回滚泄漏，或为了凑 budget 删关键 producer/wait。 2. fallback 静默改变目标/能力/相关性后仍冒充原 case。
预期产物	1. manifest.json：锁定 2026-11-09 测试计划、源 hash、seed/子流、profile、实际 target/capability 及服务配置。 2. program.ir 与 data_objects.json：保留指令身份、CFG、typed 对象、初始化及 cleanup。 3. resource_audit.json 与 event_obligations.json：独立账本/HB/phase/单位审计。 4. case_result.json 与 coverage_delta.json：区分生成/审计/接收/完成/检查，本阶段真实执行量及 NOT_RUN/BLOCKED 原因。 5. candidate_rollback_trace.jsonl、cleanup_budget_proof.json、prefix_hash_history.json。

TC-LIFE-003：多顺序 prefix 验证

TC-LIFE-003 项目	执行内容
源与阶段	SIG-LIFE-003；P2_REFERENCE_PROGRAM；CV-LIFE-001
配置与 seed	TP-REF-BASE；fixed_seed=0x534d0058（1397555288）；REFERENCE_BOUNDED
回归与当前状态	DIRECTED, RANDOM, STRESS, REPLAY；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION

TC-LIFE-003 项目	执行内容
前置条件	1. 固定 PTX9.4 清单及 SM-ARCH-S0 来源 hash 通过 preflight；独立形式/资源审计器与本阶段 harness 已实现。 2. 由 manifest 绑定精确 target、capability 及合法 form allowlist；未解析的 target/形式保持 UNRESOLVED 并阻止对应正向分支，不推定数字更大的 target 继承能力。 3. 独立语义/内存/事件模型及其允许结果集合已锁定；生成器 shadow 不能作为唯一 oracle。
参数 1	target_binding = MANIFEST_EXACT_TARGET_AND_FORM_ALLOWLIST_REQUIRED
参数 2	unresolved_capability_policy = BLOCK_POSITIVE_BRANCH_AND_REPORT_UNRESOLVED
参数 3	logical_operations = 3
参数 4	logical_milestones_each = ["partial","full"]
参数 5	logical_linearizations_upper_example = 90
参数 6	count_note = 三个独立 partial<full 对的 6 事件排列=90；实际 HB 剪枝后记录全部有界状态，非全系统穷尽
参数 7	families = ["TMA source-read/full","Fabric local-read/full","MMA A-read/full"]
参数 8	notification_delivery = 规范允许时独立乱序；不强制 partial 通知必须先于 full 观察
参数 9	finite_model_bounds = 3 操作、固定合法对象/phase、无无界 loop
初始化	1. 三种服务在同一合法项目/target 模型中确实可用，否则对应组合 blocked。 2. 各自 barrier/group/owner 及 typed 义务独立初始化，跨对象访问无未排序冲突。
执行步骤	1. 建立同一合法程序的固定 CFG/HB，列举有界逻辑 milestone 可达顺序。 2. 对 notification 交付再按允许规则探索，区分物理读取结束与用户观察到受限通知。 3. 每个可达 prefix 独立检查 use/reuse/free、phase/window 与 owner 引用，不借 generator shadow。 4. 到达各终止状态时检查义务清零；保存探索边界/状态数和反例路径。
观察点	1. logical partial/full 与可观察通知不被强制同序，所有 accepted 事件恰一次。 2. 只在匹配 wait/phase 观察后更新程序可用事实，不从模型内部早事件授予消费权。
通过条件	1. 所声明有限状态空间全部可达 prefix 被检查，无提前 free/错复用或孤儿义务。 2. 探索截断必须标 incomplete/blocked，不能将若干随机顺序称 all-schedules 证明。
失败条件	1. 只验证单一 completion 顺序即宣称全允许顺序安全，或忽略合法通知乱序。 2. 有界模型结束尚有引用/phase 责任，或把全系统活性当已证明。

TC-LIFE-003 项目	执行内容
预期产物	1. manifest.json：锁定 2026-11-09 测试计划、源 hash、seed/子流、profile、实际 target/capability 及服务配置。 2. program.ir 与 data_objects.json：保留指令身份、CFG、typed 对象、初始化及 cleanup。 3. resource_audit.json 与 event_obligations.json：独立账本/HB/phase/单位审计。 4. case_result.json 与 coverage_delta.json：区分生成/审计/接收/完成/检查，本阶段真实执行量及 NOT_RUN/BLOCKED 原因。 5. bounded_state_space_manifest.json、all_prefix_results.json、counterexample_trace.json。

TC-LIFE-004：真实满载 drain

TC-LIFE-004 项目	执行内容
源与阶段	SIG-LIFE-004；P3_SM_RTL；CV-LIFE-003
配置与 seed	TP-SM-STRESS；fixed_seed=0x534d0059（1397555289）；SM_PROGRESS
回归与当前状态	DIRECTED, STRESS, REPLAY；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. 固定 PTX9.4 清单及 SM-ARCH-S0 来源 hash 通过 preflight；独立形式/资源审计器与本阶段 harness 已实现。 2. 由 manifest 绑定精确 target、capability 及合法 form allowlist；未解析的 target/形式保持 UNRESOLVED 并阻止对应正向分支，不推定数字更大的 target 继承能力。 3. SM、对应 adapter 和 monitor 真实可用；全部事务的 owner/epoch/返回身份及服务公平合同已配置。
参数 1	target_binding = MANIFEST_EXACT_TARGET_AND_FORM_ALLOWLIST_REQUIRED
参数 2	unresolved_capability_policy = BLOCK_POSITIVE_BRANCH_AND_REPORT_UNRESOLVED
参数 3	pressure_levels = ["1","configured_capacity-1","configured_capacity","configured_capacity+1 候选受回压"]
参数 4	queue_capacity_source = 实际实现/锁定服务 profile；v1.1 数值仅显式压力配置，非 ISA 限制
参数 5	services = ["Tensor","SMEM return","barrier/control","RF writeback"]
参数 6	drain_modes = ["正常 soft-drain","明确 owner abort 另立子变体"]
参数 7	service_delay = 有限 schedule，watchdog 按工作量/服务公平预算，不猜固定指令周期

TC-LIFE-004 项目	执行内容
初始化	1. 真实 SM/adapter 有足够 monitor 暴露 typed 返回/owner，所有请求起始都来自已审计合法程序。 2. cleanup 指令/执行者与控制进度路径预留；环境保证最终解除有限反压。
执行步骤	1. 逐级施加已接受 Tensor/SMEM/RF 返回压力，超过可接收容量的候选保持未 admit 而非 ISA 非法。 2. 停止新 kernel/epoch 或新增 body 压力，但继续当前程序必要 tiles、load、commit/wait/free 及 barrier。 3. 解除有限回压，记录 wait-for 图和最老义务；正常模式等待全部终点，abort 模式取得正式取消/隔离确认。 4. 仅在所有 owner/外部引用与副作用处理闭合后回收，核对 exactly-once 及 generation 安全。
观察点	1. 没有一刀切阻断正常收尾所需指令；escape/control/返回可在公平合同下前进。 2. drop 旧 epoch 包不等于已撤销外部 store，正常 drain 与 abort 结果不混。
通过条件	1. 正常模式全部责任归还，或 abort 模式有正式取消/副作用记录且不宣称正常数值完成。 2. watchdog 报告可区分模型未兑现公平、合法长延迟与内部无进度。
失败条件	1. 满载后 commit/free 进不来形成环等，或正常 soft drain 停掉剩余 tiles。 2. 未确认外部事务就清 owner、丢返回后伪造 drain 成功，或零 accepted 事务假通过。
预期产物	1. manifest.json：锁定 2026-11-09 测试计划、源 hash、seed/子流、profile、实际 target/capability 及服务配置。 2. program.ir 与 data_objects.json：保留指令身份、CFG、typed 对象、初始化及 cleanup。 3. resource_audit.json 与 event_obligations.json：独立账本/HB/phase/单位审计。 4. case_result.json 与 coverage_delta.json：区分生成/审计/接收/完成/检查，本阶段真实执行量及 NOT_RUN/BLOCKED 原因。 5. sm_wait_for_and_drain_trace.jsonl、oldest_obligation_report.json、owner_cancel_side_effect_ledger.json。

20.22 GEN-COV-01：覆盖模型、采样点与分母治理

TC-COV-001：采样阶段分账

TC-COV-001 项目	执行内容
源与阶段	SIG-COV-001；P1_GENERATOR_UNIT；CV-COV-001
配置与 seed	TP-GEN-BASE；fixed_seed=0x534d005a（1397555290）；GEN_BOUNDED

TC-COV-001 项目	执行内容
回归与当前状态	SMOKE, DIRECTED, NEGATIVE；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. Generator/independent checker not implemented; reference or unit harness required. 2. 锁定本用例的 vPlan/source/capability/schema 及独立 checker 版本；所需具体 target/recipe 未闭合则 BLOCKED。 3. 以下数值是项目测试预算或 fixture，不是 NVIDIA 物理参数或本轮实际执行记录。
参数 1	log_origin = SYNTHETIC_UNIT
参数 2	logical_candidates = 4
参数 3	fixture_classes = ["REJECTED_PREFLIGHT","LEGALIZED_NOT_SENT","ADMITTED_NOT_COMPLETED","COMPLETED_NOT_CHECKED"]
参数 4	real_DUT_event_sink_enabled = false
参数 5	real_dynamic_runs_expected = 0
初始化	1. 创建隔离的 unit coverage 数据库，所有 fixture 加 SYNTHETIC 标志且分离 run 命名空间。 2. 参考计数表由独立人工可重算的四个 fixture 建立，不读取 aggregator 内部状态作 golden。
执行步骤	1. 依次送入四类 fixture 并在每个阶段独立导出 unit counters。 2. 核对 candidate/legalized/reject/admit/complete/checked 各计数的身份与前置事件。 3. 尝试把 SYNTHETIC 日志合并到真实 DUT_SCENARIO 数据库，预期合并边界拒绝。 4. 清空 fixture 并重放同一记录，核对 reset/去重不产生真实 run 或 DUT coverage 增量。
观察点	1. COMPLETED_NOT_CHECKED 不增加 CHECKED；preflight reject 不增加实际 admitted。 2. real_DUT_event_sink 始终 0，unit fixture 命中与真实执行记录严格隔离。
通过条件	1. 独立重算计数完全一致，全部 synthetic 标志与来源保留，误合并被具体规则拒绝。
失败条件	1. 仅程序文本存在就记执行；将 P1 日志当真实 DUT；无 checked 却报功能通过。
预期产物	1. synthetic_unit_events.jsonl 2. unit_counter_expected.json 3. coverage_aggregator_diff.json 4. manifest.json

TC-COV-002：分母和交叉 bin

TC-COV-002 项目	执行内容
源与阶段	SIG-COV-002；P1_GENERATOR_UNIT；CV-COV-002

TC-COV-002 项目	执行内容
配置与 seed	TP-GEN-BASE；fixed_seed=0x534d005b（1397555291）；GEN_BOUNDED
回归与当前状态	DIRECTED, NEGATIVE；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. Generator/independent checker not implemented; reference or unit harness required. 2. 锁定本用例的 vPlan/source/capability/schema 及独立 checker 版本；所需具体 target/recipe 未闭合则 BLOCKED。 3. 以下数值是项目测试预算或 fixture，不是 NVIDIA 物理参数或本轮实际执行记录。
参数 1	registry_fixture_kind = SYNTHETIC_BIN_REGISTRY
参数 2	bin_states = ["ELIGIBLE","ILLEGAL","DISABLED","UNRESOLVED","PROVED_UNREACHABLE","REVIEWED_WAIVER"]
参数 3	cross_axes = ["fixture_form","fixture_profile","value_class"]
参数 4	waiver_fields = ["rule","evidence","review","version"]
参数 5	UNRESOLVED_blocks_closure = true
初始化	1. 建立不冒充 PTX 合法形式的有限 fixture bin 表，记录 baseline 分母和 hash。 2. 实际 target/form registry 若未冻结则相应闭合 gate 未满足，不从 217 节数估算 tuple 数量。
执行步骤	1. 枚举六种 bin 状态及各状态合法/非法转换，逐项计算 eligible 集合。 2. 对每个排除项要求独立 rule/evidence，UNRESOLVED 仍阻止关闭而不是静默删去。 3. 用缺证据 waiver、source hash 变化和非法 cross 作单因素 mutation。 4. 比较 registry 版本/hash、可达性证明、mandatory 目标与审计预期，导出未闭合原因。
观察点	1. ILLEGAL 不属于正向分母；DISABLED/WAIVERED 按预先冻结政策显式记录，不等同已覆盖。 2. 未知 rule 或 UNSAT 频繁不能被当不可达证明。
通过条件	1. 分母集合/hash 与独立 registry 一致，缺证据或未解析项按策略 BLOCKED，不能用低百分比掩盖。
失败条件	1. 删除未命中 bin 以凑 100%；未经评审 waiver；把全部格式做笛卡尔积当合法 cross。
预期产物	1. bin_registry.json 2. denominator_audit.json 3. waiver_review.json 4. manifest.json

TC-COV-003：覆盖合并与零用例

TC-COV-003 项目	执行内容
源与阶段	SIG-COV-003；P2_REFERENCE_PROGRAM；CV-COV-003
配置与 seed	TP-REF-BASE；fixed_seed=0x534d005c（1397555292）；REFERENCE_BOUNDED
回归与当前状态	DIRECTED, NEGATIVE, REPLAY；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. Generator/independent checker not implemented; reference or unit harness required. 2. 锁定本用例的 vPlan/source/capability/schema 及独立 checker 版本；所需具体 target/recipe 未闭合则 BLOCKED。 3. 以下数值是项目测试预算或 fixture，不是 NVIDIA 物理参数或本轮实际执行记录。
参数 1	reference_log_origin = SYNTHETIC_REFERENCE
参数 2	merge_variants = ["IDENTICAL_COMPATIBLE","DUPLICATE_INSTANCE","DIFFERENT_SOURCE_HASH","DIFFERENT_PROFILE","DIFFERENT_SCHEMA","ZERO_CHECKED","CONFLICTING_DUPLICATE_RESULT"]
参数 3	compatibility_key = ["source","capability","profile","schema","backend","bin_registry"]
参数 4	logical_instances = 8
初始化	1. 物化 8 个固定 reference 事件实例以及独立 expected aggregate；不访问真实 DUT 数据库。 2. 每条 log 附完整 compatibility key 和稳定 instance/phase/fragment 身份。
执行步骤	1. 合并同 key 正常记录与重复相同实例，独立重算唯一 checked 集合。 2. 分别变异 source/profile/schema/bin registry 并请求合并，检查拒绝的具体兼容键。 3. 输入空日志、只有 complete 无 check、以及同 identity 不同结果的冲突记录。 4. 重放合并过程并核对 aggregate hash、去重及状态：无实际 checked 不能 PASS。
观察点	1. 重复回放不重复增加实例计数；冲突 duplicate 是证据完整性失败而非静默覆盖。 2. 不兼容数据库保持分开，零 checked 归类 NO_TEST/NOT_RUN 而非功能成功。
通过条件	1. 所有 merge/拒绝/去重和状态分类与独立事件集合一致，fixture 只计参考层。
失败条件	1. 无条件合并跨 profile/source 覆盖；重复膨胀；空结果 PASS；把冲突当正常重发。
预期产物	1. reference_logs.jsonl 2. merge_expected.json 3. merge_report.json 4. manifest.json

TC-COV-004：SM 事件采样一致

TC-COV-004 项目	执行内容
源与阶段	SIG-COV-004；P3_SM_RTL；CV-COV-003
配置与 seed	TP-SM-BASE；fixed_seed=0x534d005d（1397555293）；SM_PROGRESS
回归与当前状态	DIRECTED, STRESS；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. Generator/independent checker not implemented; SM RTL and corresponding service/adapter required. 2. 锁定本用例的 vPlan/source/capability/schema 及独立 checker 版本；所需具体 target/recipe 未闭合则 BLOCKED。 3. 以下数值是项目测试预算或 fixture，不是 NVIDIA 物理参数或本轮实际执行记录。
参数 1	program_recipe = 20 accepted dynamic instructions plus 4 preflight-rejected candidates, conditional on frozen legal recipe
参数 2	event_identity = ["epoch","owner","warp_thread_or_collective","instruction_instance","fragment","completion_kind"]
参数 3	sidecopy_mutations = ["DROP_ADMIT","DUPLICATE_FULL","A_READ_MISLABELED_FULL","DROP_CHECK_RESULT"]
参数 4	mutated_evidence_origin = CORRUPTED_SIDECOPY_ONLY
初始化	1. 锁定可实现的合法 recipe 与真实 event monitors；recipe 不能落实则 BLOCKED。 2. 初始化真实 run 日志和隔离的损坏侧拷贝目录，禁止侧拷贝回流真实 coverage 库。
执行步骤	1. 生成/独立审计候选后执行获准程序，收集 driver、SM admit、typed partial/full 及 checker 事件。 2. 按稳定 identity 逐项关联，检查真实 checked 集合不超过对应已完成且已验证集合。 3. 从真实日志复制只读副本，分别注入丢 admit/重复 full/Aread 误标/丢 checker 结果并要求 correlator 拒绝。 4. 比较原始真实 run 与独立 counter 结果，保存损坏副本失败原因，原始 coverage 不被 mutation 改写。
观察点	1. 只能真实 admit/complete/check 事件填 DUT 覆盖，Aread 与 fragment 不替 full。 2. sidecopy 故障验证 correlator 本身，不算 DUT 执行失败或新的 coverage。
通过条件	1. 原始 run 事件与独立 counter 一致，四类损坏侧拷贝被定位拒绝，真实数据库无污染。
失败条件	1. preflight reject 记 admit；complete 自动记 checked；Aread 误作 full；侧拷贝数据混入实际结果。

TC-COV-004 项目	执行内容
预期产物	1. actual_driver_SM_checker_events.jsonl 2. identity_correlation.json 3. coverage_recompute.json 4. corrupted_sidecopy_diagnostics.json 5. manifest.json

20.23 GEN-REPLAY-01：Seed、manifest 与确定性回放

TC-REPLAY-001：相同 manifest 重放

TC-REPLAY-001 项目	执行内容
源与阶段	SIG-REPLAY-001；P1_GENERATOR_UNIT；CV-REPLAY-001
配置与 seed	TP-GEN-BASE；fixed_seed=0x534d005e（1397555294）；GEN_BOUNDED
回归与当前状态	SMOKE, DIRECTED, REPLAY；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. 发生器/对应 unit harness 和独立审计器已实现；当前缺失，因此本 case 为 NOT_RUN/BLOCKED_MISSING_IMPLEMENTATION。 2. 锁定 PTX9.4 form/target 证据、项目 capability、IR/schema 及数值 profile；缺规则的 variant 为 BLOCKED 而非 PASS。 3. 独立 checker 不调用 generator 或 DUT 的同一合法性、decode、rounding helper；共享常量及独立边界已评审。
参数 1	parameter_status = 下列规模/调度值是待冻结的测试预算建议，不是 SM 时延、native 编码或当前运行结果。
参数 2	seed_strategy = 固定 seed 由主 testplan 赋值；RNG 子流和所有抽样结果写入完整 manifest，边界枚举不依赖随机偶然命中。
参数 3	generation_repetitions = 2
参数 4	artifact_categories = ["semantic_IR","optional_PTX_or_frozen_backend","input_data","object_layout","launch","service_schedule","rbits"]
参数 5	comparison = 规范化的确定性 artifact 字节 hash；日志时间戳/输出绝对路径等非语义字段须显式排除或规范化。
参数 6	substreams = ["opcode","operand","control","resource","timing","fault"]

TC-REPLAY-001 项目	执行内容
初始化	1. manifest 完整冻结 source/capability/profile/schema/generator/validator/tool/RNG 与所有输入；native 未定则只走 typed IR。 2. 两次生成从相同初始 RNG 状态开始，独立 hash 工具不调用 generator 比较函数。
执行步骤	1. 保存完整 manifest 并预检必需字段、输入文件和版本；生成第一次物化制品。 2. 在清洁的独立进程/工作目录重置相同子流状态，再生成第二次制品。 3. 独立 canonicalizer/hash 逐类比较代码/data/layout/launch/schedule/rbits 及子流消耗计数。 4. 记录差异与规范化规则，保留两份制品；不把 DOCX/PDF 或 wall-clock 日志二进制当刺激确定性对象。
观察点	1. 语义字段未被时间戳/路径排除规则误删；两份可检查 AST 和数据字节一致。 2. 相同 manifest 不能隐藏新 random 调用、未记录 rbits 或工具版本变更。
通过条件	1. 所有声明确定性制品一致，至少有非空程序与数据、子流使用记录。 2. 只关闭发生器制品确定性检查，不声称 SM 重跑或 DUT 结果复现。
失败条件	1. 同 manifest 刺激不同、RNG 状态未重置或只比 seed 而漏代码/data/schedule。 2. 规范化掩盖语义差异，缺 backend 却伪造 native bytes。
预期产物	1. 完整 manifest 与 source/target/capability/profile/tool/IR hash（计划产物） 2. manifest_run1.json 3. manifest_run2.json 4. deterministic_artifact_hashes.json 5. canonical_semantic_diffs.json

TC-REPLAY-002：并发顺序扰动

TC-REPLAY-002 项目	执行内容
源与阶段	SIG-REPLAY-002；P1_GENERATOR_UNIT；CV-REPLAY-002
配置与 seed	TP-GEN-BASE；fixed_seed=0x534d005f（1397555295）；GEN_BOUNDED
回归与当前状态	DIRECTED, RANDOM, STRESS, REPLAY；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. 发生器/对应 unit harness 和独立审计器已实现；当前缺失，因此本 case 为 NOT_RUN/BLOCKED_MISSING_IMPLEMENTATION。 2. 锁定 PTX9.4 form/target 证据、项目 capability、IR/schema 及数值 profile；缺规则的 variant 为 BLOCKED 而非 PASS。 3. 独立 checker 不调用 generator 或 DUT 的同一合法性、decode、rounding helper；共享常量及独立边界已评审。

TC-REPLAY-002 项目	执行内容
参数 1	parameter_status = 下列规模/调度值是待冻结的测试预算建议，不是 SM 时延、native 编码或当前运行结果。
参数 2	seed_strategy = 固定 seed 由主 testplan 赋值；RNG 子流和所有抽样结果写入完整 manifest，边界枚举不依赖随机偶然命中。
参数 3	worker_count_candidates = [1,2,4]
参数 4	case_orders = ["stable_forward","stable_reverse","shuffled_assignment"]
参数 5	campaign_case_count = 16
参数 6	variants = ["same_manifest_worker_order_only","new_manifest_opcode_weight_mutation"]
参数 7	weight_change_expectation = 改权重是新配置；只比较声明独立的稳定 draw-key/子流，不要求变化程序仍整体字节相同。
初始化	1. case ID 与子流派生键固定，不用 worker 号/文件遍历顺序/wall-clock 作为隐含 seed。 2. 拒绝重试日志与每个稳定 draw-key 可观测；变权重 variant 单列 manifest，不冒充相同配置。
执行步骤	1. 以固定 case 集合和 manifest 先在单 worker 生成基线，保存逐 case 物化 hash。 2. 仅改变 worker 数量和任务分配次序，逐 case 独立 hash 比对；随机调度顺序不得进入其刺激输入。 3. 另建只改 opcode 权重的新 manifest，审计影响传播与不相关子流在相同稳定 draw-key 上的值。 4. 保存 attempt/reject/emit 和子流计数，解释因程序结构改变产生的合法制品差异。
观察点	1. worker 扰动 variant 的同 case 刺激保持一致，case 归属不导致 seed 重派生。 2. 权重变更不会偷偷重置/共享 timing/fault 子流；非独立数据因果差异有记录。
通过条件	1. 并行/顺序生成满足声明确定性；新权重配置的差异符合记录的依赖范围。 2. CV-REPLAY-002 在独立 generator 侧 audit CHECKED 采样，不写 DUT 事件。
失败条件	1. worker 顺序改变刺激、未记录重试影响后续 RNG、错误要求新配置全程序 hash 不变。 2. 通过改 seed 规避差异，或把不相关 stream 共享状态而不报告。
预期产物	1. 完整 manifest 与 source/target/capability/profile/tool/IR hash（计划产物） 2. case_assignment_and_stream_keys.json 3. per_worker_artifact_digests.json 4. retry_rng_consumption_log.jsonl 5. weight_mutation_impact_report.json

TC-REPLAY-003：缺项与污染检测

TC-REPLAY-003 项目	执行内容
源与阶段	SIG-REPLAY-003；P2_REFERENCE_PROGRAM；CV-REPLAY-003
配置与 seed	TP-REF-BASE；fixed_seed=0x534d0060（1397555296）；REFERENCE_BOUNDED
回归与当前状态	DIRECTED, NEGATIVE, REPLAY；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. 发生器/对应 unit harness 和独立审计器已实现；当前缺失，因此本 case 为 NOT_RUN/BLOCKED_MISSING_IMPLEMENTATION。 2. 锁定 PTX9.4 form/target 证据、项目 capability、IR/schema 及数值 profile；缺规则的 variant 为 BLOCKED 而非 PASS。 3. 独立 checker 不调用 generator 或 DUT 的同一合法性、decode、rounding helper；共享常量及独立边界已评审。 4. 参考程序解释/编译路线及独立 oracle 均存在并版本锁定；结果只作 P2 证据，不填 SM RTL 覆盖。
参数 1	parameter_status = 下列规模/调度值是待冻结的测试预算建议，不是 SM 时延、native 编码或当前运行结果。
参数 2	seed_strategy = 固定 seed 由主 testplan 赋值；RNG 子流和所有抽样结果写入完整 manifest，边界枚举不依赖随机偶然命中。
参数 3	mutation_count_per_variant = 1
参数 4	manifest_fields = ["target","capability_hash","profile","IR_schema","oracle_version","tool_hash","service_schedule","rbits"]
参数 5	artifact_mutations = ["delete_required_file","flip_one_data_bit","change_IR_semantic_field","change_backend_hash_if_available"]
参数 6	native_rule = 未冻结/缺失 backend 的 native variant 显式 BLOCKED，只在实际存在的 IR/backend 制品上检查污染。
初始化	1. 存在完整非空基线 replay bundle 及可执行 P2 参考路线，独立 preflight 可核对版本和 hash。 2. 基线包只读保留，每个 negative variant 修改其副本，禁止污染原件。

TC-REPLAY-003 项目	执行内容
执行步骤	1. 先独立确认完整基线包能够在 P2 参考路线重放并取得预期检查结果。 2. 逐个删除必需字段/文件或改变一项数据、target、工具 hash、rbits，记录精确 mutation。 3. 独立 replay preflight 检查完整性/兼容性/哈希，按预期层拒绝或标 BLOCKED；不得只接受相同 seed。 4. 确认污染包未被当正常可复现证据执行/合并，恢复基线再检其仍可接受。
观察点	1. 每个拒绝对应被污染的字段/文件，而非缺工具等无关失败。 2. rbits、服务 schedule 及 oracle 版本均受完整性保护，不只是程序文本 hash。
通过条件	1. 所有单因素污染被独立发现并正确分类，完整基线保持可接受。 2. 无可用 backend 的 variant 为 blocked 而非伪造拒绝成功，原件保持原 hash。
失败条件	1. 被污染包仅凭 seed 被接纳、静默修补字段、任意非零退出都算预期拒绝。 2. 原始 bundle 被覆盖，或污染结果混进真实数值/覆盖证据。
预期产物	1. 完整 manifest 与 source/target/capability/profile/tool/IR hash（计划产物） 2. readonly_baseline_bundle_manifest.json 3. bundle_mutation_index.json 4. preflight_rule_results.json 5. baseline_integrity_recheck.json

TC-REPLAY-004：失败 signature 回放

TC-REPLAY-004 项目	执行内容
源与阶段	SIG-REPLAY-004；P2_REFERENCE_PROGRAM；CV-REPLAY-003
配置与 seed	TP-REF-BASE；fixed_seed=0x534d0061（1397555297）；REFERENCE_BOUNDED
回归与当前状态	DIRECTED, NEGATIVE, REPLAY；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. 发生器/对应 unit harness 和独立审计器已实现；当前缺失，因此本 case 为 NOT_RUN/BLOCKED_MISSING_IMPLEMENTATION。 2. 锁定 PTX9.4 form/target 证据、项目 capability、IR/schema 及数值 profile；缺规则的 variant 为 BLOCKED 而非 PASS。 3. 独立 checker 不调用 generator 或 DUT 的同一合法性、decode、rounding helper；共享常量及独立边界已评审。 4. 参考程序解释/编译路线及独立 oracle 均存在并版本锁定；结果只作 P2 证据，不填 SM RTL 覆盖。
参数 1	parameter_status = 下列规模/调度值是待冻结的测试预算建议，不是 SM 时延、native 编码或当前运行结果。

TC-REPLAY-004 项目	执行内容
参数 2	seed_strategy = 固定 seed 由主 testplan 赋值；RNG 子流和所有抽样结果写入完整 manifest，边界枚举不依赖随机偶然命中。
参数 3	repetitions = 3
参数 4	fixture_options = ["versioned_reference_component_failure","explicit_SYNTHETIC_observed_trace_corruption_for_checker_unit"]
参数 5	signature_fields = ["stage","rule_id","stable_source_operation","first_error_kind","normalized_expected_observed_relation"]
参数 6	schedule = 重放物化事件与稳定操作 ID；不能仅重新抽取同 seed 的延迟。
初始化	1. 存在合法非空基线程序、独立 oracle 与可重放失败 fixture；SYNTHETIC fixture 明确只检 checker/replay，不是实际 DUT 失败证据。 2. 签名规范化规则先冻结，不因缩减/回放难以通过临时放宽失败谓词。
执行步骤	1. 独立审计失败包的程序合法性及 fixture 注入合同，记录原始 first failure 与完整物化 schedule。 2. 在固定 P2 backend/model 版本下重放 3 次，从实际参考/fixture 观察重新计算 signature。 3. 逐次核对同一失败判据与允许集合，不将新的 timeout、UB 或预检失败当原失败复现。 4. 若不稳定，保留所有 trace 和 schedule 并输出 NONDETERMINISTIC/INCONCLUSIVE 诊断，不换 seed 抹去首个失败。
观察点	1. 归一化保留因果操作和规则，未把不同失败类型合成同 signature。 2. 原判据仍由独立 oracle 触发，允许 NaN/调度差异未制造假数值失败。
通过条件	1. 三次满足冻结的原 signature 与失败 predicate；或测试明确检出注入的不稳定并按该 variant 合同分类。 2. 稳定/不稳定 variant 分开验收，不将 INCONCLUSIVE 结果写作真实 DUT 回放 PASS。
失败条件	1. 发生新错误却称原故障复现、忽略 schedule/版本、换 seed 直到通过。 2. SYNTHETIC 失败日志被当作 SM 执行错误证据或合法集合内差异被当 mismatch。
预期产物	1. 完整 manifest 与 source/target/capability/profile/tool/IR hash（计划产物） 2. original_failure_bundle_index.json 3. replay_attempts_and_signatures.jsonl 4. independent_failure_predicate_report.json 5. nondeterminism_diagnostic_if_needed.json

20.24 GEN-REDUCE-01：保持语义与资源关系的失败缩减

TC-REDUCE-001：定义与 CFG 缩减

TC-REDUCE-001 项目	执行内容
源与阶段	SIG-REDUCE-001；P1_GENERATOR_UNIT；CV-REDUCE-001
配置与 seed	TP-GEN-BASE；fixed_seed=0x534d0062（1397555298）；GEN_BOUNDED
回归与当前状态	SMOKE, DIRECTED, NEGATIVE, REPLAY；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. 发生器/对应 unit harness 和独立审计器已实现；当前缺失，因此本 case 为 NOT_RUN/BLOCKED_MISSING_IMPLEMENTATION。 2. 锁定 PTX9.4 form/target 证据、项目 capability、IR/schema 及数值 profile；缺规则的 variant 为 BLOCKED 而非 PASS。 3. 独立 checker 不调用 generator 或 DUT 的同一合法性、decode、rounding helper；共享常量及独立边界已评审。
参数 1	parameter_status = 下列规模/调度值是待冻结的测试预算建议，不是 SM 时延、native 编码或当前运行结果。
参数 2	seed_strategy = 固定 seed 由主 testplan 赋值；RNG 子流和所有抽样结果写入完整 manifest，边界枚举不依赖随机偶然命中。
参数 3	starting_program_instruction_counts = [32,128]
参数 4	candidate_transformations = ["delete_independent_instruction","delete_producer","remove_branch_arm","replace_call","reduce_loop_count"]
参数 5	maximum_candidate_evaluations = 256
参数 6	failure_fixture = 明确 SYNTHETIC 的稳定合法程序 predicate 用于 shrinker unit；不构造假 DUT 运行结果。
初始化	1. 原始合法程序与 fixture predicate 可复现并只读保留，独立 CFG/def-use 审计已通过。 2. shrinker 与独立合法性 checker 可分别运行；候选不能仅继承原 case 的 legal 标记。

TC-REDUCE-001 项目	执行内容
执行步骤	1. 建立含失败相关链、无关链、分支与 call 的合法基线，记录 stable source ID 和合成 predicate。 2. 逐项提出删除/替换/loop 缩短候选，每次从候选 IR 重跑独立 CFG、定义、参与者与生命周期审计。 3. 拒绝缺 producer/非法目标/缺成员候选；合法候选再判原 fixture predicate，不因合法就自动接受。 4. 在预算内保留合法且仍满足原 predicate 的缩减包，同时保留所有接受/拒绝变换日志。
观察点	1. 删除支配定义、return 目标或必要成员时被独立 checker 发现。 2. 合法删除无关链不改变原 predicate，guard/CC 定义与 cleanup 仍完整。
通过条件	1. 每个接受候选均重新合法化且保留冻结 predicate，非法候选拒绝有具体理由。 2. 有限评估内终止，原始包未覆盖；unit 合成 predicate 不计 DUT 故障或覆盖。
失败条件	1. 缩成未初始化读/非法跳转却接纳，复用原 legal 标记或临时弱化 predicate。 2. 只保持同样 timeout 文本而改变失败原因，或删除原始证据。
预期产物	1. 完整 manifest 与 source/target/capability/profile/tool/IR hash（计划产物） 2. original_SYNTHETIC_unit_fixture.json 3. candidate_transform_log.jsonl 4. candidate_audit_results.jsonl 5. reduced_IR_and_stable_ID_map.json

TC-REDUCE-002：异步资源缩减

TC-REDUCE-002 项目	执行内容
源与阶段	SIG-REDUCE-002；P1_GENERATOR_UNIT；CV-REDUCE-002
配置与 seed	TP-GEN-BASE；fixed_seed=0x534d0063（1397555299）；GEN_BOUNDED
回归与当前状态	DIRECTED, NEGATIVE, REPLAY；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. 发生器/对应 unit harness 和独立审计器已实现；当前缺失，因此本 case 为 NOT_RUN/BLOCKED_MISSING_IMPLEMENTATION。 2. 锁定 PTX9.4 form/target 证据、项目 capability、IR/schema 及数值 profile；缺规则的 variant 为 BLOCKED 而非 PASS。 3. 独立 checker 不调用 generator 或 DUT 的同一合法性、decode、rounding helper；共享常量及独立边界已评审。
参数 1	parameter_status = 下列规模/调度值是待冻结的测试预算建议，不是 SM 时延、native 编码或当前运行结果。

TC-REDUCE-002 项目	执行内容
参数 2	seed_strategy = 固定 seed 由主 testplan 赋值；RNG 子流和所有抽样结果写入完整 manifest，边界枚举不依赖随机偶然命中。
参数 3	template_candidates = ["classic_group","bulk_mbarrier","TMEM_alloc_use_wait_dealloc","group2_peer_lifecycle","Fabric_submit_primary_report"]
参数 4	deletion_candidates = ["alloc","expect_tx","commit","wait","dealloc","peer_participant","submit"]
参数 5	maximum_candidate_evaluations = 256
参数 6	profile_boundary = 仅经审查的 audit-only recipe/profile 做 P1 合法性单测；不因该模板存在而开启真实 DUT 能力。
初始化	1. 每个基线的 typed obligations、HB、参与者、计数单位、phase 和 cleanup 均由独立审计确认。 2. 未实现/原文未闭合 recipe 的 variant 为 BLOCKED；失败 fixture 若合成则明确 NON_DUT。
执行步骤	1. 选择一条完整合法 async 资源模板，冻结原始对象/事件/依赖图与原 failure predicate。 2. 分别提出删除 alloc/expect/commit/wait/free/peer/submit 的单项候选，不自动跨 family 补 wait。 3. 独立 resource/HB/全路径 checker 重算 typed count、对象存活和进度前置条件；不使用预测完成周期释放资源。 4. 只让合法且原 predicate 仍成立的候选进入结果，记录 UB/新死锁/不同错误的拒绝原因。
观察点	1. 缺 wait/free/expect 或 peer 造成的 use-after-free、过早 phase 完成和孤儿责任被检出。 2. bytes/arrivals/Fabric16B、source-read/full 与 allocation/permit 不被共用无类型计数。
通过条件	1. 所有接受候选保持原模板合法性与失败 predicate，无新 UB/新超时顶替原故障。 2. 每次候选的独立 HB/obligation 结果保存，原包完整，P1 不声称异步 DUT 执行。
失败条件	1. 删除 peer 或同步后仍接纳、用另一家族 wait 掩盖缺口、将发射当完成释放。 2. 未知 recipe 被假定合法，或合成模型日志被算为真实 SM/TMA/Fabric 覆盖。
预期产物	1. 完整 manifest 与 source/target/capability/profile/tool/IR hash（计划产物） 2. original_async_recipe_fixture.json 3. typed_obligation_HB_graphs.json 4. lifecycle_deletion_audits.jsonl 5. accepted_candidate_predicate_checks.json

TC-REDUCE-003：数值与调度缩减

TC-REDUCE-003 项目	执行内容
源与阶段	SIG-REDUCE-003；P2_REFERENCE_PROGRAM；CV-REDUCE-003
配置与 seed	TP-REF-BASE；fixed_seed=0x534d0064（1397555300）；REFERENCE_BOUNDED
回归与当前状态	DIRECTED, RANDOM, REPLAY；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. 发生器/对应 unit harness 和独立审计器已实现；当前缺失，因此本 case 为 NOT_RUN/BLOCKED_MISSING_IMPLEMENTATION。 2. 锁定 PTX9.4 form/target 证据、项目 capability、IR/schema 及数值 profile；缺规则的 variant 为 BLOCKED 而非 PASS。 3. 独立 checker 不调用 generator 或 DUT 的同一合法性、decode、rounding helper；共享常量及独立边界已评审。 4. 参考程序解释/编译路线及独立 oracle 均存在并版本锁定；结果只作 P2 证据，不填 SM RTL 覆盖。
参数 1	parameter_status = 下列规模/调度值是待冻结的测试预算建议，不是 SM 时延、native 编码或当前运行结果。
参数 2	seed_strategy = 固定 seed 由主 testplan 赋值；RNG 子流和所有抽样结果写入完整 manifest，边界枚举不依赖随机偶然命中。
参数 3	transform_dimensions = ["FP_operand_bits","scale_value","metadata_entries","rbits","service_event_delays"]
参数 4	maximum_candidate_evaluations = 512
参数 5	preserved_fields = ["target_profile","oracle_version","rule_id","stable_failure_operation","typed_count_units","report_contract"]
参数 6	predicate_policy = 原包先冻结误差/bit/allowed-outcome 失败谓词与签名规范化；不能为接受候选临时放宽。
初始化	1. 原始失败在固定 P2 参考路线可复现，独立 oracle 与合法性已确认，原始包只读。 2. 选定 scale/metadata/rbits 模板实际有已审查格式/约束；native 缺失不冒充有执行后端。

TC-REDUCE-003 项目	执行内容
执行步骤	1. 依次降低数据复杂度和 schedule 延迟/事件数量，保持 program/对象/参与者与故障注入合同。 2. 每个候选独立复审 tuple/CFG/HB/metadata/scale 边界及 profile，重算 golden 或允许集合。 3. 执行 P2 参考/fixture 并重新判原 failure predicate，拒绝只产生新 timeout、UB 或允许 NaN 差异的候选。 4. 接受的候选保存新物化制品与完整 manifest，直至预算或局部不再缩减；原件不覆盖。
观察点	1. 缩 rbits 或 FP 值后仍保留原舍入见证，不把未规定 payload 差异改造为假 mismatch。 2. 缩 schedule 没有破坏公平/错误注入窗、report/计数或必要 source/full 顺序。
通过条件	1. 最终候选独立合法且仍触发冻结的原失败谓词，golden/allowed-set 与新输入匹配。 2. 数据/调度变换因果可追溯，不把局部最小称全局最小或实际 SM 验证。
失败条件	1. 沿用旧 golden、变化 profile、把允许结果当错值或以新 UB/死锁冒充原故障。 2. schedule 缩减撤销了原故障却通过弱化签名接纳。
预期产物	1. 完整 manifest 与 source/target/capability/profile/tool/IR hash（计划产物） 2. numeric_schedule_candidate_log.jsonl 3. fresh_candidate_oracles.json 4. same_failure_witness_checks.json 5. original_and_reduced_bundle_indices.json

TC-REDUCE-004：最小化可复现包

TC-REDUCE-004 项目	执行内容
源与阶段	SIG-REDUCE-004；P2_REFERENCE_PROGRAM；CV-REDUCE-003
配置与 seed	TP-REF-BASE；fixed_seed=0x534d0065（1397555301）；REFERENCE_BOUNDED
回归与当前状态	DIRECTED, REPLAY；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. 发生器/对应 unit harness 和独立审计器已实现；当前缺失，因此本 case 为 NOT_RUN/BLOCKED_MISSING_IMPLEMENTATION。 2. 锁定 PTX9.4 form/target 证据、项目 capability、IR/schema 及数值 profile；缺规则的 variant 为 BLOCKED 而非 PASS。 3. 独立 checker 不调用 generator 或 DUT 的同一合法性、decode、rounding helper；共享常量及独立边界已评审。 4. 参考程序解释/编译路线及独立 oracle 均存在并版本锁定；结果只作 P2 证据，不填 SM RTL 覆盖。

TC-REDUCE-004 项目	执行内容
参数 1	parameter_status = 下列规模/调度值是待冻结的测试预算建议，不是 SM 时延、native 编码或当前运行结果。
参数 2	seed_strategy = 固定 seed 由主 testplan 赋值；RNG 子流和所有抽样结果写入完整 manifest，边界枚举不依赖随机偶然命中。
参数 3	candidate_evaluation_caps = [64,256,1024]
参数 4	wall_time_budget_seconds_proposed = 120
参数 5	final_bundle_replay_repetitions = 2
参数 6	terminal_reasons = ["no_more_accepted_reduction","candidate_budget","wall_time_budget","nondeterministic_or_blocked"]
初始化	1. 存在可复现 P2 失败包、独立 audit/oracle 及已评审 shrinker 变换；停止/错误分类规则冻结。 2. 原始包、每次候选和最后合法见证分别保存；中止时不得只留下部分损坏包。
执行步骤	1. 从原始只读失败包运行有界缩减，逐候选重新独立审计与原 predicate 检查。 2. 在无可接受变换、次数或时间上限时保存最后一个合法仍失败候选及精确停止原因。 3. 用独立 replay 入口/参考路线在清洁目录重放最终物化包 2 次，校验全部 hash、oracle 与同一失败 signature。 4. 归档原始/缩减 manifest、变换/拒绝日志、未能缩减的依赖和新旧体积，明确仅局部最小。
观察点	1. 预算耗尽仍产出完整最后合法包，未把尚未验证 candidate 当最终见证。 2. 最终两次重放满足原 predicate；不稳定则保留并分类，不能当稳定复现通过。
通过条件	1. 在配置预算内终止，输出包独立合法、完整且可重复触发同一 P2 失败判据。 2. 原始证据保留，停止/缩减程度报告真实，不宣称数学全局最小或 DUT 故障已修复。
失败条件	1. 无限缩减、最终包缺文件/hash、最后候选未复审，或重放成新 timeout/UB 仍称原故障。 2. 超时/缺工具被写为 PASS，原始包被覆盖或丢失失败因果。
预期产物	1. 完整 manifest 与 source/target/capability/profile/tool/IR hash（计划产物） 2. original_bundle_manifest.json 3. reduced_bundle_manifest.json 4. transform_and_rejection_history.jsonl 5. final_independent_replay_results.json 6. bounded_stop_reason.json

20.25 GEN-REGRESS-01：分层回归、分布质量与退出标准

TC-REGRESS-001：计划/状态完整性

TC-REGRESS-001 项目	执行内容
源与阶段	SIG-REGRESS-001；P1_GENERATOR_UNIT；CV-REGRESS-001
配置与 seed	TP-GEN-BASE；fixed_seed=0x534d0066（1397555302）；GEN_BOUNDED
回归与当前状态	SMOKE, DIRECTED, NEGATIVE；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. Generator/independent checker not implemented; reference or unit harness required. 2. 锁定本用例的 vPlan/source/capability/schema 及独立 checker 版本；所需具体 target/recipe 未闭合则 BLOCKED。 3. 以下数值是项目测试预算或 fixture，不是 NVIDIA 物理参数或本轮实际执行记录。
参数 1	planned_source_counts = {"features":25,"testpoints":106,"coverage_points":75,"instruction_topics":217,"special_register_topics":35}
参数 2	unit_fixture_mutations = ["SOURCE_HASH","REVERSE_LINK","UNKNOWN_CV","FALSE_EXECUTION_CLAIM"]
参数 3	case_scope = future P1 pipeline-unit test; current standalone P0 tool invocation is reported separately
初始化	1. 冻结 vPlan 与本 testplan 源 hash，准备 future generator/unit pipeline 的未执行结果目录。 2. 所有 106 源 case 初始状态 NOT_RUN，不把独立 P0 校验的 PASS 写回 case 执行状态。
执行步骤	1. 通过 P1 unit harness 载入完整 source→TC→suite 映射，调用独立 schema/association 审计接口。 2. 对正常 fixture 验证计数、双向关联、scope 和 source hash。 3. 逐一注入四种 metadata 故障并检查具体拒绝诊断，再恢复 fixture。 4. 核对未来 unit 结果与独立 P0 报告分离、动态执行数的来源和 manifest 绑定。
观察点	1. 缺映射/未知 CV/错 hash/伪造执行均不能静默接受。 2. 当前只运行 check_testplan.py 不等同本 TC 已执行。
通过条件	1. P1 实际实施后正常 fixture 通过、故障 fixture 对应拒绝且状态/证据分类准确。
失败条件	1. 将静态 P0 PASS 当 106 动态 case 完成；反向关联丢失；源变化仍合并旧覆盖。

TC-REGRESS-001 项目	执行内容
预期产物	1. pipeline_unit_association_report.json 2. metadata_mutation_diagnostics.json 3. manifest.json

TC-REGRESS-002：随机分布与稀有事件

TC-REGRESS-002 项目	执行内容
源与阶段	SIG-REGRESS-002；P1_GENERATOR_UNIT；CV-REGRESS-002
配置与 seed	TP-GEN-BASE；fixed_seed=0x534d0067（1397555303）；GEN_BOUNDED
回归与当前状态	DIRECTED, NEGATIVE, RANDOM, STRESS；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. Generator/independent checker not implemented; reference or unit harness required. 2. 锁定本用例的 vPlan/source/capability/schema 及独立 checker 版本；所需具体 target/recipe 未闭合则 BLOCKED。 3. 以下数值是项目测试预算或 fixture，不是 NVIDIA 物理参数或本轮实际执行记录。
参数 1	draw_counts_per_feasible_state = [1024,4096]
参数 2	weight_sets = [[1,1,1],[0,1,7],[0,0,0]]
参数 3	rare_form_min_quota = 4
参数 4	max_constraint_attempts_per_candidate = 256
参数 5	statistical_policy = {"alpha":0.001,"minimum_expected_bin_count":5,"multiple_testing_correction":"Bonferroni if the chosen statistical test assumptions hold; otherwise predeclare exact/merged-bin method"}
参数 6	throughput_ratio_used_as_sampling_weight = false
初始化	1. 构造有限可独立重算的可行状态，明确合法项、关闭项与 UNSAT 原因。 2. 锁定 draw count、统计方法、RNG 子流和 seed；不得看到结果后调阈值。
执行步骤	1. 分别运行等权、含零权重和全零权重配置；全零应产生有界诊断，不无限重试。 2. 为每次 attempt 记录条件可行集合、拒绝理由和最终 emitted/checked 分类。 3. 在两个样本量下按预先冻结方法审计条件分布，同时用定向 quota 保证 rare 合法项命中。 4. 触发最大 constraint attempts 并验证可解释 UNSAT/fallback 策略与可回放日志，不以 Rubin 峰值比例作为权重。

TC-REGRESS-002 项目	执行内容
观察点	1. 统计分布按条件可行集合比较，不能要求有限样本 histogram 精确等于权重。 2. 零权重功能不被随机选中；rare 定向 quota 与随机概率分开计。
通过条件	1. 分布/配额满足预声明判据，全零/UNSAT 有限终止，配置和拒绝偏移可解释。
失败条件	1. 事后改 alpha/seed 直到通过；合法稀有项永久饥饿；全零挂死；静默取消原定 hazard。
预期产物	1. sampling_attempts.jsonl 2. conditional_histograms.json 3. statistical_method_and_result.json 4. quota_UNSAT_report.json 5. manifest.json

TC-REGRESS-003：分层失败与退出

TC-REGRESS-003 项目	执行内容
源与阶段	SIG-REGRESS-003；P2_REFERENCE_PROGRAM；CV-REGRESS-003
配置与 seed	TP-REF-BASE；fixed_seed=0x534d0068（1397555304）；REFERENCE_BOUNDED
回归与当前状态	DIRECTED, NEGATIVE；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. Generator/independent checker not implemented; reference or unit harness required. 2. 锁定本用例的 vPlan/source/capability/schema 及独立 checker 版本；所需具体 target/recipe 未闭合则 BLOCKED。 3. 以下数值是项目测试预算或 fixture，不是 NVIDIA 物理参数或本轮实际执行记录。
参数 1	status_fixture_kind = SYNTHETIC_CAMPAIGN
参数 2	fixtures = ["MISSING_TOOL","NO_MODEL","ZERO_CASE","ZERO_CHECKED","UNCLOSED_BINS","UNREVIEWED_WAIVER","OLD_MALU_EVIDENCE_ONLY","VALID_CHECKED_EVIDENCE"]
参数 3	PASS_requires_nonempty_checked = true
参数 4	dependency_closed_never_silently_skipped = true
初始化	1. 初始化隔离的 reference campaign 账本，所有 fixture 明确非真实执行。 2. VALID_CHECKED_EVIDENCE 是合成状态机测试输入，不可用作 SM 功能验收证明。

TC-REGRESS-003 项目	执行内容
执行步骤	1. 依次向独立 campaign 分类器提供八种 fixture，并记录工具/gate、checked、coverage 和 waiver 字段。 2. 核对 missing dependency 为 BLOCKED、未发起为 NOT_RUN、真实错误为 FAIL，而非统归 PASS 或 skip。 3. 检查零 checked、未闭合 mandatory 和旧 MALU 证据不能满足 generator/SM 退出条件。 4. 将 fixture 结论与预声明 truth table 比对，导出错误路径/状态转换及所有源 hash。
观察点	1. PASS 只能由非空且符合 scope 的实际证据成立；synthetic truth table 不写真实 run 结果。 2. 未审查 waiver、capability 未解析、缺工具保持显式未关闭状态。
通过条件	1. 状态机与 truth table 一致且无虚假 PASS、覆盖缩分母或旧证据替代。
失败条件	1. 缺工具变 PASS；零 case 通过；用 10,861 旧 MALU 结果证明本 106case 执行；静默 skip 必要项。
预期产物	1. campaign_status_truth_table.json 2. status_transition_report.json 3. evidence_scope_audit.json 4. manifest.json

TC-REGRESS-004：未来 SM 回归关闭

TC-REGRESS-004 项目	执行内容
源与阶段	SIG-REGRESS-004；P3_SM_RTL；CV-REGRESS-003
配置与 seed	TP-SM-BASE；fixed_seed=0x534d0069（1397555305）；SM_PROGRESS
回归与当前状态	DIRECTED, RANDOM, STRESS；NOT_RUN / BLOCKED_MISSING_IMPLEMENTATION
前置条件	1. Generator/independent checker not implemented; SM RTL and corresponding service/adapter required. 2. 锁定本用例的 vPlan/source/capability/schema 及独立 checker 版本；所需具体 target/recipe 未闭合则 BLOCKED。 3. 以下数值是项目测试预算或 fixture，不是 NVIDIA 物理参数或本轮实际执行记录。 4. 外层调度已完成冻结 enabled P3 叶子集合所需证据；本观察器只读这些证据，不调度自身或任意 campaign orchestrator。
参数 1	required_phase = P3_SM_RTL
参数 2	SM_case_count_in_source = 23

TC-REGRESS-004 项目	执行内容
参数 3	regression_stages = ["P1_P2_ENTRY_GATES","P3_SMOKE_RECIPE","P3_DIRECTED","P3_RANDOM","P3_STRESS","TRIAGE_CLOSURE"]
参数 4	P4_closed_separately = true
参数 5	mandatory_bins = frozen eligible P3 bins, unresolved keeps gate closed
参数 6	cleanup_may_not_be_truncated = true
参数 7	execution_role = NON_REENTRANT_CAMPAIGN_OBSERVER
参数 8	dispatches_child_tests = false
参数 9	excluded_dispatch_roles = ["CAMPAIGN_OBSERVER","CAMPAIGN_ORCHESTRATOR"]
参数 10	required_upstream_P3_case_ids = ["TC-ISA-004","TC-IR-004","TC-DEP-004","TC-ALU-004","TC-FP-004","TC-CVT-004","TC-CTRL-004","TC-COLL-004","TC-MEM-004","TC-SMEM-004","TC-SYNC-004","TC-ASYNC-004","TC-TMEM-004","TC-TMEM-005","TC-MATRIX-004","TC-MATRIX-005","TC-MATRIX-006","TC-SYS-004","TC-NEG-004","TC-ENV-004","TC-LIFE-004","TC-COV-004"]
参数 11	upstream_leaf_count = 22
参数 12	evidence_policy = Read already-completed, hash-compatible evidence for the frozen enabled P3 leaf scope. Missing required evidence => BLOCKED; selected suite alone need not close G3. Never launch a TC or invoke this observer recursively.
初始化	1. 先证明 P1/P2 前置实现和已启用 P3 capability/recipe/model 就绪，否则不启动本 case。 2. 冻结本次 P3 eligible case/bin 集合，P4 仍独立 blocked，不宣称全 SoC 关闭。
执行步骤	1. 只读加载外层调度已完成的 P3 叶子 case evidence index、capability/recipe/profile/bin registry 与 source hash；核对自身不在 upstream leaf 名单。 2. 对冻结 enabled scope 按 smoke、定向、random、stress 分类已有证据，记录缺失/blocked 项；不调用任何 TC、runner 或 campaign observer。 3. 验证实际 checker、typed completion/owner cleanup、coverage 与 triage/replay 包；证据不兼容或未结清时不关闭 G3。 4. 生成本观察器独立检查结果，仅对已满足条件的 enabled P3 范围判退出；保留 23 源 P3case 统计，22 叶子+本观察器，P4 另行关闭。

TC-REGRESS-004 项目	执行内容
观察点	1. 实际 checked 非空，stage/provenance 可追踪；timeout 有 wait graph 和公平假设分类。 2. 每个 run 结束资源闭合或有正式 abort 确认，未 triage 失败不被重跑成功覆盖。 3. dispatches_child_tests=false；只读上游 22 个 P3 叶子身份，任何 observer/orchestrator 自调用均违反合同，矩阵行顺序不等于执行依赖顺序。
通过条件	1. P3 启用范围满足冻结退出标准，证据包完整可重放，无零用例/未归类失败假通过。
失败条件	1. P3 smoke 失败仍跑压力；P4 缺服务却宣称全 SoC 通过；删除未命中 bin；缺正式收尾。
预期产物	1. SM_campaign_summary.json 2. enabled_case_bin_registry.json 3. actual_checker_and_coverage.json 4. triage_and_replay_index.json 5. manifest.json